



סמל בית ספר: בן יהודה

שם הפרויקט:

PACMAN ANDROID EDITION

שם החלופה: תכנות טלפונים חכמים אנדרואיד (Flutter)



created by
Tomer

שם התלמיד: תומר כץ

ת.ז התלמיד: 333040616

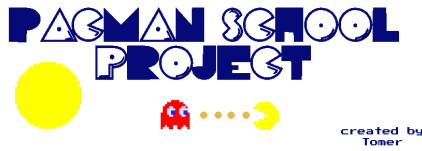
שם המנחה: שרה סולומון

תאריך הגשה: מאי, 2026

תוכן עניינים

2.....	תוכן עניינים
6.....	טבלת תכולות הפרויקט
7.....	מבוא
7.....	רקע
7.....	תהליך המחקר
7.....	אתגרים מרכזיים
7.....	חידושים
8.....	תיאור תחום הידע
8.....	אובייקטים נחוצים
8.....	סוגי נתונים
8.....	ייצוג מידע
8.....	תיאור פעולות על המצב
9.....	מבנה / ארכיטקטורה של הפרויקט
9.....	מסך הבית
11.....	תפריט עליון (App Bar)
12.....	תפריט צד (Sidebar)
14.....	מסך המשחק
16.....	מסך דירוגים
18.....	מסך בונסים
20.....	דיאלוג בונס
22.....	הגדרות
24.....	דיאלוג שינוי צבע
26.....	מסך כניסה (Login)
28.....	מסך הרשמה (Sign up)
31.....	מסך הסברים (Read Me)
32.....	תרשים מסכים
33.....	תרשים UML של הפרויקט
39.....	מימוש הפרויקט
39.....	Model
39.....	My App
40.....	Default Firebase Options
41.....	Game User
43.....	Session
44.....	Animated Character
46.....	Sprite Animation
48.....	Controller

Pacman Android Edition



תומר כץ

49	Controller State
50	Dpad Button
50	Dpad Button State
52	End Game Buttons
53	End Game Buttons State
55	Ghost
61	Ghost State
62	Blinky
64	Pinky
66	Inky
67	Clyde
70	Game Map
74	Map Pallet
75	Maze Painter
78	Bonus
80	Bonus Type
82	Direction
84	Dot
86	Game Manager
90	Audio State
91	Game Screen
95	Game Screen State
97	Game Timer
99	Pacman
101	Super Dot
103	Collectables Screen
104	Collectables Screen State
105	Bonus Card
106	Bonus Card State
108	Home Screen
109	Home Screen State
110	Leaderboard Screen
111	Leaderboard Screen State
112	Login
113	Login State
115	Settings
116	Settings State
120	Signup
121	Signup State
124	Read Me

125.....	Read Me State
126.....	AppBar
127.....	Constants
129.....	Game Displayer
130.....	Game Displayer State
132.....	Loading Screen
133.....	Loading Screen State
134.....	Sidebar
135.....	Sidebar State
136.....	Utils
137.....	main
138.....	בסיס הנתונים
138.....	Firestore Authentication
139.....	Realtime DataBase
141.....	מדריך למשתמש
141.....	PAC-MAN — OFFICIAL INSTRUCTION MANUAL
142.....	פרטי הגרסה ודרישות מערכת
143.....	מסך הכניסה ((Login
144.....	מסך ההרשמה ((Sign Up
145.....	תפריט עליון ((AppBar
145.....	תפריט צד ((Sidebar
146.....	מסך הבית ((Home Screen
147.....	מסך המשחק ((Game Screen
147.....	מפת המבוך
148.....	מידע עליון
148.....	השלט
149.....	בונוסים
149.....	חיים ובונוס חיים
150.....	הרוחות
150.....	אכילת רוחות
151.....	מסך הדירוגים ((Leaderboard
152.....	מסך הבונוסים ((Collectables
152.....	דיאלוג הבונוס
153.....	מסך ההגדרות ((Settings
153.....	דיאלוג שינוי צבע
154.....	מסך הסבר ((ReadMe
154.....	אפשר לגלול ברחבי העמוד בתוך ה Google Docs ולראות את כל המסמך
155.....	מגבלות ואילוצים
156.....	סיכום אישי ורפלקציה
157.....	ביבליוגרפיה

158	נספחים
158	מערכת האנימציה המותאמת ((Custom Sprite Engine
158	רקע והחלטה הנדסית
158	ארכיטקטורת המערכת
158	1. מחלקת Animated Character (התשתית הלוגית)
158	2. מחלקת Sprite Animation (מנוע הרינדור)
159	פיצ'רים מתקדמים במימוש
160	הסבר אלגוריתם רוחות
160	1. אלגוריתם ה-BFS (Breadth-First Search)
160	2. טכניקת ה-"First Direction" (ההחלטה המיידית)
160	3. התמודדות עם "מנהרות" ((Map Wrap-around
160	סיכום התהליך בתוך ה-Update Loop
161	דוגמא
162	קוד המקור+javadoc
395	מילון מושגים
400	מסך סיום

טבלת תכולות הפרויקט

סעיף במחווך	נושאים שמומשו	שם בFlutter	מחלקות הנושאים	הסבר
6	יצירת אנימציה, לא כולל שימוש במחלקת אנימציה!		SpriteAnimation	כתיבת מחלקת אנימציה פרטית המחליפה תמונה על פי שרון
9	בינה אנושית		complexChase - Ghost	הגעה לpacman באלגוריתם חכם (מפורט יותר בהמשך)
10	SharedPrefer ences	SharedPrefere nces	Settings	שמירה של בחירת צבעים חדשה לאלמנטים במשחק
	Count DownTimer	Timer	GameManager	מעביר כל 5 שניות את המצב של הרוחות מ - scatter ל - chase

מבוא

רקע

הפרויקט נושא את השם **Pacman School Project** ומציע למשתמש חוויית משחק של הקלאסיקה המוכרת פאקמן, ישירות על מסך הטלפון. קהל היעד הוא שחקנים שגדלו על המשחק המקורי ומחפשים דרך נוחה ומוכרת לחזור אליו בעידן הנייד — בלי להזדקק לקונסולה ישנה או למחשב שולחני. הבחירה בנושא נבעה מקרבה אישית ואותנטית: כמעריך ותיק של המשחק, רציתי לתכנת משהו שאני מכיר לעומק, שמעורר בי עניין אמיתי, ושיהפוך את תהליך הלמידה לחוויה מהנה ומשמעותית.

תהליך המחקר

לפני שנכתבה שורת קוד אחת, הוקדש זמן ניכר למחקר מעמיק על המשחק המקורי ועל המכניקות שהפכו אותו לאגדי. אחד הממצאים המעניינים ביותר היה שאלגוריתם המעקב של כל אחת מארבע הרוחות שונה זה מזה — מנגנון שנועד לאפשר ארבויות מתוכננות ומגוונות על השחקן, ולהפוך כל מפגש לאתגר ייחודי שדורש חשיבה ולא רק תגובות מהירות. לאחר שהבסיס התגבש, נחקרו תוספות שיוכלו להעשיר את חוויית המשחק מעבר למקור ולהוסיף לו ערך ייחודי. במקביל, נלמדה סביבת העבודה לעומק, כדי להבטיח שהמימוש הטכני יהיה מבוסס, יציב ומותאם לדרישות הפרויקט.

אתגרים מרכזיים

האתגר הבולט ביותר במהלך הפיתוח היה בתחום האנימציה. Flutter, כסביבת עבודה, אינה מספקת מחלקת אנימציה מובנית ומסודרת להחלפת תמונות — פונקציה חיונית למשחק מסוג זה, שבו כל דמות חייבת לנוע בצורה חלקה ורציפה. הפתרון הטוב ביותר שנמצא בקרב ספריות צד שלישי דרש שינויים דרסטיים מדי במבנה הקוד הקיים, שינויים שהיו פוגעים ביציבות הפרויקט ובאפשרות לתחזק אותו בעתיד. לכן, התקבלה ההחלטה לכתוב מחלקת אנימציה עצמאית ומותאמת אישית — כזו שנותנת מענה מדויק לצרכי הפרויקט, שומרת על מבנה קוד נקי, ומאפשרת גמישות לשינויים עתידיים. מעבר לאתגר הטכני, הפרויקט ממלא גם צורך פרקטי אמיתי: לשחק בפאקמן בכל מקום ובכל זמן, ישירות מהטלפון, ללא תלות בחיבור לאינטרנט או בצידוד נוסף.

חידושים

הפרויקט אינו עוצר בשחזור נאמן של המקור — הוא מרחיב עליו ומוסיף תוכן ומאפיינים חדשים שלא נכללו במשחק המקורי. התוספות תוכנו בקפידה כך שישתלבו באופן טבעי עם הרוח הקלאסית של המשחק, ויעניקו לשחקנים ותיקים תחושת חידוש וגיוון — מבלי לאבד את הקסם שהפך את פאקמן לאחד המשחקים האהובים בהיסטוריה.

תיאור תחום הידע

אובייקטים נחוצים

- שחקן - Pacman - פקמן צריך לאכול כמה שיותר נקודות ולברוח מהרוחות שמנסות לעצור אותו
- הרוחות - הדמויות הללו מנסים לעצור את השחקן ולתפוס אותו, ממשים את דמויות Antagonist
- שלט - דרך השליטה של המשתמש בpacman
- מבוך - מפת המשחק שמראה איפה כל הנקודות נשארות. בנוסף המבוך מאפשר מעבר בין צד ימין לשמאל.
- נקודות - היעד המרכזי של השחקן הוא לאסוף את כמה שיותר מהם. יש שני סוגים של נקודות אחד רגיל ואחד שמאפשר אכילה של רוחות
- בונוס - פירות וחפצים הנוצרים במרכז המסך המביאים לפקמן בונוס לניקוד, בנוסף יש אתגר באפליקציה לאסוף את כל הבונוסים

סוגי נתונים

- ניקוד - כמות הניקוד שהשחקן צבר
- שיא - כמות הנקודות הכי טוב שהשחקן צבר
- חיים - כמות הפעמים שנשאר לשחקן אפשרות למות עד שנגמר המשחק

ייצוג מידע

- מסך המשחק מראה את כל דמויות המשחק, נקודות, השלט, מספר הנקודות השחקן צבר, שיא הנקודות שצבר אי פעם ומספר החיים שנשארו במספר הפעמים שפקמן במסך
- מפת המשחק מפת המבוך - מערך דו מימדי של מספרים, כל מספר מסמל אובייקט אחר במפה
- השלט מורכב מ 4 כפתורים לכל כיוון הבוחרים את הכיוון הרצוי שפקמן יסתובב אליו

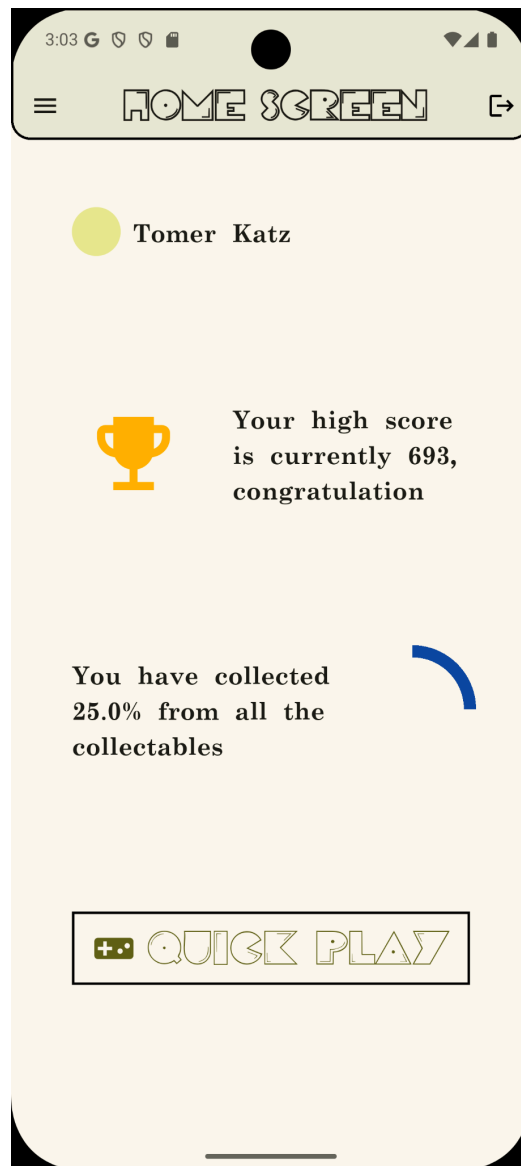
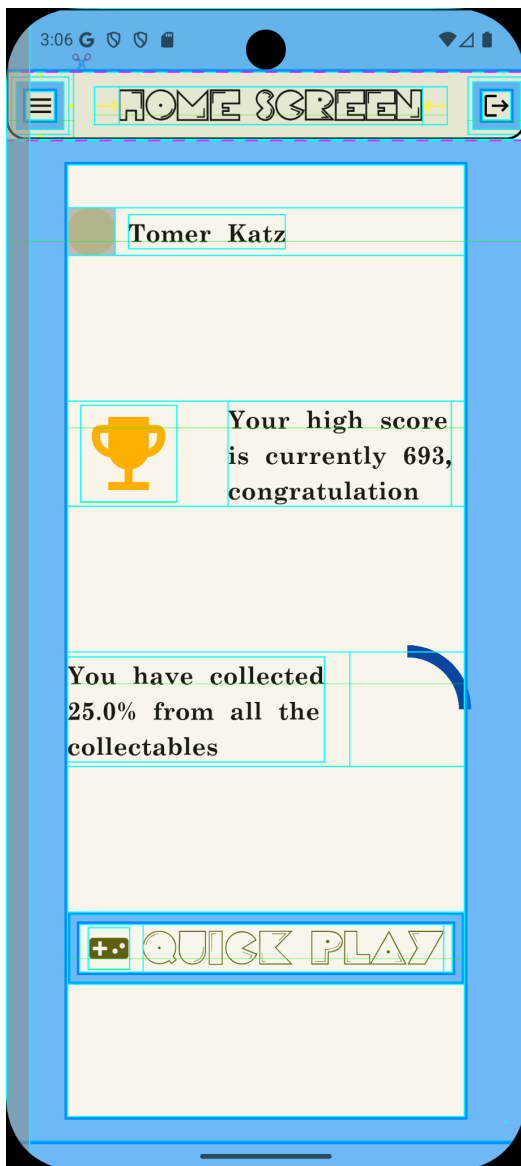
תיאור פעולות על המצב

- כל סייקל יש בדיקה של כל נקודה, רוח ובונוס האם פקמן אכל אותה ואם כן הוספה לניקוד
- יש בדיקה לרוח אם הצליחו להגיע לפקמן, אם כן הורדת חיים
- אם הניקוד הנצבר יותר גדול מהשיא השיא מתעדכן על פי הניקוד במשחק
- בסוף משחק יש בדיקה אם הניקוד היה יותר מהשיא אם כן עדכון של השיא במוסד נתונים

מבנה / ארכיטקטורה של הפרויקט

מסך הבית

שם המסך: מסך הבית (Home Screen)
תפקיד המסך: מסך מרכזי המאפשר כניסה ראשית לאפליקציה, תצוגה של פרטים חשובים וכדומה
תיאור המסך: המסך מציג מידע לגבי המשתמש, כמו השם שלו, השיא שלו וכמה הוא מתקדם במעקב ערך הבונוסים. בנוסף יש כפתור למשחק מהיר.
תמונה:

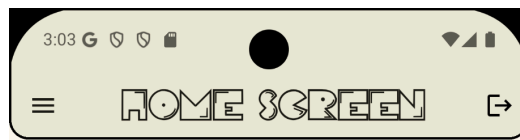
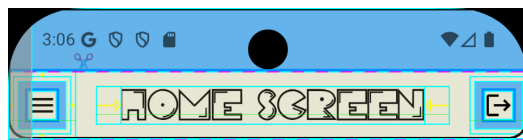


הסבר על אלמנטים במסך:

אלמנט תצוגה	שם האלמנט	תפקיד
Text	שם המשתמש	מראה את השם של המשתמש המחובר
Text	ניקוד שיא	מראה למשתמש מה השיא שלו
Text	התקדמות בבונוסים	מראה למשתמש אחוז התקדמות במציאת כל הבונוסים
CircularProgressIndicator	גרף התקדמות בבונוסים עגול	מראה למשתמש התקדמות במציאת הבונוסים בגרף עגול
GestureDetector	כפתור למשחק	מעבר למשחק מהיר

תפריט עליון (App Bar)

שם המסך: תפקיד עליון
תפקיד המסך: תצוגה של שם המסך הנוכחי ואפשרות לכפתור לניווט לתפריט צד והתנתקות. התפריט נמצא בכל מסך.
תיאור המסך: המסך מציג את שם המסך המשתנה, כפתור ליציאה מהמשתמש וכפתור להכנסות לתפריט צד תמונה:

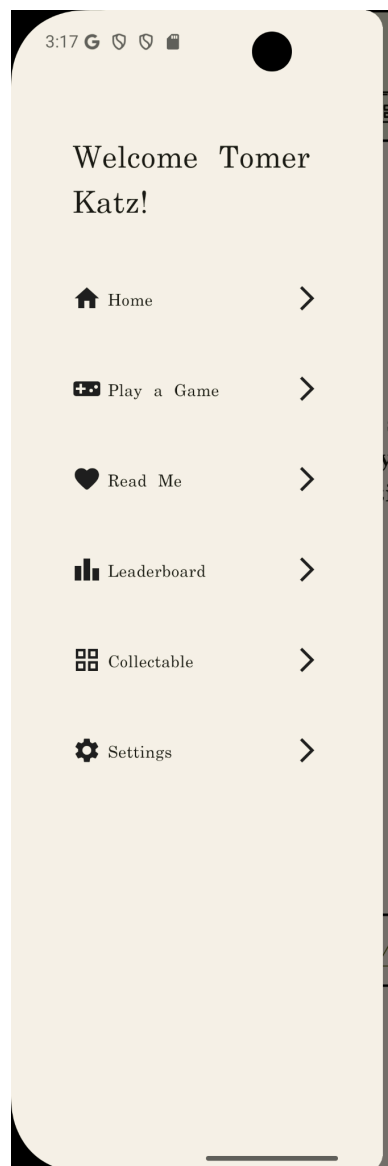
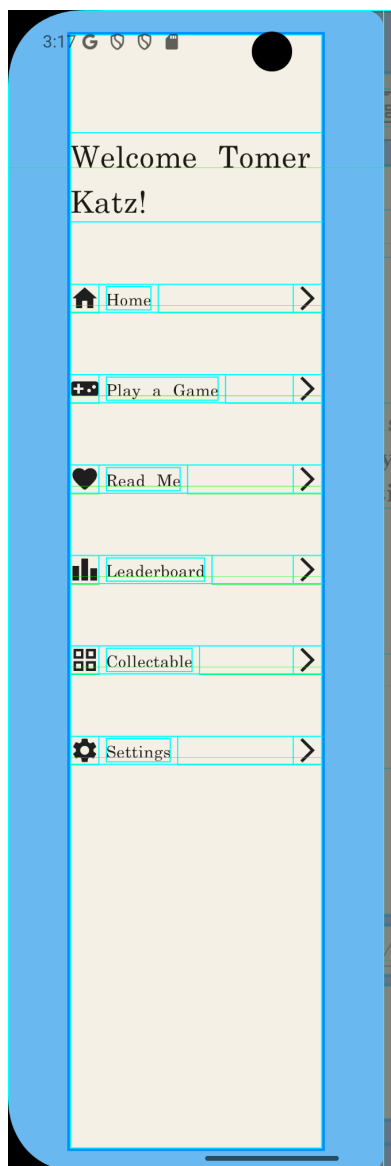


הסבר על אלמנטים במסך:

אלמנט תצוגה	שם האלמנט	תפקיד
Text	Title	שורת טקסט המראה מה המסך הנוכחי
GestureDetector	כפתור התנתקות	אפשרות למשתמש להתנתק באפליקציה, שולח למסך כניסה
GestureDetector	כפתור תפריט צד	פותח את התפריט צד

תפריט צד (Sidebar)

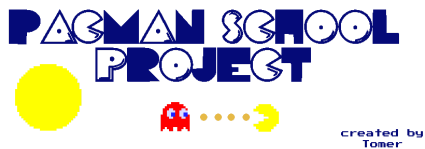
שם המסך: תפריט צד
תפקיד המסך: ניווט בין מסכי התוכנה
תיאור המסך: במסך יש תחילה שלום למשתמש ואז רשימה של כל המסכים באפליקציה ואפשרות לניווט ביניהם על ידי לחיצה עליהם
תמונה:



הסבר על אלמנטים במסך:

אלמנט תצוגה	שם האלמנט	תפקיד
Text	שם המשתמש	מראה את השם של המשתמש

Pacman Android Edition

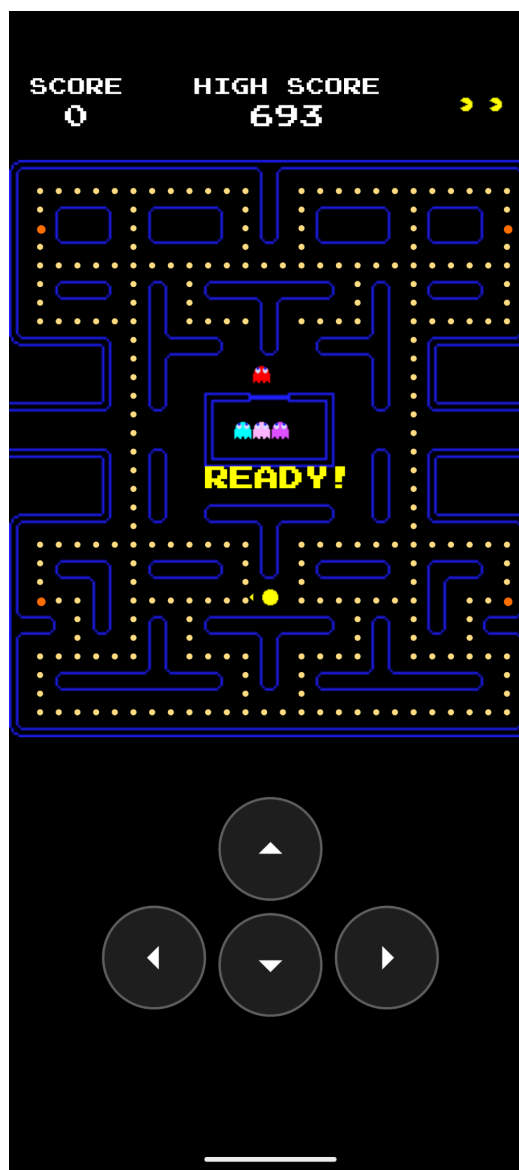


תומר כץ

תפקיד	שם האלמנט	אלמנט תצוגה
המחובר		
כפתור לכל מסך יחד עם אייקון ושם המסך, שולח למסך הרלוונטי	שורת מסך	GestureDetector

מסך המשחק

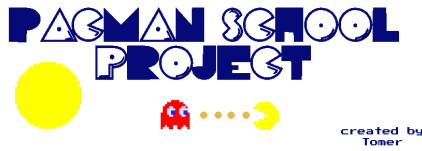
שם: מסך המשחק (Game Displayer)
 תפקיד המסך: הרצה של המשחק
 תיאור המסך: מסך המשחק של pacman, יש שלט בחלק התחתון של המסך וניקוד וחיים בחלק העליון
 תמונה:



הסבר על אלמנטים במסך:

אלמנט תצוגה	שם האלמנט	תפקיד
Controller	שלט משחק	כפתורים לשחקן לשלוט בפקמן

Pacman Android Edition

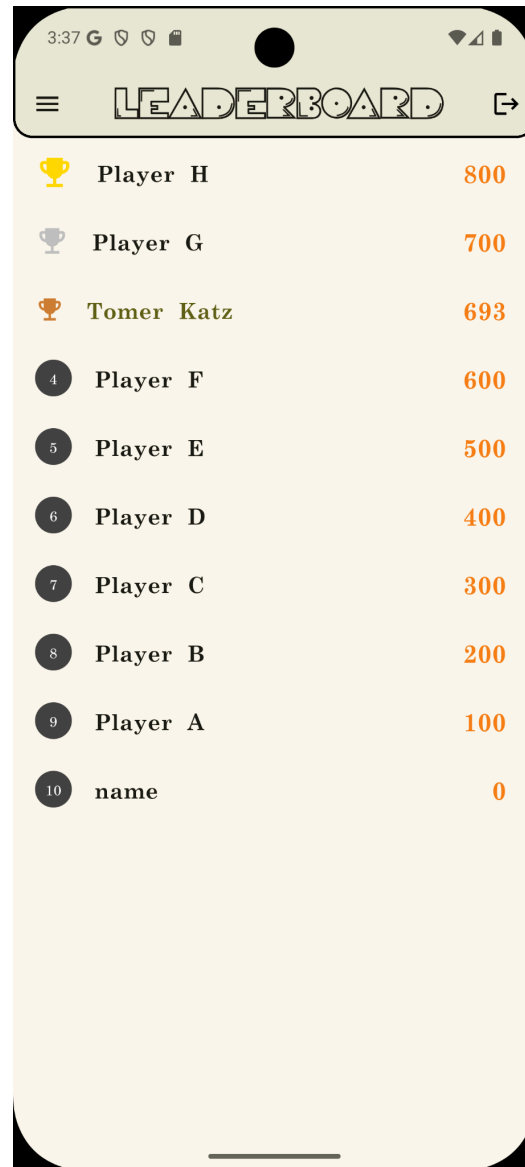
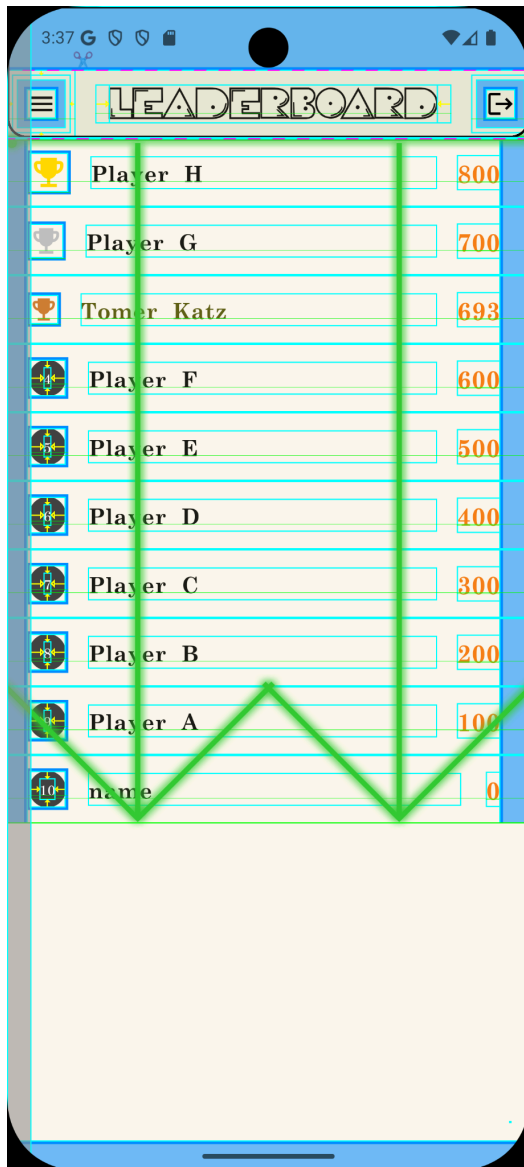


תומר כץ

תפקיד	שם האלמנט	אלמנט תצוגה
תצוגה של החיים שנשארו לפקמן, מקסימום 3 שורות של 3 חיים	Lives	ListView
מראה את הניקוד שיא של המשתמש	High Score	Text
מראה את הניקוד הנוכחי של המשחק	Score	Text
הודעת התחלה של המשחק, מופיע בהתחלה ובסוף משחק	StartingMessage	Text
מראה את המפה של המבוך	Map	Image
כל הנקודות שפקמן צריך לאסוף	Dot	Dots
הנקודות הגדולות שמביאות לפקמן אפשרות לאכול את הרוחות לזמן קצר	SuperDot	SuperDots
בונוס מופיע באופן רנדומלי במרכז המבוך	Bonus	Bonus
מראה את כל הבונוסים שנאספו	בונוסים שנאספו	ListView
תצוגה של פקמן ומיקומו במבוך	Pacman	AnimatedCharacter
כל הרוחות הרודפות אחרי פקמן	Ghosts[] (Blinky,Pinky, Inky, Clyde)	AnimatedCharacter[]

מסך דירוגים

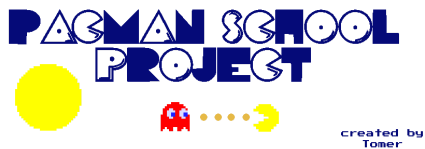
שם מסך: מסך דירוגים (Leaderboard)
תפקיד המסך: תצוגה של המיקום של הניקוד של כל השחקנים
תיאור המסך: ניתן לראות רשימה של כל השחקנים ומה הניקוד שלהם
תמונה:



הסבר על אלמנטים במסך:

אלמנט תצוגה	שם האלמנט	תפקיד
ListView	שורת דירוג	מראה את הדירוג של כל

Pacman Android Edition

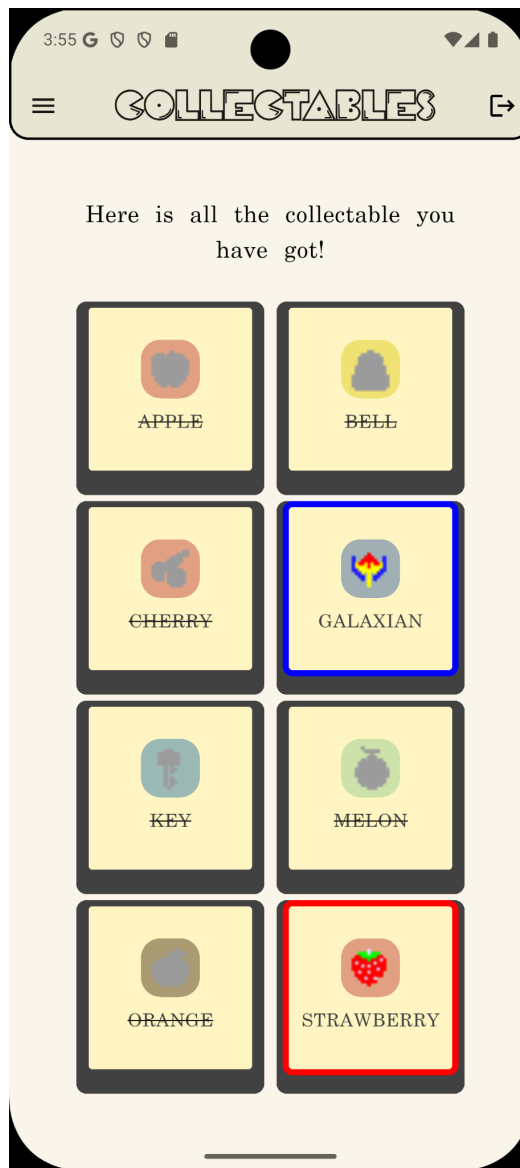
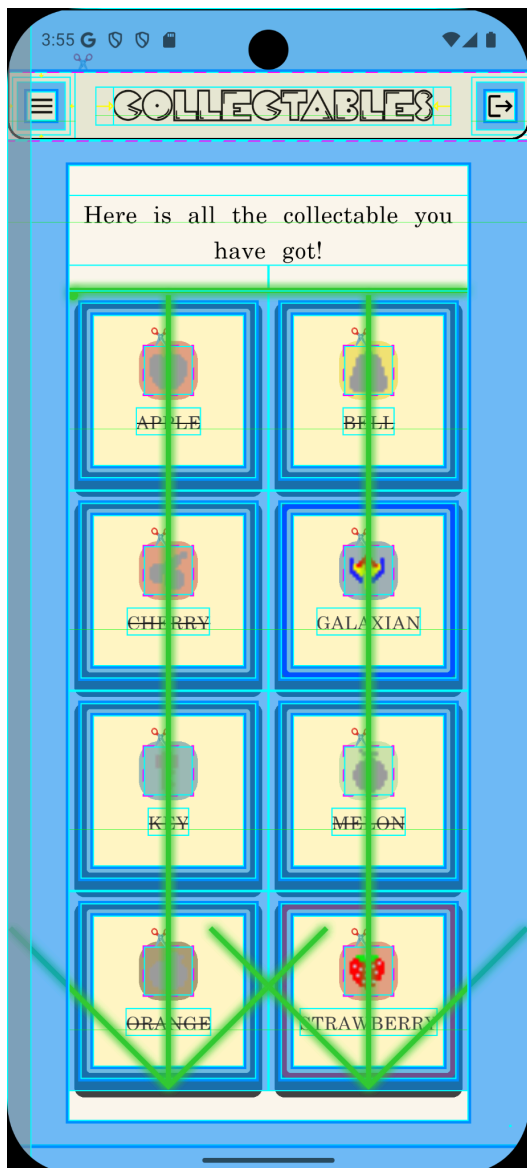


תומר כץ

אלמנט תצוגה	שם האלמנט	תפקיד
		משתמש [אלמנטים למטה הם חלק מהשורה]
Text	שם המשתמש	מראה את השם של המשתמש (בצבע מיוחד עם זה המשתמש המחובר)
Text	ניקוד השיא של המשתמש	מראה את הניקוד שיא של השורת דירוג הנוכחית
Icon	דירוג המשתמש (שלוש מקומות גבוהים)	דרגת המשתמש ל 3 מקומות הכי גבוהים
CircularAvatar	דירוג המשתמש (מקום 4 ומעלה)	מראה את הדירוג של המשתמש

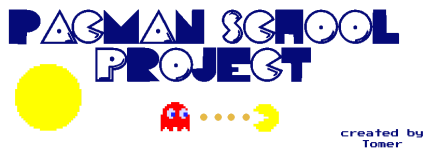
מסך בונוסים

שם מסך: מסך הבונוסים או אספנות (Collectables)
תפקיד המסך: מעקב אחרי הבונוסים שהמשתמש אסף איזה אסף ואילו לא וכמה
תיאור המסך: רשימה של כל שמונת הבונוסים במשחק, בונוסים שהמשתמש לא אסף מוחבאים ואילו
שאסף פותחים תצוגה של כמה
תמונה:



הסבר על אלמנטים במסך:

Pacman Android Edition

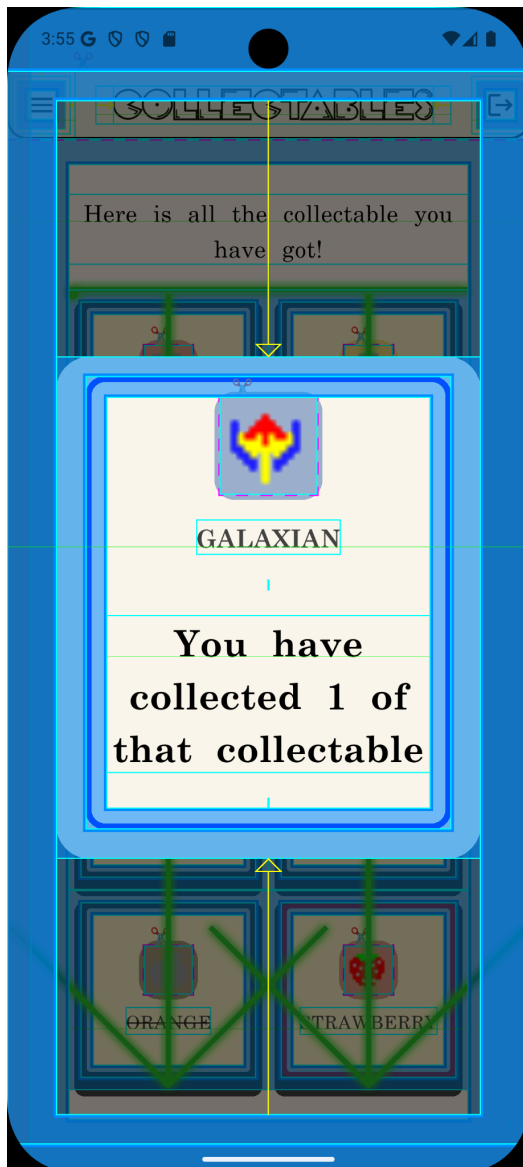


תומר כץ

אלמנט תצוגה	שם האלמנט	תפקיד
Text	הסבר	הסבר על הבונוסים והמסך
ListView	Bonus List	רשימה של כל הבונוסים, בונוס שלא נאסף יראה באפור [כל אלמנט הלאה יהיה על כל אחד מהרשימה]
GestureDetector	כפתור בונוס	פותח את דיאלוג הבונוס אם נאסף לפחות פעם אחת
AnimatedContainer	BonusCard	כפתור עם אנימציה לחיצה
Image	תמונת בונוס	מראה את התמונה של הבונוס, בצבע אם נאסף

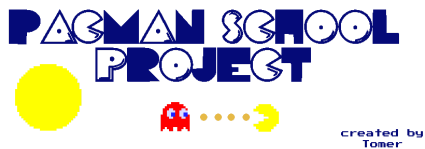
דיאלוג בונים

שם מסך: דיאלוג בונים (Bonus Dialog)
תפקיד המסך: מראה כמות פעמים שנאסף בונים מסויים
תיאור מסך: תצוגה של הבונים עם שמו וכמות הפעמים שנאסף
תמונה:



הסבר על אלמנטים במסך:

Pacman Android Edition

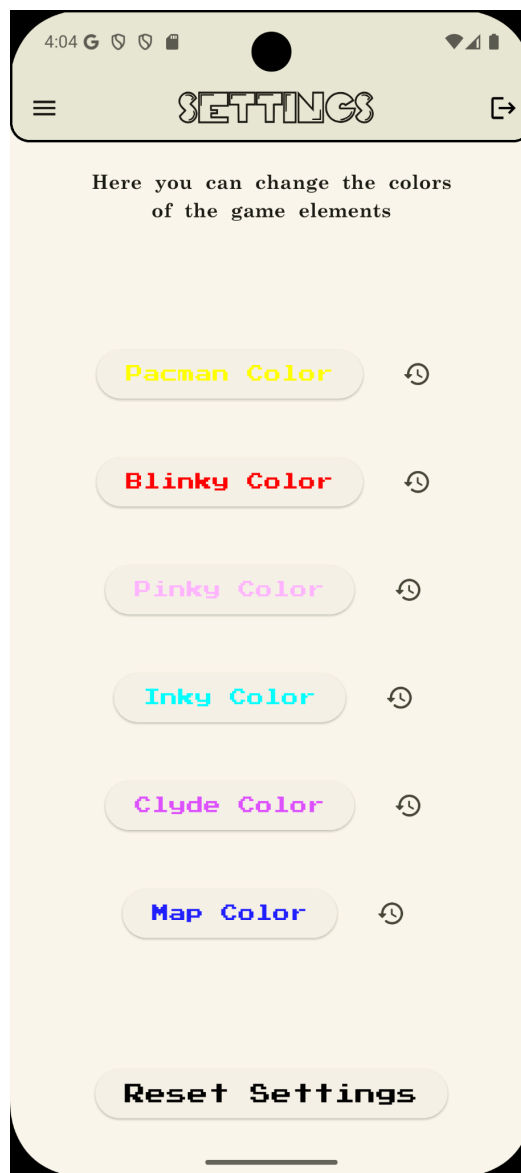
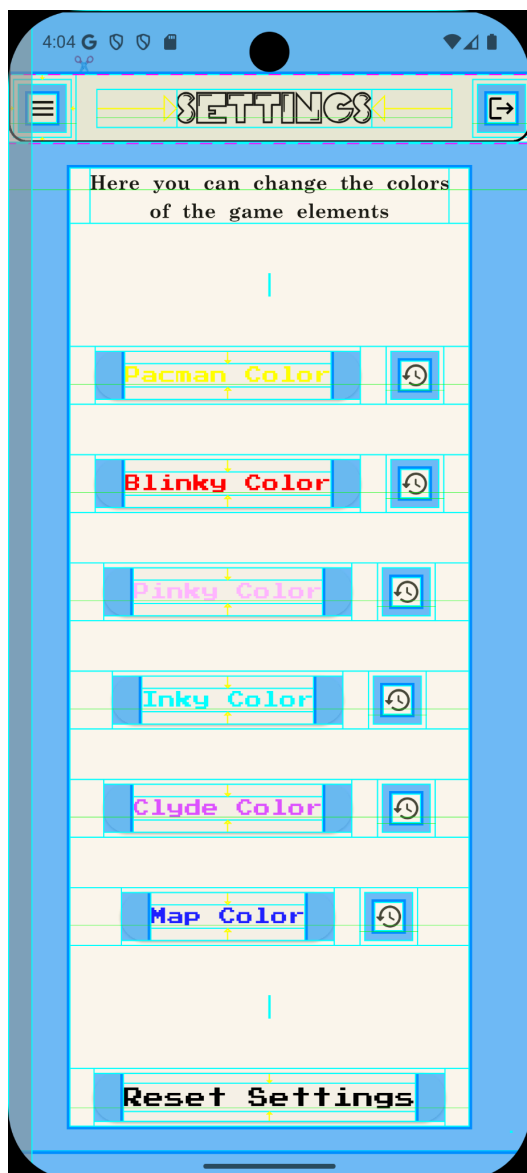


תומר כץ

אלמנט תצוגה	שם האלמנט	תפקיד
Image	תמונת הבונוס	תמונה של הבונוס הנבחר
Container	Container	מוסיף לעיצוב
Text	שם הבונוס	שם הבונוס הנבחר
Text	כמות הפעמים שנאסף הבונוס	מראה למשתמש כמות הפעמים שנאסף הבונוס

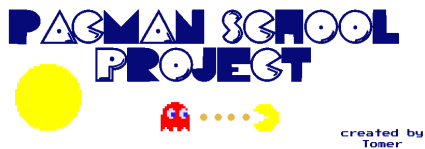
הגדרות

שם המסך: הגדרות
תפקיד המסך: אפשרות לשינוי הצבעים של אלמנטי המשחק
תיאור המסך: יש כפתור שפותח דיאלוג לשינוי צבע לפקמן, כל הרוחות והמפה עצמה. בנוסף לכל אלמנט יש כפתור לחזור לצבע המקורי וכפתור בתחתית העמוד שמשחזר את כל הצבעים.
תמונה:



הסבר על אלמנטים במסך:

Pacman Android Edition

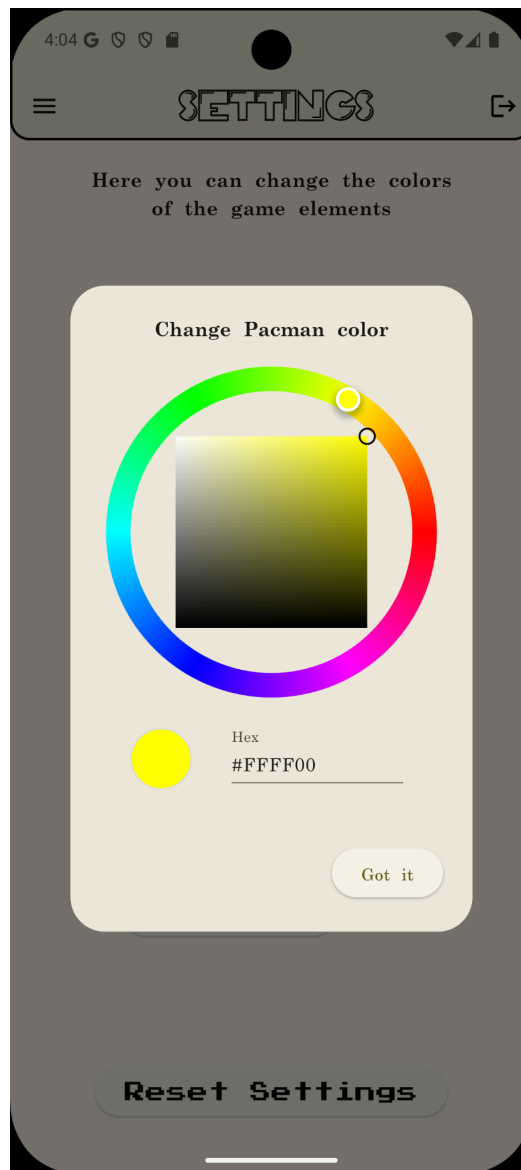
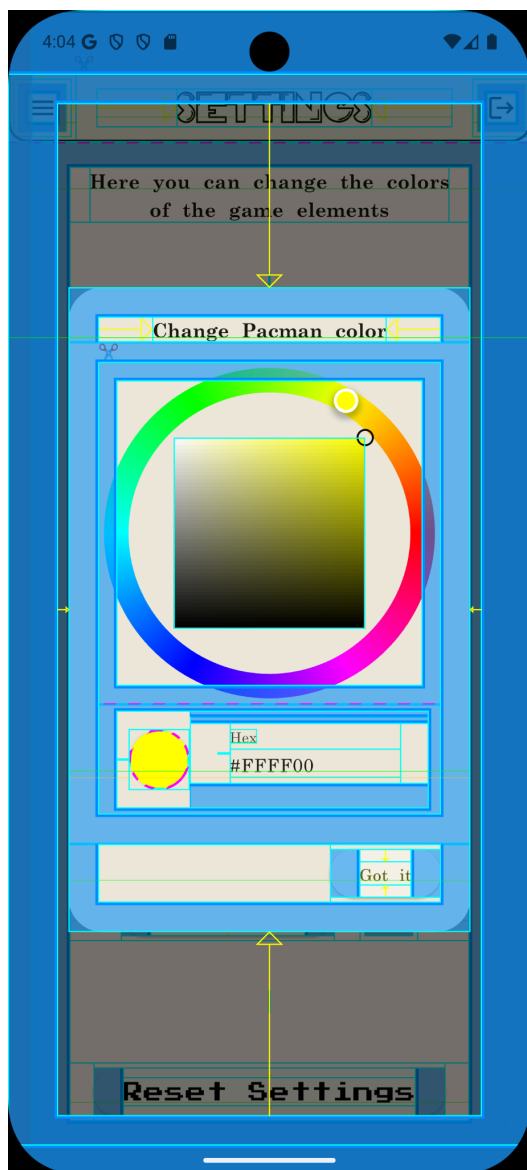


תומר כץ

אלמנט תצוגה	שם האלמנט	תפקיד
Text	הסבר	הסבר על הבונסים והמסך
ElevatedButton	כפתור שינוי צבע	כפתור הפותח את דיאלוג שינוי צבע [קיים 5 פעמים לכל אלמנט], טקסט בצבע הנוכחי של האלמנט
IconButton	כפתור שחזור ספציפי	ליד כל כפתור שינוי צבע יש כפתור שחזור לצבע מקורי של האלמנט
ElevatedButton	כפתור שחזור כללי	כפתור לשחזור של כל הצבעים

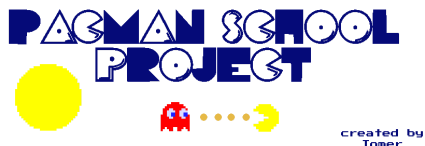
דיאלוג שינוי צבע

שם המסך: דיאלוג שינוי צבע
תפקיד המסך: אפשרות לשינוי צבע לאלמנט משחק
תיאור המסך: אופציה לבחירת צבע שמשנה את הצבע של האלמנט הנבחר



הסבר על אלמנטים במסך:

Pacman Android Edition

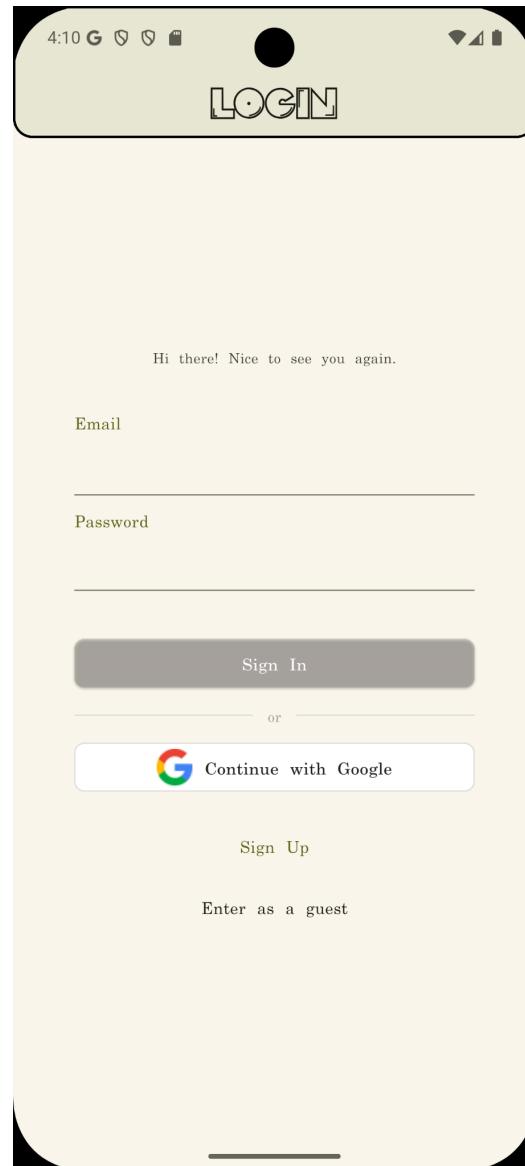
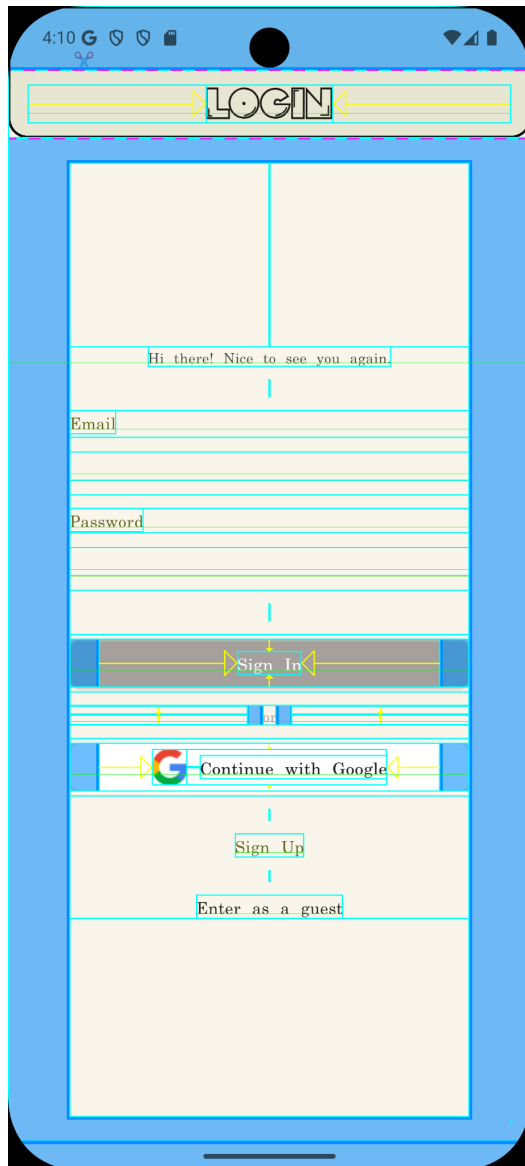


תומר כץ

אלמנט תצוגה	שם האלמנט	תפקיד
Text	כותרת הדיאלוג	רשום איזה אלמנט נבחר
HueRingPicker	אלמנט בחירת צבע	משנה את הצבע של האלמנט הנבחר
ElevatedButton	כפתור יציאה	כפתור חזרה ושמירה

מסך כניסה (Login)

שם מסך: מסך כניסה (Login)
תפקיד המסך: מאפשר כניסה של המשתמש דרך אימייל וסיסמא או גוגל
תיאור המסך: תיבת טקסט בשביל הכנסת אימייל ותיבת טקסט בשביל הכנסת אימייל, בנוסף כפתור כניסה וכפתור לנסות דרך גוגל. יש גם אפשרות להיכנס בתור אורח או ללכת למסך ההרשמה להירשם תמונה:

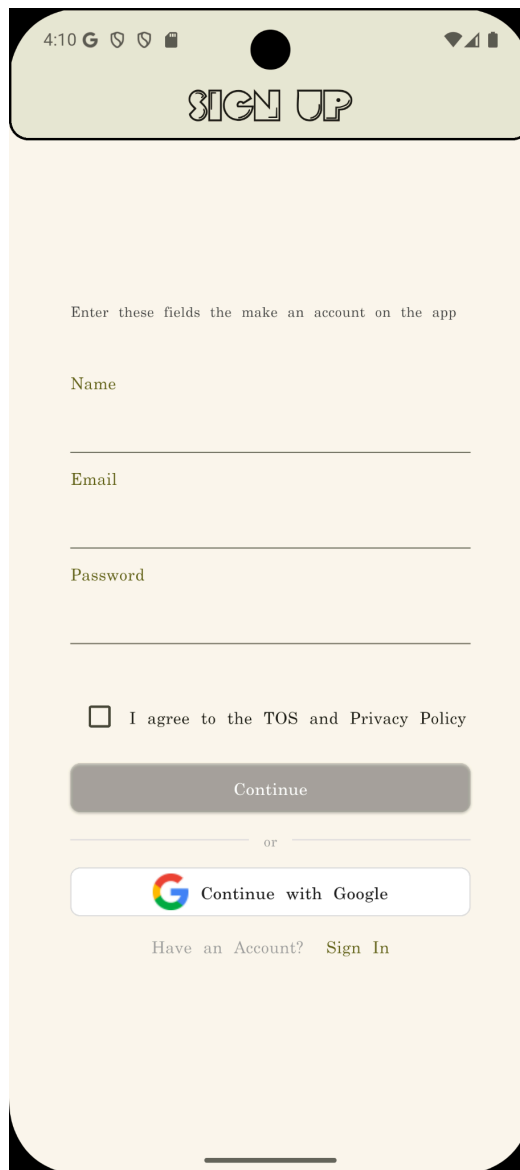
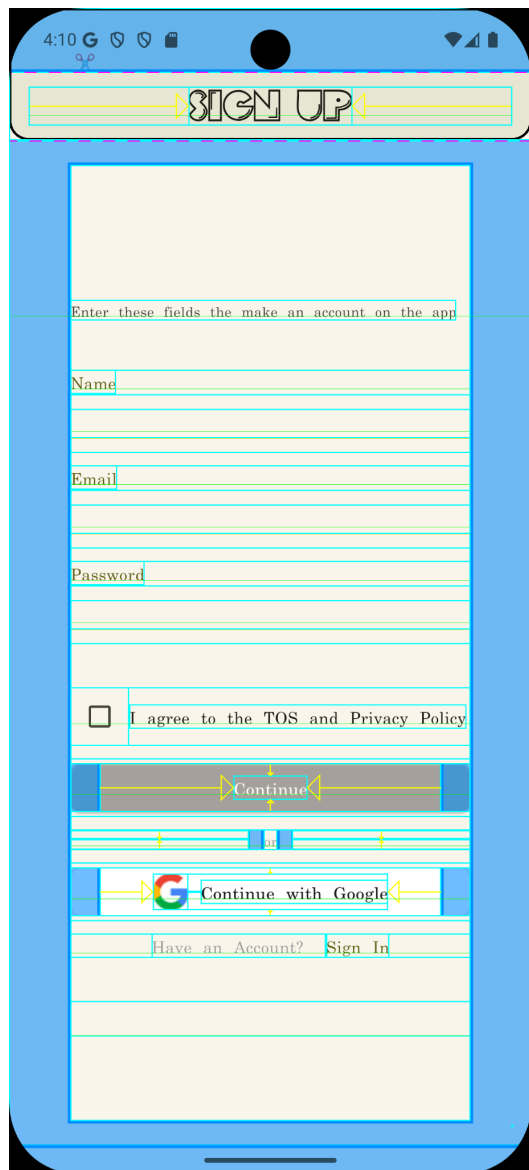


הסבר על אלמנטים במסך:

אלמנט תצוגה	שם האלמנט	תפקיד
Text	הסבר	הסבר למשתמש למלא את השדות הבאים
Text	כותרת email	כותרת לשדה טקסט אימייל
TextField	שדה טקסט email	שדה טקסט להכנסת אימייל
Text	כותרת סיסמא	כותרת לשדה טקסט סיסמא
TextField	שדה טקסט סיסמא	שדה טקסט להכנסת סיסמא
ElevatedButton	כפתור כניסה	בודק את הסיסמא ואימייל ומכניס את המשתמש
Divider	Divider	עיצוב
OutlinedButton	כפתור כניסה דרך גוגל	מחבר את המשתמש דרך חשבון גוגל
GestureDetector	לינק לעמוד הרשמה	שולח את המשתמש להירשם אם אין לו חשבון
GestureDetector	לינק כניסה אנונימית	שולח את המשתמש לעמוד הכניסה בלי משתמש

מסך הרשמה (Sign up)

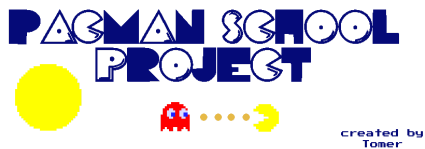
שם: מסך הרשמה (Sign up)
תפקיד המסך: אפשרות להרשמת משתמשים
תיאור המסך: תיבות טקסט לשם, אימייל וסיסמא. כפתור לאשר תנאי שימוש, כפתור להרשמה וכפתור הרשמה דרך גוגל. בנוסף קישור לעמוד הכניסה
תמונה:



הסבר על אלמנטים במסך:

אלמנט תצוגה	שם האלמנט	תפקיד
Text	הסבר	הסבר למשתמש למלא את השדות הבאים
Text	כותרת שם	כותרת לשדה טקסט שם
TextField	שדה טקסט שם	שדה טקסט להכנסת שם
Text	כותרת email	כותרת לשדה טקסט אימייל
TextField	שדה טקסט email	שדה טקסט להכנסת אימייל
Text	כותרת סיסמא	כותרת לשדה טקסט סיסמא
TextField	שדה טקסט סיסמא	שדה טקסט להכנסת סיסמא
CheckBox	תיבת סימון של תנאי שימוש	שדה סימון של הסכמה לתנאי שימוש
Text	כותרת תנאי שימוש	מסביר על תיבת סימון
ElevatedButton	כפתור הרשמה	רושם המשתמש החדש
Divider	Divider	עיצוב
OutlinedButton	כפתור כניסה דרך גוגל	רושם את המשתמש דרך חשבון גוגל
Text	הסבר	שואל את המשתמש אם יש לו חשבון קיים

Pacman Android Edition

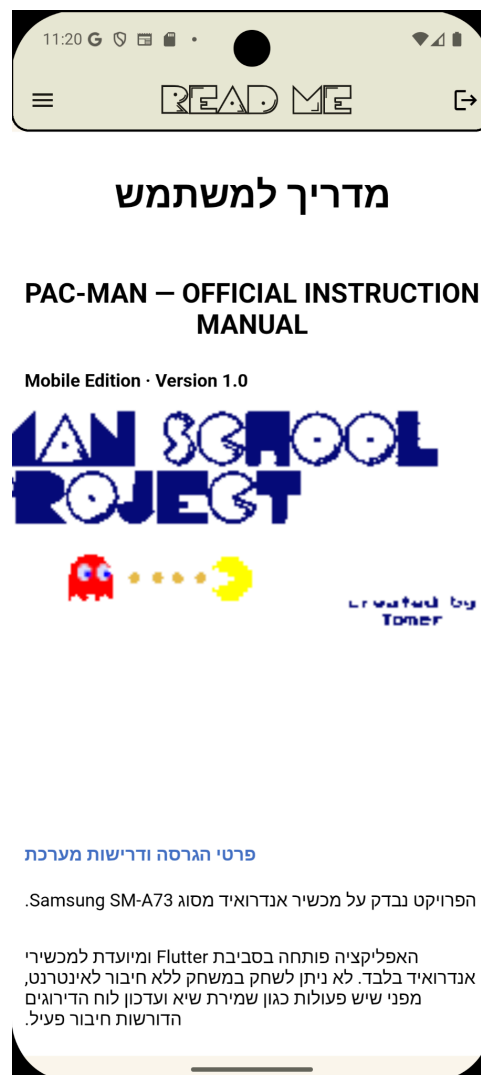
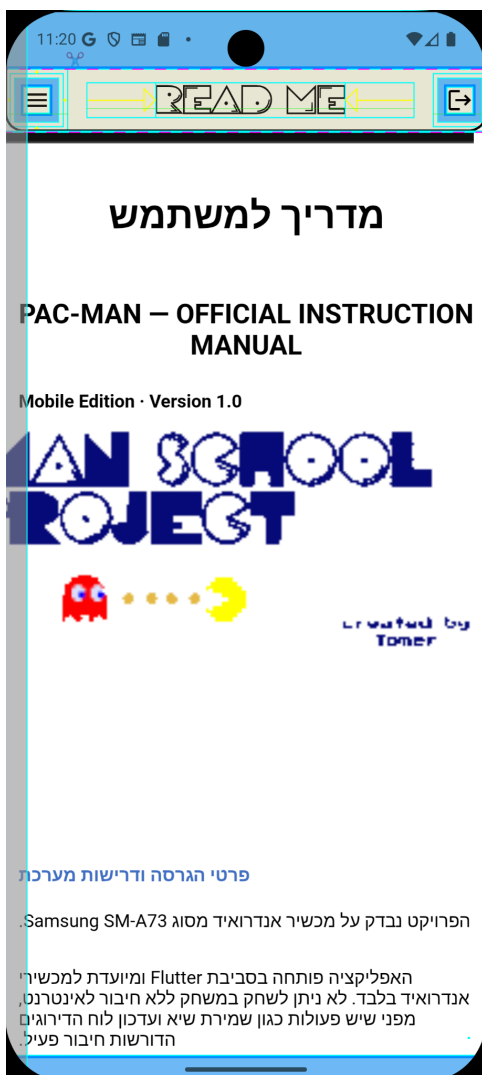


תומר כץ

תפקיד	שם האלמנט	אלמנט תצוגה
שולח את המשתמש לעמוד כניסה	לינק לעמוד כניסה	GestureDetector

מסך הסברים (Read Me)

שם: מסך הסברים (Read Me)
תפקיד המסך: להסביר למשתמש דברים
תיאור המסך: קישור לתיק פרויקט הנוכחי
תמונה:



הסבר על אלמנטים במסך:

אלמנט תצוגה	שם האלמנט	תפקיד
WebViewWidget	תצוגה מהאינטרנט	מראה באפליקציה את המסמך הנוכחי

תרשים מסכים

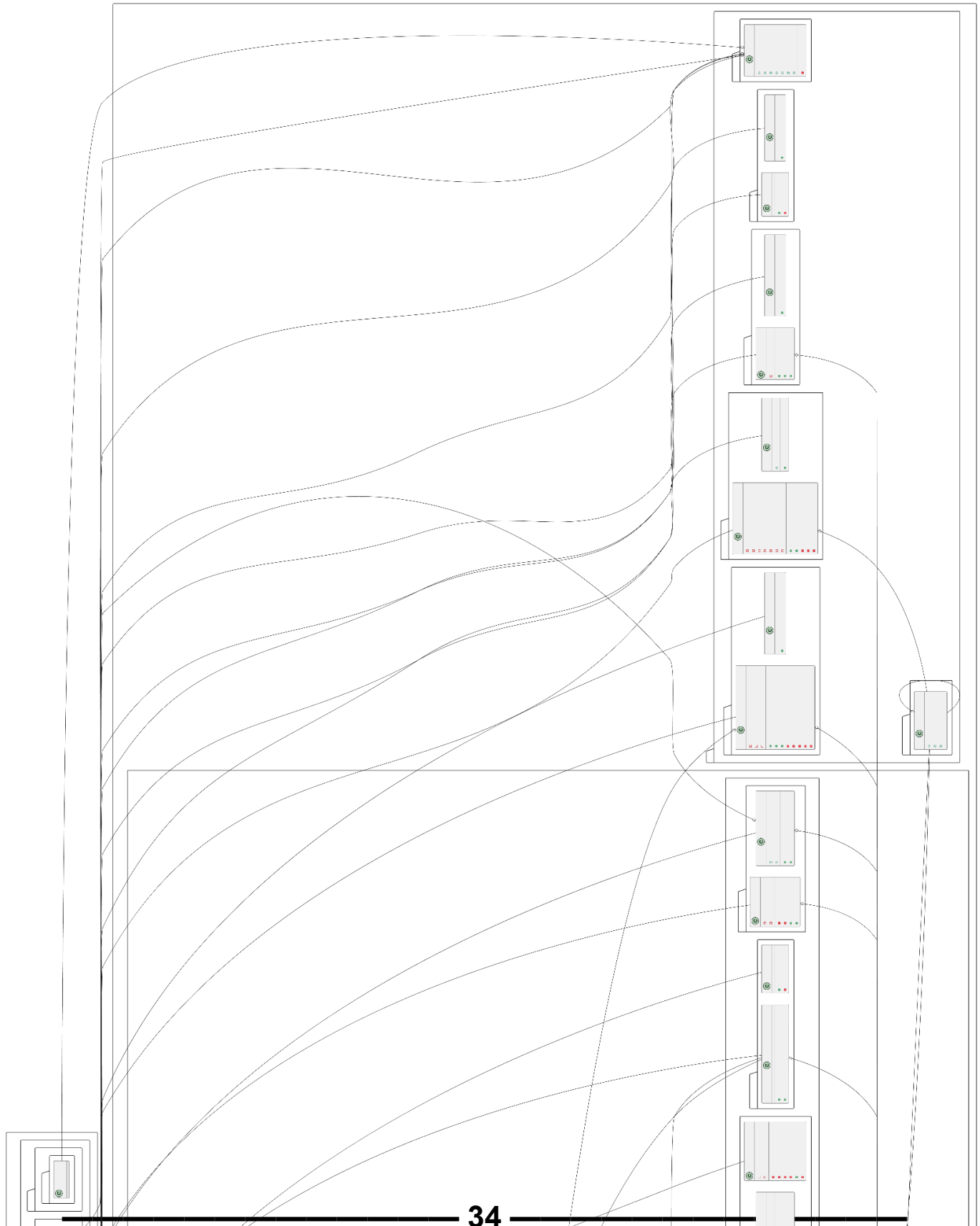


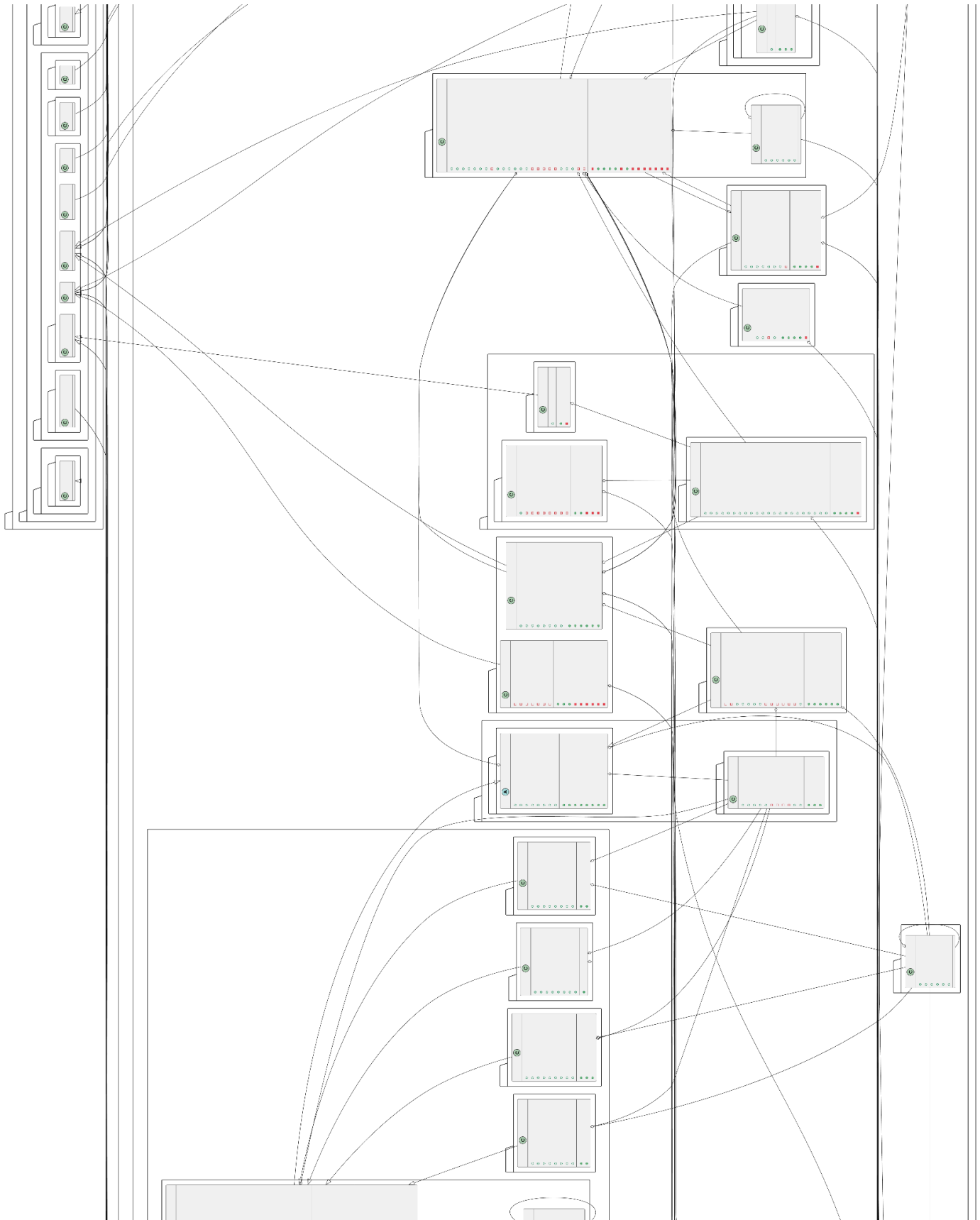
מקרא:

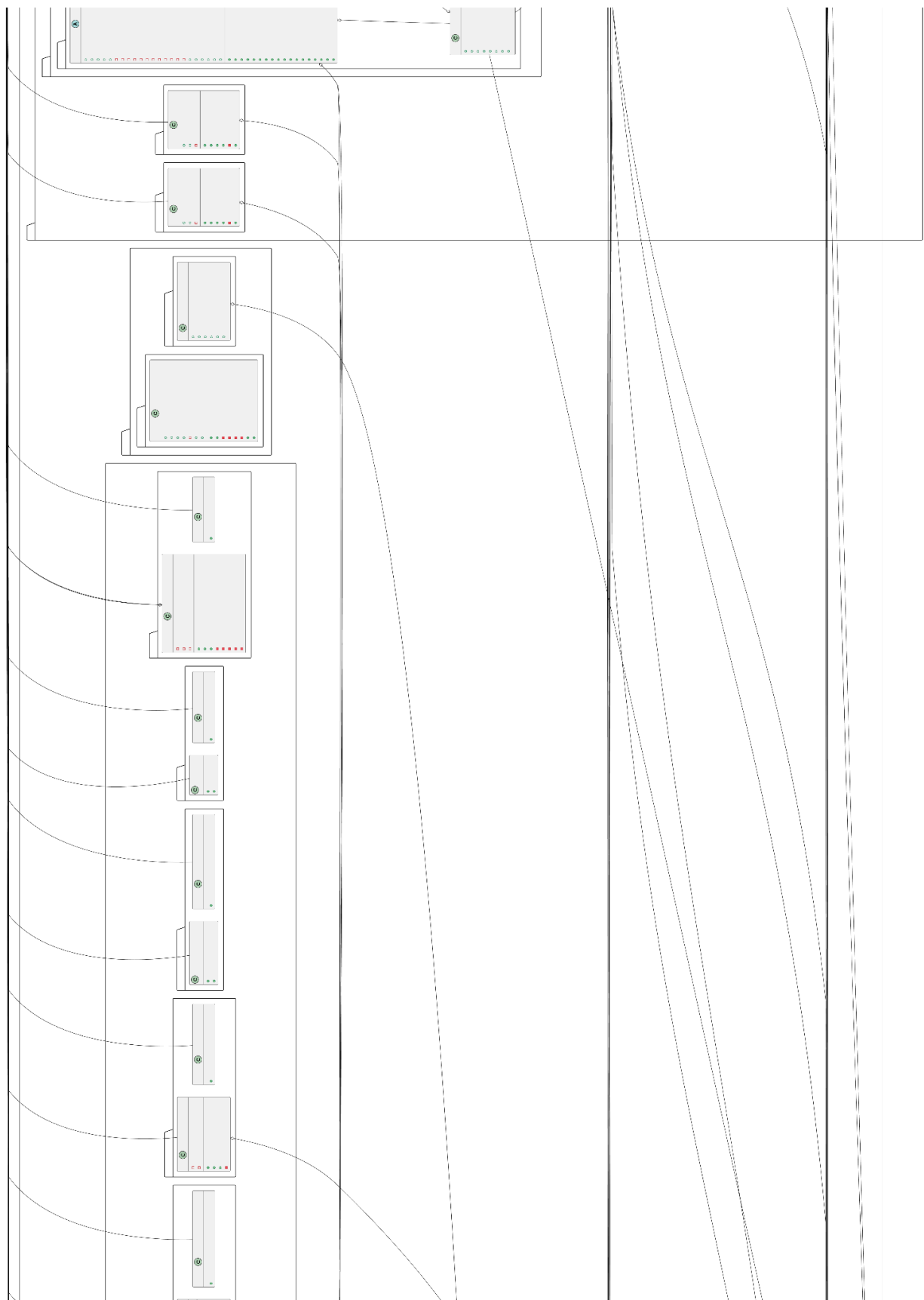
- מסכים : ●
- תפריטים : ●
- דיאלוגים : ●
- קשרים / הערות : ●

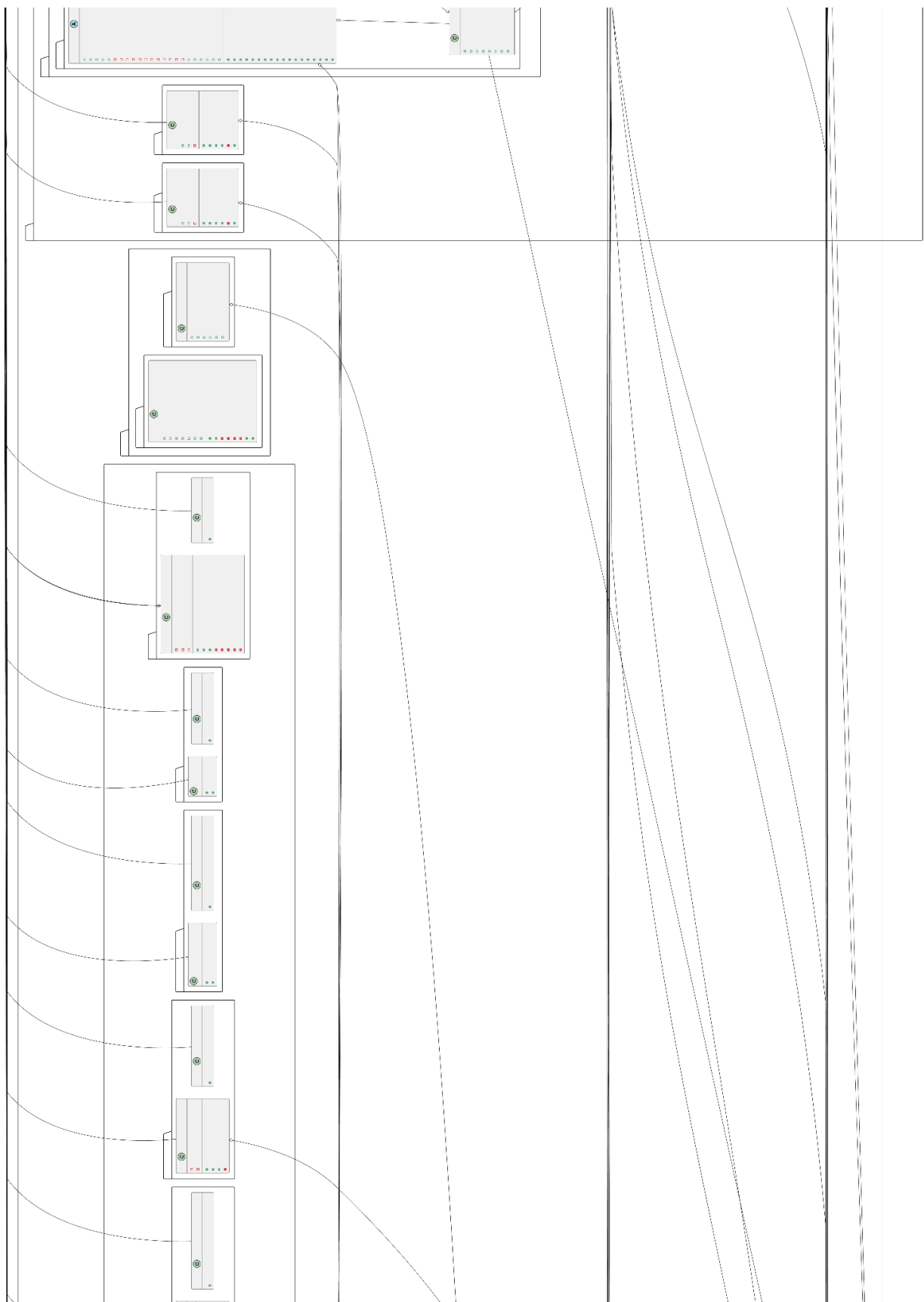
תרשים UML של הפרויקט

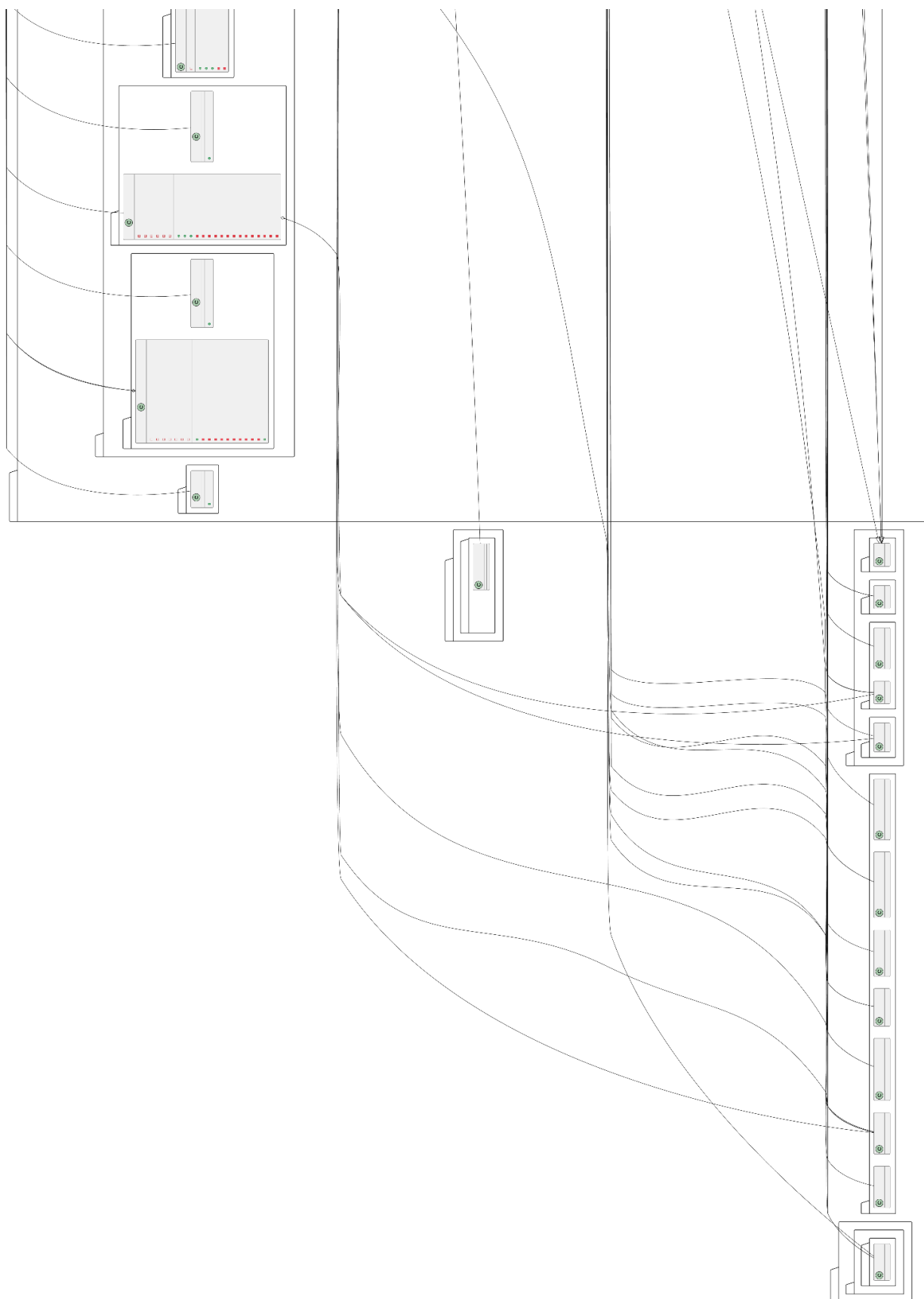
בגלל גודל הפרויקט אני חייב לחלק את הוורק ל-4 חלקים שונים. כל חלק יהיה עמוד מהעמודים הבאים











מימוש הפרויקט

מחלקות לפי מודל ה-MVC

Model

My App

שם המחלקה: MyApp

סוג מחלקה: **Stateless Widget**

הורשה: StatelessWidget

תפקיד: מכיל את כל האפליקציה

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
Override	מייצר את התצוגה של האפליקציה	Widget	BuildContext	build

Default Firebase Options

שם המחלקה: DefaultFirebaseOptions

סוג מחלקה: רגיל

הורשה:

תפקיד: שומר את הסיסמאות של החיבור עם Firebase

תכונות:

הערות	תיאור	סוג	תכונה
static getter	מקבל את הנתוני firebase על פי המכשיר	FirebaseOptions getter	currentPlatform
static const	נותני ה firebase של web	FirebaseOptions	web
static const	נותני ה firebase של android	FirebaseOptions	android
static const	נותני ה firebase של ios	FirebaseOptions	ios
static const	נותני ה firebase של macos	FirebaseOptions	macos
static const	נותני ה firebase של windows	FirebaseOptions	windows

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
-------	-------	-------	-----------	-------

Game User

שם המחלקה: GameUser

סוג מחלקה: רגיל

הורשה:

תפקיד: מחלקה למשתמש של האפליקציה

תכונות:

תכונה	סוג	תיאור	הערות
uid	String	id של המשתמש	final
name	String	שם המשתמש	
_highScore	int	השיא של המשתמש	
email	String	האימייל של המשתמש	
bonusTable	Map<BonusType, int>	כמות הבונוסים שהמשתמש השיג	

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
_updateScore	Int score	void	מעדכן את השיא בבסיס הנתונים	async private
addBonus	BonusType bonus	void	מוסיף למפה אחד בבונוס שנאסף	
_updateBonusTable	BonusType bonus	void	מעדכן את הבונוס הספציפי בבסיס הנתונים	async private

הערות	תיאור	מחזיר	Parameter	פעולה
constructor	מאתחל משתמש על פי parameters	GameUser	Required this.uid, this.name, int? Highscore, this.email, this.bonusTable	GameUser
Constractor	מאתחל משתמש על פי בסיס נתונים	GameUser	Required this.uid	GameUser.fromUId
	מחזיר אם המשתמש קיבל את המידע מבסיס הנתונים	bool		haveInitalized
async	מעדכן את המידע מן בסיס הנתונים	void	DataRefrence ref	setData
async, private	מקבל את המידע על הבונוסים מן בסיס הנתונים	Future<MapEntry<BonusType, int>>	BonusType bonus, DataBaseRefrence ref	_getBonusEntry
Override, oParator	בודק אם המשתמש שבבדיקה עם אותו uid	bool	Object other	==
Override, getter	מחזיר את הhash של uid	int		hasCode
Override	רושם את התכונות בדרך מסודרת בהדפסה	String		toString

Session

שם המחלקה: Session

סוג מחלקה: Static <no class>

הורשה:

תפקיד: מכיל תכונות אשר חשובות בהרצה הנוכחית של האפליקציה

תכונות:

תכונה	סוג	תיאור	הערות
currentUser	GameUser	המשתמש הנוכחי	static
isAnonymous	bool	משתנה לבדוק אם המשתמש אנונימי	static

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
-------	-----------	-------	-------	-------

Animated Character

שם המחלקה: AnimatedCharacter

סוג מחלקה: Abstract

הורשה:

תפקיד: דמות במשחק

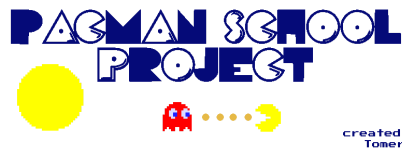
תכונות:

הערות	תיאור	סוג	תכונה
final	לינק למנהל משחק של הדמות	GameManager	gameManager
abstract final	שם הדמות	String	name
abstract	ערך ה x במערכת הצירים של המשחק	double	x
abstract	ערך ה y במערכת הצירים של המשחק	double	y
abstract	הכיוון הנוכחי של הדמות	Direction	currentDirection
abstract	הכיוון הבא של הדמות	Direction	nextDirection
late	רשימה של כל האנימציות של הדמות	List<SpriteAnimation>	animations
late	האנימציה הנוכחית של הדמות	SpriteAnimation	currentAnimation

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
Constructor	מחייב קבלה של המנהל משחק	AnimatedCharacter	Required this.gameManager	AnimatedCharacter

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
	מחזיר את דמות המשחק על פי הערכים בצירים והאנימציה הנוכחית	Widget	double tileSize, {Color? color}	getSprite
abstract	מעדכן את התזוזה של הדמות	void	double dt	move
abstract	מעדכן את הבחירה והאנימציה הנוכחית	void	double dt	updateAnimation
	מעדכן את כל הערכים שצריכים עדכון בדמות	void	double dt	update
	מחזיר את מפת המשחק של הנוכחי	GameMap		getGameMap
	מחזיר את מסך המשחק של הנוכחי	GameScreen		get GameScreen
	מחזיר את ה pacman של המשחק הנוכחי	Pacman		getPacman
abstract	מאתחל את הדמות	void		reset

Sprite Animation

שם המחלקה: SpriteAnimation

סוג מחלקה: רגיל

הורשה:

תפקיד: אנימציה של דמויות במשחק

תכונות:

תכונה	סוג	תיאור	הערות
characterName	String	שם הדמות של האנימציה	final
animationCharacter	String	שם האנימציה	final
_path	List<String>	הכתובת בהם שמורים התמונות של כל שלב באנימציה	final private
spriteAmount	int	מספר הפריימים באנימציה	final
_clock	double	מודד זמן בין כל פריים באנימציה	private
_currentIndex	int	הפריים הנוכחי באנימציה	private
isLoop	bool	האם האנימציה חוזרת על עצמה	final
isPaused	bool	האם האנימציה עצורה	
isDone	bool	האם האנימציה נגמרה	
_started	bool	האם האנימציה התחילה	private
frameSpeed	double	מהירות המעבר פריימים	final
_kAssetsPath	String	תחילת הכתובת של התמונות של הדמויות	private const
_kSpriteFileExtension	String	סוף הכתובת של התמונות של הדמויות	private const

פעולות:

הערות	תיאור	סוג	Parameter	פעולה
constructor	מאתחל את הגדרות האנימציה (לא אוטומטית פועל)	SpriteAnimation	required this.chracterName , required this.animationNa me, required this.spriteAmount, this.isLoop, this.frameSpeed	SpriteAnimation
	מחזיר את התמונה הנוכחית	Image	Color? color	getImage
	מעדכן את האנימציה	void	double dt	update
	מבטל את האנימציה	void		idle

Controller

שם המחלקה: Controller

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: שלט המשחק הנותן שליטה למשתמש

תכונות:

הערות	תיאור	סוג	תכונה
final	פונקציה שתפעל כל פעם שיש שינוי בכיוון	void Function(Direction direction)	onDirectionChanged

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor, const	מאתחל את השלט	Controller	super.key, required this.onDirectionC hange	Controller
override	יוצר את המצב של המחלקה	State<Controller>		createState

Controller State

שם המחלקה: ControllerState

סוג מחלקה: State of Stateful Widget

הורשה: <State<Controller

תפקיד: מצב של השלט

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	מגדיר את ה-widget של השלט	Widget	BuildContext context	build
	יוצר ארבע כפתורי Dpad בשביל כל כיוון	Widget	double btnSize	dpad

Dpad Button

שם המחלקה: DpadButton

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: כפתור לכיוון

תכונות:

הערות	תיאור	סוג	תכונה
final	גודל הכפתור	double	size
final	אייקון שיהיה על הכפתור	IconData	icon
final	פעולה שתקרה כאשר הכפתור נלחץ	VoidCallBack	onPressed

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor, const	מאתחל את הכפתור	DpadButton	super.key, required this.size, required this.icon, required this.onPressed	DpadButton
override	יוצר את המצב של המחלקה	State<DpadButton>		createState

Dpad Button State

שם המחלקה: DpadButtonState

סוג מחלקה: State of Stateful Widget

הורשה: <State<DpadButton

תפקיד: מצב של הכפתור dpad

תכונות:

תכונה	סוג	תיאור	הערות
_isPressed	bool	האם הכפתור לחוץ	private
_repeatTimer	Timer?	לוחץ כל פעם מחדש שהכפתור לחוץ	private, nullable

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
_startPress		void	פונקציה הנקראת בתחילת לחיצה	private
_endPress		void	פונקציה הנקראת בסוף לחיצה	private
dispose		void	משחררת תכונות בסוף state	override
build	BuildContext build	Widget	יוצר כפתור עם אייקון בפנים	override

End Game Buttons

שם המחלקה: EndGameButtons

סוג מחלקה: **Stateful Widget**

הורשה: Stateful Widget

תפקיד: כפתורי סיום משחק (כפתור אתחול משחק, כפתור יציאה)

תכונות:

הערות	תיאור	סוג	תכונה
final	גודל הכפתור	double	size
final	פעולה הנקראת בלחיצה על כפתור אתחול משחק	VoidCallback	onRetry
final	פעולה הנקראת בלחיצה על כפתור יציאה	VoidCallack	onExit

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor, const	מאתחל את הכפתורים	EndGameButtons	super.key, required this.size, required this.onRetry, required this.onExit	EndGameButtons
override	יוצר את המצב של המחלקה	State<EndGameButtons>		createState

End Game Buttons State

שם המחלקה: EndGameButtonsState

סוג מחלקה: State of Stateful Widget

הורשה: <State<EndGameButtons

תפקיד: מצב של כפתורי הסיום משחק

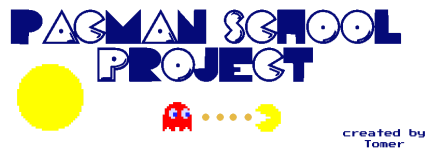
תכונות:

תכונה	סוג	תיאור	הערות
_isRetryPressed	bool	האם כפתור האתחול לחוץ	private
_isExitPressed	bool	האם כפתור היציאה לחוץ	private

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
_startRetryPress		void	כפתור ה Retry התחיל להילחץ	private
_endRetryPress		void	כפתור ה Retry הפסיק להילחץ	private
_startExitPress		void	כפתור ה Exit התחיל להילחץ	private

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
private	כפתור ה Exit הפסיק להילחץ	void		_endExitPress
override	יוצר כפתור Retry וכפתור Exit	Widget	BuildContext build	build

Ghost

שם המחלקה: Ghost

סוג מחלקה: Abstract ▾

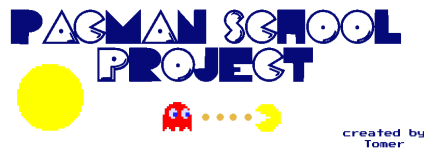
הורשה: Animated Character

תפקיד: רוח שרודפת אחרי pacman

תכונות:

הערות	תיאור	סוג	תכונה
	המצב של הרוח	GhostState	state
abstract	מיקום בו הרוחות ילחו במצב scatter	(int x, int y)	scatterLocation
	האם הרוח התחילה מצב frightened	bool	hasStartedFrightened Mode
final	תכונה שמביאה ערכים רנדומליים	Random	_random
	מהירות הרוחות	double	speed
	מיקום התחלתי של הרוחות	(int x, int y)	startingPosition

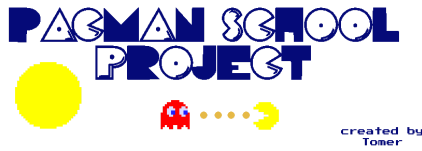
Pacman Android Edition



תומר כץ

הערות	תיאור	סוג	תכונה
late	אנימציה למעלה	SpriteAnimation	up
late	אנימציה ימינה	SpriteAnimation	right
late	אנימציה למטה	SpriteAnimation	down
late	אנימציה שמאלה	SpriteAnimation	left
late	אנימציה נאכל למעלה	SpriteAnimation	eatenUp
late	אנימציה נאכל ימינה	SpriteAnimation	eatenRight
late	אנימציה נאכל למטה	SpriteAnimation	eatenDown
late	אנימציה נאכל שמאלה	SpriteAnimation	eatenLeft

Pacman Android Edition



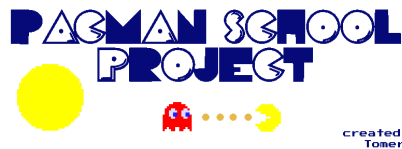
תומר כץ

תכונה	סוג	תיאור	הערות
frightened0	SpriteAnimation	אנימציה נבהל 0	late
frightened1	SpriteAnimation	אנימציה נבהל 1	late
idle	SpriteAnimation	אנימציה בטלה	late
_currentEyeAnimation	SpriteAnimation?	אנימציה נוכחית	private nullable
animations	List<SpriteAnimation>	רשימת האנימציות	override getter
onDeath	void Function()?	פעולה שתקרה כאשר הרוח תהרוג	final nullable
onEaten	void Function()?	פעולה שתקרה כאשר הרוח תאכל	final nullable

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor	מאתכל את הרוח	Ghost	required super.GameManager, this.onDeath, this.onEaten	Ghost
override	מחזיר תצוגה של הרוח	Widget	double tileSize, {Color? color}	getSprite
	מעדכן את המצב של הרוח	void		updateState
override	מאתחל את הרוח למקום ההתחלתי ומצב התחלתי	void		reset
abstract	תזוזה במצב chase	void	double dt, double pacmanX, double pacmanY, Direction pacmanDirection	chase
	תזוזה במצב scatter	void	double dt	scatter

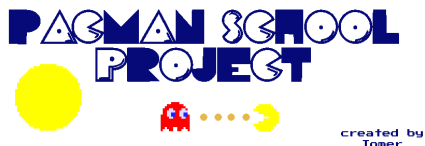
Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
	תזוזה במצב frightened	void	double dt	frightened
	תזוזה במצב eaten	void	double dt	eaten
override	תזוזה של הרוח הולכת לפונקציה על פי המצב הנוכחי	void	double dt	move
override	בוחר ומעדכן את האנימציה הנוכחית	void	double dt	updateAnimation
override	פונקציה שמעדכנת את מצב הרוח	void	double dt	update
	מחזיר הכיוון שהרוח צריכה ללכת	Direction?	int tx, int ty, int cx, int cy	ghostChase
	מתקדם לכיוון הנוכחי צעד אחד	void	double dt	stepForward
	מחזיר הכיוון שהרוח צריכה ללכת בחישוב פשוט	Direction?	Int tx, int ty, int cx, int cy	simpleChase

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
	מחזיר הכיוון שהרוח צריכה ללכת בחישוב מסובך	Direction?	int tx, int ty, int cx, int cy	complexChase

Ghost State

שם המחלקה: GhostState

סוג מחלקה: Enum

הורשה: Enum

תפקיד: מצבים שהרוחות יכולות להיות

אופציות:

הערות	תיאור	אופציה
	מעקב אחרי pacman	chase
	מעקב לנקודה מסויימת שונה בין רוח לרוח	scatter
	מצב נבהל מ pacman התחלה	frightened0
	מצב נבהל מ pacman סוף	frightened1
	מצב נאכל על ידי pacman	eaten
	מצב בטלה	idle

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
-------	-------	-------	-----------	-------

Blinky

שם המחלקה: Blinky

סוג מחלקה: רגיל

הורשה: Ghost

תפקיד: הרוח Blinky

תכונות:

הערות	תיאור	סוג	תכונה
override	שם הרוח	String	name
override	כיוון נוכחי	Direction	currentDirection
override	כיוון הבא	Direction	nextDirection
override	ערך ה x במערכת הצירים של המשחק	double	x
override	ערך ה y במערכת הצירים של המשחק	double	y
override getter	אנימציה בטלה	SpriteAnimation	idle
override	אזור ההתחלה של blinky	(int x, int y)	startingLocation
override late	אזור ה scatter של blinky	(int, int)	scatterLocation

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor	מאתחל את Blinky	Blinky	required super.gameManager, super.onDeath, super.onEaten	Blinky
	עוקב אחרי Pacman ישירות	void	double dt, double pacmanX, double pacmanY, Direction pacmanDirection	chase

Pinky

שם המחלקה: Pinky

סוג מחלקה: רגיל

הורשה: Ghost

תפקיד: הרוח Pinky

תכונות:

תכונה	סוג	תיאור	הערות
name	String	שם הרוח	override
currentDirection	Direction	כיוון נוכחי	override
nextDirection	Direction	כיוון הבא	override
x	double	ערך ה x במערכת הצירים של המשחק	override
y	double	ערך ה y במערכת הצירים של המשחק	override
idle	SpriteAnimation	אנימצית בטלה	override getter
startingLocation	(int x, int y)	אזור ההתחלה של Pinky	override
scatterLocation	(int, int)	אזור ה scatter של Pinky	override late

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor	מאתחל את Pinky	Pinky	required super.gameManager, super.onDeath, super.onEaten	Pinky
override	עוקב אחרי pacman 4 צעדים קדימה	void	double dt, double pacmanX, double pacmanY, Direction pacmanDirection	chase

Inky

שם המחלקה: Inky

סוג מחלקה: רגיל

הורשה: Ghost

תפקיד: הרוח Inky

תכונות:

הערות	תיאור	סוג	תכונה
override	שם הרוח	String	name
override	כיוון נוכחי	Direction	currentDirection
override	כיוון הבא	Direction	nextDirection
override	ערך ה x במערכת הצירים של המשחק	double	x
override	ערך ה y במערכת הצירים של המשחק	double	y
override getter	אנימצית בטלה	SpriteAnimation	idle
override	אזור ההתחלה של Inky	(int x, int y)	startingLocation
override late	אזור ה scatter של Inky	(int, int)	scatterLocation
static	מיקום נוכחי של Blinky	(double, double)	blinkyLocation

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor	מאתחל את Inky	Inky	required super.gameManager, super.onDeath, super.onEaten	Inky
override	עוקב אחרי הממוצע של מיקום נוכחי של Blinky ומטרה של Pinky	void	double dt, double pacmanX, double pacmanY, Direction pacmanDirection	chase

Clyde

שם המחלקה: Clyde

סוג מחלקה: רגיל

הורשה: Ghost

תפקיד: הרוח Clyde

תכונות:

תכונה	סוג	תיאור	הערות
name	String	שם הרוח	override
currentDirection	Direction	כיוון נוכחי	override
nextDirection	Direction	כיוון הבא	override
x	double	ערך ה x במערכת הצירים של המשחק	override
y	double	ערך ה y במערכת הצירים של המשחק	override
idle	SpriteAnimation	אנימציה בטלה	override getter
startingLocation	(int x, int y)	אזור ההתחלה של Clyde	override
scatterLocation	(int, int)	אזור ה scatter של Clyde	override late

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor	מאתחל את Clyde	Clyde	required super.gameManager, super.onDeath, super.onEaten	Clyde
override	עוקב אחר pacman ירישות עד שנהיה יותר מדי קרוב	void	double dt, double pacmanX, double pacmanY, Direction pacmanDirection	chase

Game Map

שם המחלקה: GameMap

סוג מחלקה: רגיל

הורשה:

תפקיד: מפת המשחק

תכונות:

תכונה	סוג	תיאור	הערות
map	List<List<int>>	מפת המבוך במערך דו מימדי של int, עוקב אחר המספרים הסידוריים	
kEmpty	int	המספר הסידורי של אזור ריק במבוך	static const
kWall	int	המספר הסידורי של קיר במבוך	static const
kSmallDot	int	המספר הסידורי של נקודה במבוך	static const
kSuperDot	int	המספר הסידורי של סופר נקודה במבוך	static const
kBonus	int	המספר הסידורי של אזור הבונוס במבוך	static const
kTunnel	int	המספר הסידורי של אזור המעבר במבוך	static const

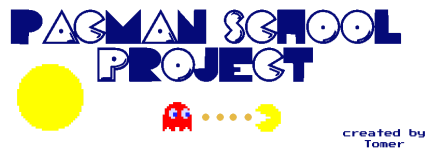
תכונה	סוג	תיאור	הערות
kGhostHouseDoor	int	המספר הסידורי של דלת בית הרוחות	static const
kGhostHouseInterior	int	המספר הסידורי של בתוך בית הרוחות	static const
kStartingPosition	int	המספר הסידורי של אזור ההתחלה במבוך	static const
mapAssetPath	String	כתובת התמונה של המבוך	final
kCols	int	כמות הטורים במבוך	final
kRows	int	כמות השורות במבוך	final
startingX	double	ערך x של אזור ההתחלה	final
startingY	double	ערך y של אזור ההתחלה	final
ghostHouseX	int	ערך x של בית הרוחות	final
ghostHouseY	int	ערך y של בית הרוחות	final
bonusX	double	ערך x של אזור הבונוס	final

תכונה	סוג	תיאור	הערות
bonusY	double	ערך y של אזור הבונוס	final
dots	List<Dot>	רשימת הנקודות במבוך	late final
superDots	List<SuperDot>	רשימת הסופר נקודות במבוך	late final
onDotEaten	void Function()	פעולה כאשר אוכלים נקודה	final
onSuperDotEaten	void Function()	פעולה כאשר אוכלים סופר נקודה	final
remainingDots	int	מספר הנקודות שנאר	late

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
GameMap	Required this.map, required this.onDotEaten, required this.onSuperDotEaten	GameMap	מאתכל את GameMap	constructor

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
private	פונקציית התחלה של GameMap	void		_init
	איפוס של מפת המשחק	void		reset
	בודק אם בכיוון הנתון, במיקום הנתון יש קיר	bool	int col, int row, Direction direction, {bool? isPacman}	isWall

Map Pallet

שם המחלקה: MapPallet

סוג מחלקה: **Stateless Widget**

הורשה: StatelessWidget

תפקיד: מפת משחק צבוע

תכונות:

תכונה	סוג	תיאור	הערות
gameMap	GameMap	מפת המשחק	final

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
MapPallet	super.key, required this.gameMap	MapPallet	מאתחל את הטבלת צבעים	const constructor
build	BuildContext context	Widget	מרנדר את המפה עם צבעים מיוחדים במקום תמונה	override
_computeTileSize	Size screenSize	double	מחשב את גודל ה tile הטוב ביותר על פי גודל המסך	private

Maze Painter

שם המחלקה: Maze Painter

סוג מחלקה: רגיל

הורשה: Custom Painter

תפקיד: צייר של המבוך (עוזר ל Map Pallet)

תכונות:

תכונה	סוג	תיאור	הערות
gameMap	GameMap	מפת המשחק	final
_wallColor	Color	צבע של קיר	static const private
_smallDotColor	Color	צבע של נקודה	static const private
_superDotColor	Color	צבע של סופר נקודה	static const private
_bonusColor	Color	צבע של אזור בונוס	static const private
_tunnelColor	Color	צבע של אזור התעלה	static const private
_ghostHouseDoorColor	Color	צבע של דלת בית הרוחות	static const private
_ghostHouseInteriorColor	Color	צבע של בית הרוחות	static const private
_startingPositionColor	Color	צבע של המיקום ההתחלתי	static const private

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
const constructor	מאתחל את הצייר	MazePainter	super.key, required this.gameMap	MazePainter
override	מצייר את כל המבוך על פי המספרים הסידוריים	void	Canvas canvas, Size size	paint
override	בודק אם צריך לצייר מחדש	bool	covariant CustomPainter oldDelegate	shouldRepaint
private	מצייר tile אחד	void	Canvas canvas, { required int row, required int col, required double tileSize}	_paintTile
private	מצייר ריבוע	void	Canvas canvas, { required int left, required int top, required double tileSize, required Color color}	_drawRect

הערות	תיאור	מחזיר	Parameter	פעולה
private	מצייר עיגול	void	Canvas canvas, { required Offset center, required double radius, required Color color}	_drawCircle

Bonus

שם המחלקה: Bonus

סוג מחלקה: רגיל

הורשה:

תפקיד: משרת בונוס במשחק

תכונות:

תכונה	סוג	תיאור	הערות
name	String	שם	final
isVisible	bool	האם צריך להיות נראה	
currentType	BonusType	הסוג הנוכחי של ה Bonus	
_random	Random	אובייקט רנדומלי	final
gameManager	GameManager	מנהל המשחק של הבונוס	final
x	double	ערך ה x במערכת הצירים של המשחק	late
y	double	ערך ה y במערכת הצירים של המשחק	late
onEaten	void Function(BonusType bonusType)?	פעולה שתקרה כאשר הבונוס יאכל	final nullable

תכונה	סוג	תיאור	הערות
bonusAssetsPath	String	תחילת כתובת של הבונוסים	static const
bonusesAssetsEndFile	String	סיום הכתובת של הבונוסים	static const

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
Bonus	required this.gameManager, required this.onEaten	Bonus	מתחיל את Bonus	constructor
_getPacman		Pacman	מחזיר את pacman	private
update	double dt	void	מעדכן את bonus	
setVisible		void	מראה את הבונוס	
getSprite	double tileSize	Widget	מחזיר את התמונה של הבונוס הנוכחי אם נראה	

Bonus Type

שם המחלקה: BonusType

סוג מחלקה: Enum

הורשה: Enum

תפקיד: סוגי הבונוס

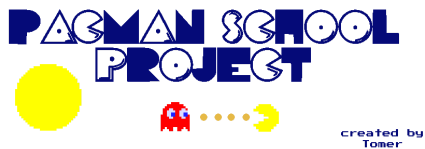
אופציות:

הערות	תיאור	אופציה
	תפוח	apple
	פעמון	bell
	דובדבן	cherry
	galaxian	galaxian
	מפתח	key
	מלון	melon
	תפוז	orange
	תות	strawberry

תכונות:

הערות	תיאור	סוג	תכונה
final	צבע הרקע המומלץ לבונוס	Color	bgColor

Pacman Android Edition



תומר כץ

תכונה	סוג	תיאור	הערות
borderColor	Color	צבע border המומלץ לבונוס	final

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
-------	-----------	-------	-------	-------

Direction

שם המחלקה: Direction

סוג מחלקה: Enum

הורשה: Enum

תפקיד: סוגי כיוונים

אופציות:

הערות	תיאור	אופציה
	למעלה	up
	למטה	down
	שמאלה	left
	ימינה	right

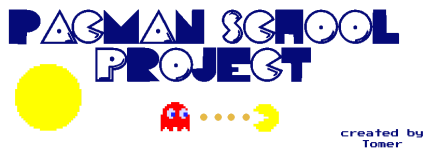
תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
	מחזיר כיוון הפוך	Direction		opposite
	מחזיר בתור מטריצה של כיוון	(int, int)		toDelta

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
	זווית ברדיאנים של הכיוון	double		toAngle

Dot

שם המחלקה: Dot

סוג מחלקה: **Stateless Widget**

הורשה: StatelessWidget

תפקיד: נקודה במשחק

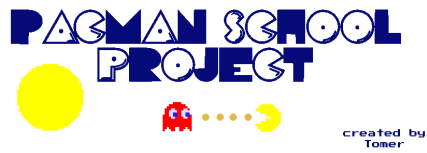
תכונות:

הערות	תיאור	סוג	תכונה
final	ערך ה x במערכת הצירים של המשחק	int	x
final	ערך ה y במערכת הצירים של המשחק	int	y
private	האם הנקודה נאכלה	bool	_isEaten
final	פעולה שתקרה כאשר תאכל	void Function()	onEaten

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor	מאתחל את Dot	Dot	this.x, this.y, {super.key, required this.onEaten}	Dot

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
override	מרנדר נקודה קטנה	Widget	BuildContext context	build
	מרנדר את הנקודה במקום שלה אם נראית	Widget	double tileSize	showOnStack
	בודק אם הנקודה נאכלה, אם כן קורה ל (<code>eat</code>)	void	double pacmanX, double pacmanY	checkIfEaten
private	פונקציה כאשר נאכלה הנקודה	void		<code>_eat</code>
	מאפסת את הנקודה	void		reset

Game Manager

שם המחלקה: GameManager

סוג מחלקה: רגיל

הורשה:

תפקיד: מנהל המשחק

תכונות:

תכונה	סוג	תיאור	הערות
score	int	ניקוד המשחק	
highScore	int	שיא המשתמש	
screenNum	int	מספר מסך	
nextGhostEatenScore	int	הניקוד הבא של אכילת רוח	
_gameTimers	List<GameTimer>	טיימרי המשחק	late final
gameMap	GameMap	מפת המשחק	late final
pacman	Pacman	פקמן	late final
isGameOver	bool	האם המשחק נגמר	
controller	Controller	שלט המשחק	late final
currentScreen	GameScreen	מסך המשחק הנוכחי	late
endGameButtons	EndGameButtons	כפתורי סוף המשחק	late
isPaused	bool	האם המשחק נעצר	
ghosts	List<Ghost>	רשימת כל הרוחות	late final

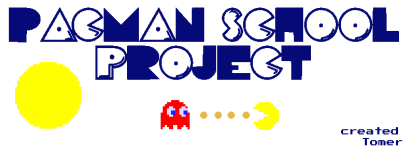
תכונה	סוג	תיאור	הערות
lives	int	מספר החיים שנשארו	
_gameMessage	Widget Function(double tileSize)	הודעת המשחק (הודעה בהתחלה ובסוף)	private
dt	double	קצב עדכון הלולאה המרכזית	final
_periodTimer	Timer	שעון הלולאה המרכזי	late
_stopWatch	StopWatch	שעון של כמה זמן עבר במשחק	late
_lastTickTime	double	הבדל הזמן בין כל סייקל	
bonusesTaken	List<BonusType>	רשימת בונוסים שנלקחו	late
bonus	Bonus	הבונוס	late
_bonusTimer	GameTimer	שעון למתי להראות בונוס	private
onGameOver	void Function(int score)	פעולה שתקרה כאשר המשחק מסתיים	late
onRetry	void Function()	פעולה שתקרה כאשר המשתמש ירצה לשחק מחדש	late
_getHighScoreCallbac k	int Function()?	מחזיר את השיא של המשתמש מהמציג	nullable

תכונה	סוג	תיאור	הערות
onExit	void Function()	פעולה שתקרה כאשר המשתמש ירצה לצאת מהמשחק	late
onGettingBonus	void Function(BonusType)	פעולה שתקרה כאשר משיגים בונוס	late
_audioPlayer	AudioPlayer	נגן מוזיקה של המשחק	final private
_hasUpdatedNextScreen	bool	האם המשחק החדש עודכן	private
_currentAudioState	_AudioState	מצב המוזיקת רקע	private

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
_onDeath		void	פעולה שתקרה כאשר pacman ימות	private
_onGhostEaten		void	פעולה שתקרה כאשר רוח תאכל	private
_onBonusEaten	BonusType type	void	פעולה שתקרה כאשר הבונוס יאכל	private
initGame		void	פעולה שמתחילה את המשחק	

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
	פונקציית הלולאה הראשית של המשחק	void	double dt	_update
Constructor	מתחיל את המנהל משחק	GameManager	bool? start, int Function()? highScore, void Function(int score)? onGameOver, void Function()? onRetry, void Function()? onExit, void Function(BonusType bonus)? onGettingBonus	GameManager
private	פעולה שנקראת כאשר נאכל נקודה	void		_onDotEaten
private	פעולה שנקראת כאשר נאכל סופר נקודה	void		_onSuperDotEaten
private	פעולה שנקראת כאשר נגמר המשחק	void		_triggerGameOver

Audio State

שם המחלקה: _AudioState

סוג מחלקה: Enum

הורשה: Enum

תפקיד: סוגי המוזיקה ברקע של המשחק

אופציות:

הערות	תיאור	אופציה
	בטלה	idle
	מוזיקה רגילה	normal
	מוזיקה כאשר הרוחות נבהלות	blue
	מוזיקה כאשר הרוחות נאכלות	eye

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
-------	-------	-------	-----------	-------

Game Screen

שם המחלקה: GameScreen

סוג מחלקה: Stateful Widget

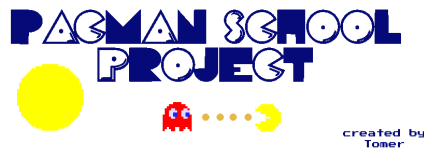
הורשה: Stateful Widget

תפקיד: מסך המשחק

תכונות:

תכונה	סוג	תיאור	הערות
gameMessage	Widget Function(double tileSize)	הודעת המשחק (הודעה בהתחלה ובסוף)	final
controller	Controller	שלט המשחק	final
endGameButtons	EndGameButtons	כפתורי סוף המשחק	final
isGameOver	bool Function()	פונקציית האם המשחק נגמר	final
pacman	Pacman	פקמן	final
ghosts	List<Ghost>	הרוחות	final
bonus	Bonus	הבונוס	final
bonusesTakedn	List<BonusType>	רשימת הבונוסים שנלקחו	final
gameMap	GameMap	מפת המשחק	final

Pacman Android Edition



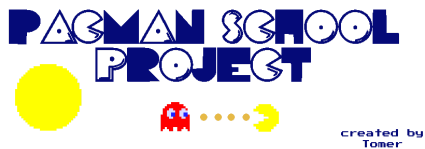
תומר כץ

תכונה	סוג	תיאור	הערות
getScore	int Function()	פונקציה להשיג את הניקוד	final
getLives	int Function()	פונקציה להשיג את כמות החיים	final
getHighScore	int Function()	פונקציה להשיג את השיא של המשתמש	final
dt	double	זמן סייקל בשביל עדכון רציף גם של המסך	final

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
const constructor	מאתחל את מסך המשחק	GameScreen	super.key, required this.gameMessage, required this.controller, required this.gameMap, required this.pacman, required this.getScore, required this.getLives, required this.getHighScore, required this.ghosts, required this.bonus, required this.dt, required this.bonusesTaken, required this.endGameButtons, required this.isGameOver	GameScreen

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
override	יוצר מצבים למסך משחק	State<GameScreen>		createState

Game Screen State

שם המחלקה: GameState

סוג מחלקה: **State of Stateful Widget**

הורשה: <State<GameScreen

תפקיד: מצב של מסך המשחק

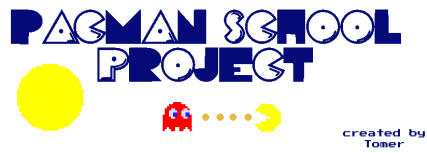
תכונות:

הערות	תיאור	סוג	תכונה
nullable private	מעדכן את המסך מחזורית	Timer?	_uiTimer
nullable private	צבע מיוחד ל pacman	Color?	_customPacmanColor
nullable private	צבע מיוחד ל Blinky	Color?	_customBlinkyColor
nullable private	צבע מיוחד ל Pinky	Color?	_customPinkyColor
nullable private	צבע מיוחד ל Inky	Color?	_customInkyColor
nullable private	צבע מיוחד ל Clyde	Color?	_customClydeColor
nullable private	צבע מיוחד למפה	Color?	_customMapColor

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	מרנדר את המסך של המשחק עם כל האלמנטים	Widget	BuildContext context	build
override	מאתחל את ה state	void		initState
async private	לוקח את הצבעים מהמידע השמור	void		_loadCustomColors
private	מחזיר את הצבע של הרוח	Color?	String name	_getGhostcolor
override	מנקה תכונות כשמסיימים להשתמש בהם	void		dispose

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
private	מרנדר את ה HUD	Widget		_buildHUD
private	בונה אייקונים של Pacman על פי כמות החיים	Widget	int lives	_buildLivesIndicator
private	בונה רשימה של כל הבונוסים שנאספו במשחק הנוכחי	Widget	List<BonusType> bonuses	_buildBonusStrip
private	טור של שם ואז ניקוד	Widget	String label, int value	_buildScoreColumn

Game Timer

שם המחלקה: GameTimer

סוג מחלקה: רגיל

הורשה:

תפקיד: טיימר בזמן משחק

תכונות:

הערות	תיאור	סוג	תכונה
final nullable	פעולה שנקראת בסיום הטיימר	void Function()?	onEnd
final	הזמן שרוצים להגיע אליו בטיימר	double	duration
private	שעון של הטיימר שסופר קדימה עד שמגיע ל duration	double	_clock
	האם הטיימר פועל	bool	isOn
	האם הטיימר מחזורי	bool	isLoop

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor	מאתחל טיימר משחק	GameTimer	this.duration, { required this.onEnd, this.isOn, this.isLoop }	GameTimer
	מעדכן את השעון של הטיימר	void	double dt	update
	מתחיל את הטיימר	void		start
	עוצר את הטיימר	void		pause
private	מאפס את הטיימר	void		_reset

Pacman

שם המחלקה: Pacman

סוג מחלקה: רגיל

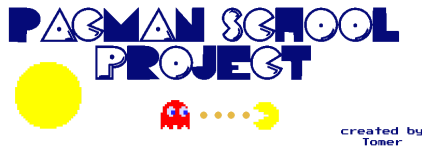
הורשה: Animated Charecter

תפקיד: פקמן

תכונות:

הערות	תיאור	סוג	תכונה
static const	שם	String	_kName
override	כיוון נוכחי	Direction	currentDirection
override	כיוון הבא	Direction	nextDirection
override	ערך ה x במערכת הצירים במשחק	double	x
override	ערך ה y במערכת הצירים במשחק	double	y
static const	מהירות של pacman	double	_kSpeed
late	אנימצית מוות	SpriteAnimation	_deathAnimation
late	אנימצית בטלה	SpriteAnimation	_idleAnimation
late	אנימצית תזוזה למעלה	SpriteAnimation	_upAnimation
late	אנימצית תזוזה ימינה	SpriteAnimation	_rightAnimation
late	אנימצית תזוזה למטה	SpriteAnimation	_downAnimation
late	אנימצית תזוזה שמאלה	SpriteAnimation	_leftAnimation

Pacman Android Edition



תומר כץ

תכונה	סוג	תיאור	הערות
animations	List<SpriteAnimation>	רשימת האנימציות	override
name	String	מעדכן את השם של super	override

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
Pacman	required super.gameManager	Pacman	מאתחל את Pacman	constructor
updateAnimation	double dt	void	בוחר ומעדכן אנימציות	override
move	double dt	void	מזיז את pacman	override
reset		void	מאפס את Pacman לנקודת התחלה	override
nextDirectionarrow	double tileSize, {Color? color}	Widget	מרנדר חץ המראה מה הכיוון הבא של Pacman	
playDeath		void	מפעיל אנימצית מוות	
updateDeathAnimation	double dt	void	מעדכן אנימצית מוות	

Super Dot

שם המחלקה: SuperDot

סוג מחלקה: **Stateless Widget**

הורשה: Stateless Widget

תפקיד: סופר נקודה

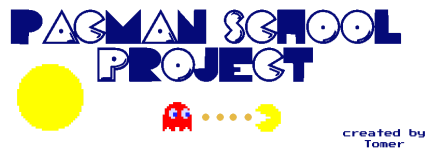
תכונות:

תכונה	סוג	תיאור	הערות
x	int	ערך ה x במערכת הצירים במשחק	final
y	int	ערך ה y במערכת הצירים במשחק	final
_isEaten	bool	האם הסופר נקודה נאכלה	private
onEaten	void Function()	פקודה שפועלת כאשר הסופר נקודה נאכלת	final

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
SuperDot	this.x, this.y, { super.key, required this.onEaten }	SuperDot	מאתחל את הסופר נקודה	constructor

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה נקודה יחסית גדולה	Widget	BuildContext context	build
	מראה את הסופר נקודה במקום שלה אם לא נאכלה	Widget	double tileSize	showOnStack
	בודק אם Pacman אכל את הסופר נקודה	void	double pacmanX, double pacmanY	checkIfEaten
private	פעולה הנקראת כאשר הנקודה נאכלה	void		_eat
	מאפס את הסופר נקודה	void		reset

Collectables Screen

שם המחלקה: CollectablesScreen

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: עוקב אחרי הבונוסים

תכונות:

תכונה	סוג	תיאור	הערות
-------	-----	-------	-------

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
CollectablesScreen	super.key	CollectablesScreen	מאתחל את המסך	const constructor
createStates		State<CollectablesScreen	יוצר מצבים	override

Collectables Screen State

שם המחלקה: CollectablesScreenState

סוג מחלקה: **State of Stateful Widget**

הורשה: <State> Collectables Screen

תפקיד: מצב של מסך הבונוסים

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה את המסך עם כל ה collectables	Widget	BuildContext context	build
override	מנקה את כל התכונות שלא בשימוש	void		dispose

Bonus Card

שם המחלקה: BonusCard

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: בעצם כפתור / card שיש לכל בונוס במסך בונוסים

תכונות:

תכונה	סוג	תיאור	הערות
i	int	של הבונוס index	final

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
BonusCard	super.key	BonusCard	מאתחל את הכרטיס	const constructor
createStates		State<BonusCard >	יוצר מצבים	override

Bonus Card State

שם המחלקה: BonusCardState

סוג מחלקה: State of Stateful Widget

הורשה: Bonus Card <State>

תפקיד: מצב של כרטיס של בונוס

תכונות:

תכונה	סוג	תיאור	הערות
_assetDir	String	תחילת הכתובת של כל הבונוסים	static const private
_assetExt	String	סוף הכתובת של כל הבונוסים	static const private
_isPressed	bool	האם הכפתור נלחץ	private
_isTimerFinished	bool	האם הטיימר של הלחיצה סיים	private
_bonus	bonusType	סוג הבונוס של הקלף	late private
_haveCollected	bool	האם המשתמש השיג את הבונוס הזה	private
_collectedAmount	int	מספר הפעמים שהמשתמש השיג את הבונוס	private

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה את הקלף של הבונוס	Widget	BuildContext context	build
override	מאתחל את המצבים	void		initState
	הדיאלוג של אותו בונוס	void	BuildContext context	bonusDialog
	תמונה של הבונוס	Image	double scale	image

Home Screen

שם המחלקה: HomeScreen

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: מסך הבית

תכונות:

תכונה	סוג	תיאור	הערות
-------	-----	-------	-------

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
HomeScreen	super.key	HomeScreen	מאתחל את המסך	const constructor
createStates		State<HomeScreen>	יוצר מצבים	override

Home Screen State

שם המחלקה: HomeScreenState

סוג מחלקה: State of Stateful Widget

הורשה: <State> Home Screen

תפקיד: מצב של מסך הבית

תכונות:

הערות	תיאור	סוג	תכונה
private	התקדמות בבונסים	double	_progress
final private	נגן המוזיקה של המרכזי	AudioPlayer	_audioPlayer

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה את מסך הבית	Widget	BuildContext context	build
override	מאתחל את המצבים	void		initState
private	משיג את אחוז התקדמות במציאת כל הבונסים	void		_getBonusProgress
override	מנקה את כל התכונות שלא בשימוש	void		dispose

Leaderboard Screen

שם המחלקה: LeaderboardScreen

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: מסך דירוגים

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
const constructor	מאתחל את המסך	LeaderboardScreen	super.key	LeaderboardScreen
override	יוצר מצבים	State<LeaderboardScreen>		createStates

Leaderboard Screen State

שם המחלקה: LeaderboardScreenState

סוג מחלקה: **State of Stateful Widget**

הורשה: <State> Leaderboard Screen

תפקיד: מצב של מסך דירוגים

תכונות:

תכונה	סוג	תיאור	הערות
_users	List<GameUser>	רשימה של כל המשתמשים	final

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
build	BuildContext context	Widget	בונה את מסך הדירוגים	override
initState		void	מאתחל את המצבים	override
_loadUsers		void	מקבל את כל המשתמשים מבסיס הנתונים	async private
dispose		void	מנקה את כל התכונות שלא בשימוש	override

הערות	תיאור	מחזיר	Parameter	פעולה
private	מחזיר גביע ל 3 מקומות ראשונים ומספר ל 4 ומטה	Widget		_rankBadge

Login

שם המחלקה: Login

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: מסך הכניסה

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
const constructor	מאתחל את המסך	Login	super.key	Login
override	יוצר מצבים	State<Login>		createStates

Login State

שם המחלקה: LoginState

סוג מחלקה: State of Stateful Widget

הורשה: <State> Login

תפקיד: מצב של מסך הכניסה

תכונות:

הערות	תיאור	סוג	תכונה
private	שדה אימייל controller	TextEditingController	_emailController
private	שדה סיסמא controller	TextEditingController	_passwordController
private	האם הכניסה דרך גוגל נטענת	bool	_googleLoading

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה את מסך הכניסה	Widget	BuildContext context	build
override	מאתחל את המצבים	void		initState
async private	בודק האם הייתה כניסה שמורה פנימית אחרונה	void		_checkPrefs

הערות	תיאור	מחזיר	Parameter	פעולה
override	מנקה את כל התכונות שלא בשימוש	void		dispose
async private	משיג איזה בונסים היו שמורים למשתמש האחרון	Future<MapEntry<BonusType, int>>	BonusType bonus, SharedPreferences prefs	_bonusTablePrefs
async private	שומר את הפרטים של המשתמש במכשיר	void	GameUser user	_savePrefs
async private	מחיקת הפרטים של המשתמש במכשיר	void		_removePrefs
async private	כניסה דרך גוגל	void		_signInWithGoogle

Settings

שם המחלקה: Settings

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: מסך הגדרות של שינוי צבע

תכונות:

תכונה	סוג	תיאור	הערות
-------	-----	-------	-------

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
Settings	super.key	Settings	מאתחל את המסך	const constructor
createStates		State<Settings>	יוצר מצבים	override

Settings State

שם המחלקה: SettingsState

סוג מחלקה: State of Stateful Widget

הורשה: <State> Settings Screen

תפקיד: מצב של מסך הגדרות

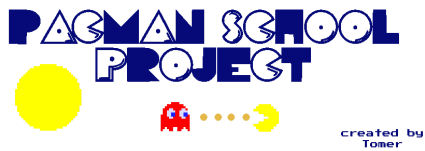
תכונות:

תכונה	סוג	תיאור	הערות
_pacmanPickedColor	Color	הצבע הנבחר של Pacman	private
_blinkyPickedColor	Color	הצבע הנבחר של Blinky	private
_pinkyPickedColor	Color	הצבע הנבחר של Pinky	private
_inkyPickedColor	Color	הצבע הנבחר של Inky	private
_clydePickedColor	Color	הצבע הנבחר של Clyde	private
_mapPickedColor	Color	הצבע הנבחר של המפה	private

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה את מסך ההגדרות	Widget	BuildContext context	build
override	מאתחל את המצבים	void		initState
async private	משיג את הצבעים מהמידע השמור במכשיר	void		_loadColors
override	מנקה את כל התכונות שלא בשימוש	void		dispose
private	מעביר מספר מ 0-1 ל 0-255	int	double x	_floatToInt8
private	מחזיר את הצבע הנבחר של הדמות	Color	String character	_getCharacterColor
async private	שומר לאותה דמות את הצבע במכשיר	void	String character	_saveColor

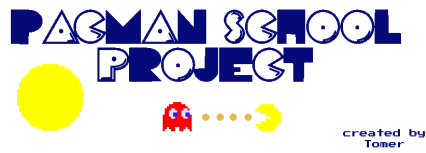
Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
async private	פותח דיאלוג של בחירת צבע לדמות הרצויה	void	BuildContext context, String character	_showColorDialog
private	מאפס את הצבע של Pacman	void		_resetPacmanColor
private	מאפס את הצבע של Blinky	void		_resetBlinkyColor
private	מאפס את הצבע של Pinky	void		_resetPinkyColor
private	מאפס את הצבע של Inky	void		_resetInkyColor
private	מאפס את הצבע של Clyde	void		_resetClydeColor
private	מאפס את הצבע של המפה	void		_resetMapColor

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
private	מאפס את הצבע של הדמות הנבחרת	void	String character	_resetColor
private	יוצר שורה של כפתור לבחור צבע ואיפוס לאותה דמות	Widget	String character	_colorPickerRow
private	לוקח את הצבע מבסיס הנתונים של הדמות	Color?	SharedPreferences prefs, String character	_readColorPrefs

Signup

שם המחלקה: Signup

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: מסך הרשמה

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
const constructor	מאתחל את המסך	Signup	super.key	Signup
override	יוצר מצבים	State<Signup>		createStates

Signup State

שם המחלקה: SignupState

סוג מחלקה: State of Stateful Widget

הורשה: <State> Signup

תפקיד: מצב של מסך הרשמה

תכונות:

תכונה	סוג	תיאור	הערות
_acceptedTerms	bool	האם המשתמש אישור תנאי שימוש	private
_isGoogleLoading	bool	האם כניסה דרך גוגל נטענת	private
_nameController	TextEditingController	של שדה שם Controller	final private
_emailController	TextEditingController	של שדה אימייל Controller	final private
_passwordController	TextEditingController	של שדה סיסמא Controller	final private
_message	String	הודעת שגיאה	private
_canContinue	bool	האם אפשר להירשם או שיש שדות ריקים	private getter

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה את מסך ההרשמה	Widget	BuildContext context	build
async private	שומר את הפרטים של המשתמש במכשיר	void	GameUser user	_saveUserPreferences
async private	הרשמה דרך גוגל	void		_signUpWithGoogle
override	מנקה את כל התכונות שלא בשימוש	void		dispose
async private	בודק אם משתמש חדש להרשמה דרך גוגל	Future<bool>	String uid	_isNewUser
async private	יוצר משתמש חדש ב database	void	required String userId, required String name, required String email	_createDatabase User

הערות	תיאור	מחזיר	Parameter	פעולה
async private	מגדיר את המשתמש הנוכחי	void	required String userId, required String name, required String email	_setCurrentUser
private	שולח למסך כניסה	void		_goToLoginScreen
async private	הרשמה דרך אימייל וסיסמא	void		_singUpWithEmail
private	יוצר שדה למידע בשביל ההרשמה	Widget	required String label, required TextEditingController controller, bool? obscureText, TextInputType keyboardType	_buildInputField

[Read Me](#)

שם המחלקה: ReadMe

סוג מחלקה: **Stateful Widget**

הורשה: Stateful Widget

תפקיד: מסך הסבר

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
const constructor	מאתחל את המסך	Signup	super.key	Signup
override	יוצר מצבים	State<Signup>		createStates

Read Me State

שם המחלקה: ReadMeState

סוג מחלקה: State of Stateful Widget

הורשה: <State ReadMe

תפקיד: מצב של מסך הסבר

תכונות:

הערות	תיאור	סוג	תכונה
final private	שלט שמתחבר לאתר אינטרנט	WebViewController	_controller

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה את מסך ההרשמה	Widget	BuildContext context	build
override	מנקה את כל התכונות שלא בשימוש	void		dispose

AppBar

שם המחלקה: AppBar

סוג מחלקה: Stateful Widget

הורשה: AppBar

תפקיד: תפריט עליון

תכונות:

הערות	תיאור	סוג	תכונה
final	כותרת	Text	header
final	ה context של המסך	BuildContext	context
override getter	הכותרת של ה AppBar	Widget?	title
final	האם המשתמש יכול לצאת מהחשבון במסך הזה	bool	canLogout
override getter	כפתורים בצד ה AppBar	List<Widgets>?	actions
override getter	קרבה בציר z של ה AppBar	double?	elevation
override getter	צורת ה AppBar	ShapeBorder?	shape

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
constructor	אתחול של התפריט העליון	AppBar	super.key, required this.context, this.header, this.canLogout	AppBar
async private	מנתק את המשתמש	void		_clearPrefs

Constants

שם המחלקה: Constants

סוג מחלקה: Static <no class>

הורשה:

תפקיד: תכונות קבועות

תכונות:

תכונה	סוג	תיאור	הערות
defaultMap	List<List<int>>	מפת המשחק על פי המספרים הסידוריים	const static
pacmanColor	Color	הצבע הרגיל של Pacman	const static
blinkyColor	Color	הצבע הרגיל של Blinky	const static
pinkyColor	Color	הצבע הרגיל של Pinky	const static
inkyColor	Color	הצבע הרגיל של Inky	const static
clydeColor	Color	הצבע הרגיל של Clyde	const static
mapColor	Color	הצבע הרגיל של המפה	const static
headerTextStyle	TextStyle	סטייל הטקסט של כותרות	const static

תכונה	סוג	תיאור	הערות
anonymousId	String	ה id של משתמש אנונימי	const static
anonymousName	String	השם של משתמש אנונימי	const static
anonymousHighScore	int	השיא של משתמש אנונימי	const static
anonymousEmail	String	האימייל של משתמש אנונימי	const static
anonymousBonusTable	Map<BonusType, int>	ה טבלת בונוסים של משתמש אנונימי	const static

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
-------	-----------	-------	-------	-------

Game Displayer

שם המחלקה: GameDisplayer

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: מרנדר את המשחק

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
const constructor	מאתחל את המסך	GameDisplayer	super.key	GameDisplayer
override	יוצר מצבים	State<GameDisplayer>		createStates

Game Displayer State

שם המחלקה: GameState

סוג מחלקה: **State of Stateful Widget**

הורשה: <State> Game Displayer

תפקיד: מצב של מרנדר המשחק

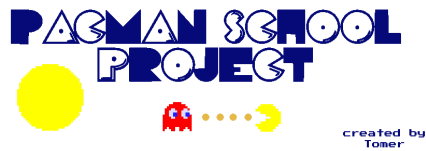
תכונות:

הערות	תיאור	סוג	תכונה
late private	מנהל המשחק	GameManager	_gameManager
late private	משחק חדש כאשר המשתמש רוצה להתחיל מחדש	GameManager	_nextGame
Final private	נגן המוזיקה שיעצור את המוזיקה הרגילה של האפליקציה	AudioPlayer	_audioPlayer

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה את המשחק	Widget	BuildContext context	build
override	מאתחל את המצבים	void		initState

Pacman Android Edition



תומר כץ

הערות	תיאור	מחזיר	Parameter	פעולה
private	פקודה הנקראת כאשר המשחק נגמר	void	int score	_onGameOver
override	מנקה את כל התכונות שלא בשימוש	void		dispose
private	פקודה הנקראת כאשר המשתמש רוצה לשחק מחדש	void		_onRetry
private	פקודה הנקראת כאשר המשתמש רוצה לצאת	void		_onExit
private	פקודה הנקראת כאשר המשתמש השיג בונוס	void	BonusType	onBonusCollected
private	בונה את המשחק עם המנהל	Gamemanager		_buildGameManager

Loading Screen

שם המחלקה: LoadingScreen

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: מסך טעינה

תכונות:

תכונה	סוג	תיאור	הערות
-------	-----	-------	-------

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
LoadingScreen	super.key	LoadingScreen	מתחיל את המסך	const constructor
createStates		State<LoadingScreen>	יוצר מצבים	override

Loading Screen State

שם המחלקה: LoadingScreenState

סוג מחלקה: **State of Stateful Widget**

הורשה: <State> Loading Screen

תפקיד: מצב של מסך טעינה

תכונות:

תכונה	סוג	תיאור	הערות
_timer	Timer	טיימר להעביר למסך הבית	late private final

פעולות:

פעולה	Parameter	מחזיר	תיאור	הערות
build	BuildContext context	Widget	בונה את מסך הטעינה	override
initState		void	מאתחל את המצבים	override
dispose		void	מנקה את כל התכונות שלא בשימוש	override

Sidebar

שם המחלקה: Sidebar

סוג מחלקה: Stateful Widget

הורשה: Stateful Widget

תפקיד: תפריט צד לניווט נוח

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
const constructor	מאתחל את התפריט	Sidebar	super.key	Sidebar
override	יוצר מצבים	State<Sidebar>		createStates

Sidebar State

שם המחלקה: Sidebar

סוג מחלקה: **State of Stateful Widget**

הורשה: <State> Sidebar

תפקיד: מצב של התפריט צד

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
override	בונה את התפריט	Widget	BuildContext context	build

Utils

שם המחלקה: Utils

סוג מחלקה: Static <no class>

הורשה:

תפקיד: תשתיות של פעולות חשובות

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
static	מחזיר מרחק על פי פוטנציאל	double	num x, num y	dist
static	מעביר את ה parameter ל T	T<T extends Object>	dynamic object	cast
static	מנסה להעביר את ה parameter ל T	T<T extends Object>?	dynamic object	tryCast

main

שם המחלקה: main

סוג מחלקה: Static <no class>

הורשה:

תפקיד: תחילת הרצת הקוד

תכונות:

הערות	תיאור	סוג	תכונה
-------	-------	-----	-------

פעולות:

הערות	תיאור	מחזיר	Parameter	פעולה
async	מתחיל את האפליקציה	void		main

בסיס הנתונים

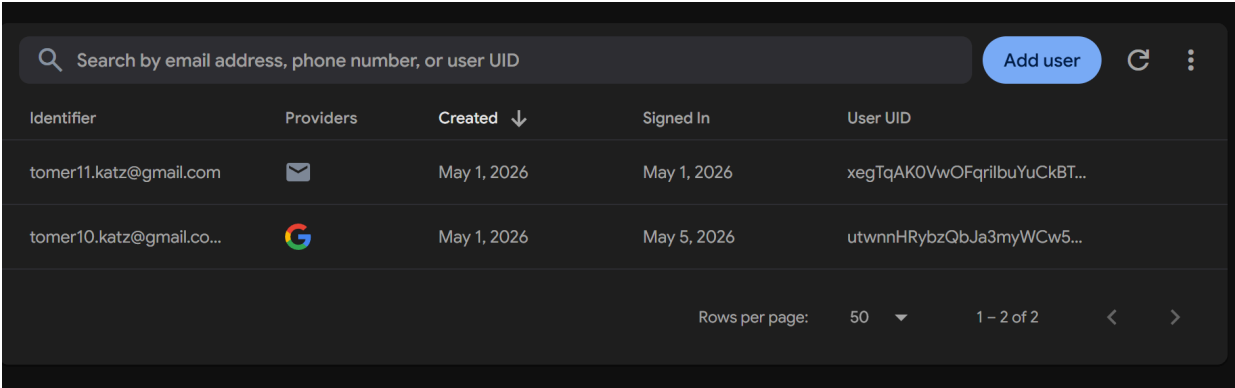
כל הפעולות על בסיס הנתונים מתבצעות דרך המחלות של Firebase ו- Google

- firebase_core
- flutterfire_cli
- firebase_database
- google_sign_in
- firebase_auth

Firestore Authentication

ב- authentication נשמר האימייל, הסיסמא וה id של המשתמש. ה id משמש גם ב Realtime DataBase.

ישנה אפשרות להתחבר ל Authentication דרך או אימייל וסיסמא, או דרך חשבון גוגל



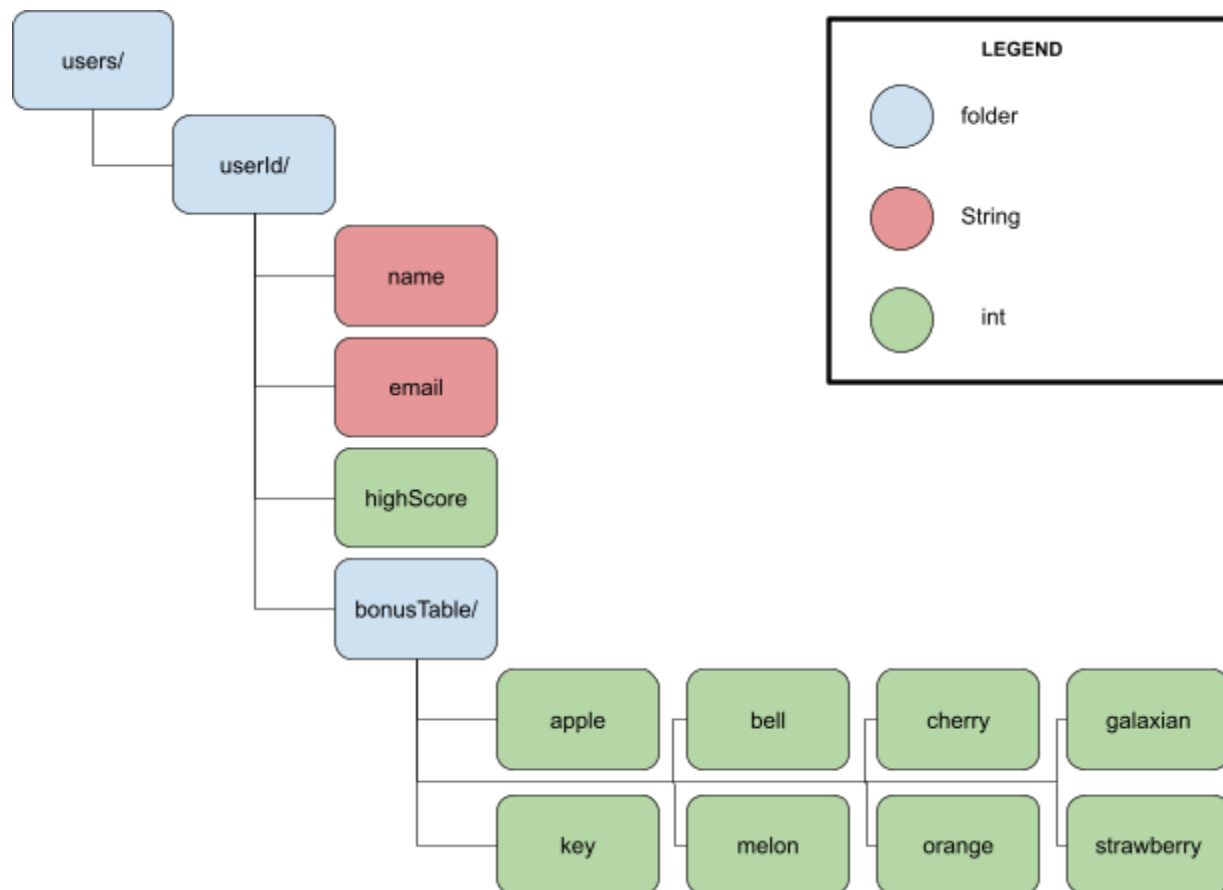
Identifier	Providers	Created ↓	Signed In	User UID
tomer11.katz@gmail.com		May 1, 2026	May 1, 2026	xegTqAK0VwOFqrlbuYuCkBT...
tomer10.katz@gmail.co...		May 1, 2026	May 5, 2026	utwnnHRybzQbJa3myWCw5...

Rows per page: 50 1 - 2 of 2

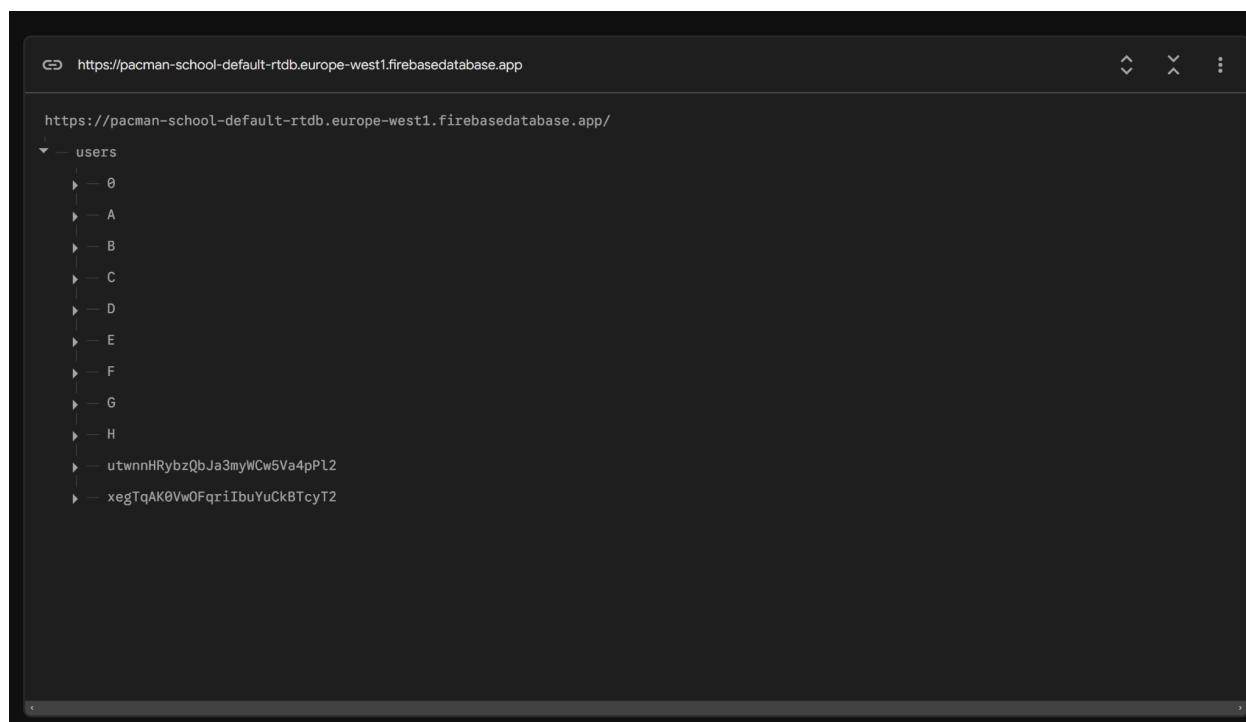
Realtime DataBase

ב - Real-time Database נשמרים כל הנתונים של המשתמש. הנתונים כאן בשיטת hashmap.

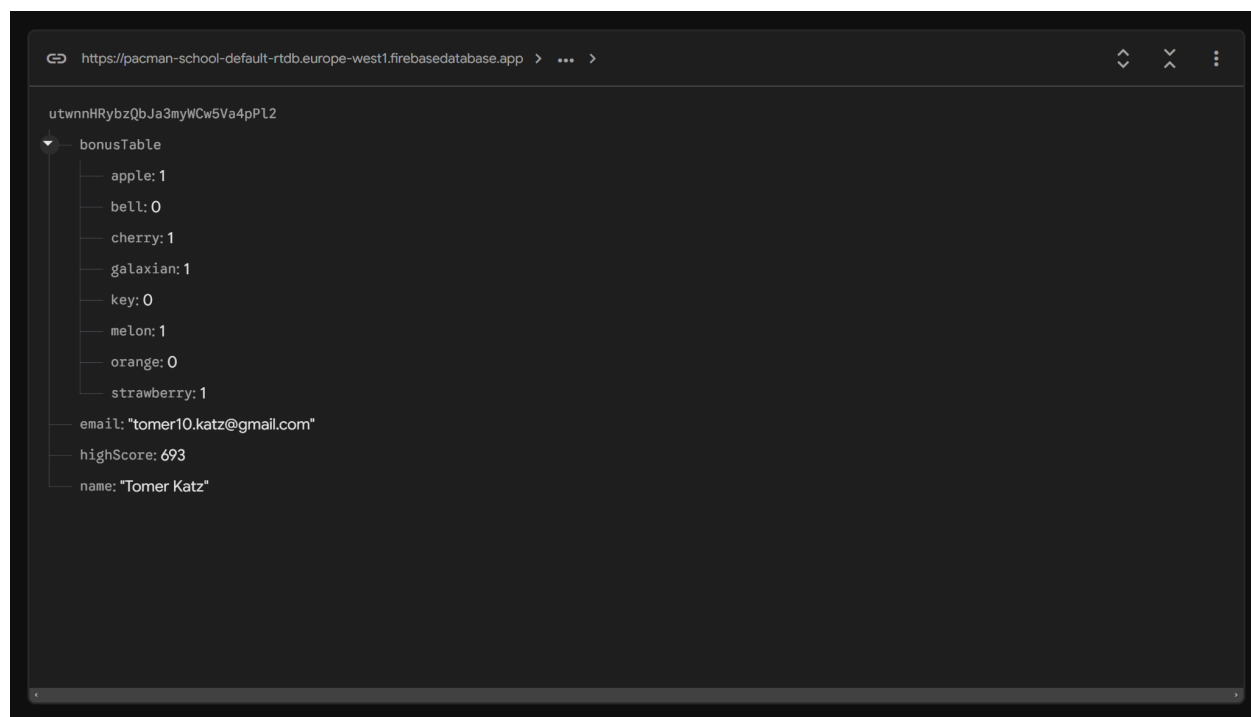
תרשים שמירת הנתונים ב- Real-time Database:



רשימה של כל המשתמשים תחת users



כל הערכים שיש לכל משתמש:

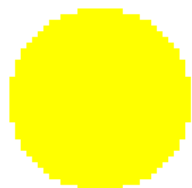


מדריך למשתמש

PAC-MAN — OFFICIAL INSTRUCTION MANUAL

Mobile Edition · Version 1.0

**PACMAN SCHOOL
PROJECT**



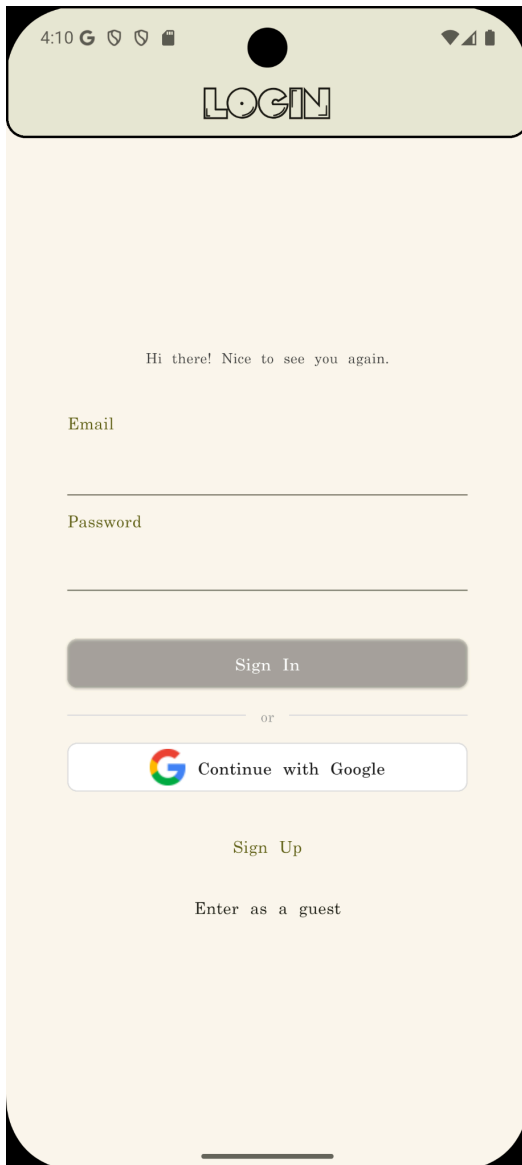
created by
Tomer

פרטי הגרסה ודרישות מערכת

הפרויקט נבדק על מכשיר אנדרואיד מסוג Samsung SM-A73.

האפליקציה פותחה בסביבת Flutter ומיועדת למכשירי אנדרואיד בלבד. לא ניתן לשחק במשחק ללא חיבור לאינטרנט, מפני שיש פעולות כגון שמירת שיא ועדכון לוח הדירוגים הדורשות חיבור פעיל.

מסך הכניסה (Login)



זהו המסך הראשון שמוצג למשתמש שטרם התחבר.
המסך מאפשר שלוש דרכי כניסה:

כניסה דרך אימייל וסיסמא — יש למלא את שדה האימייל ואת שדה הסיסמא ולאחר מכן ללחוץ על כפתור הכניסה. המערכת תבדוק את הפרטים מול מסד הנתונים ותכניס את המשתמש אם קיים.

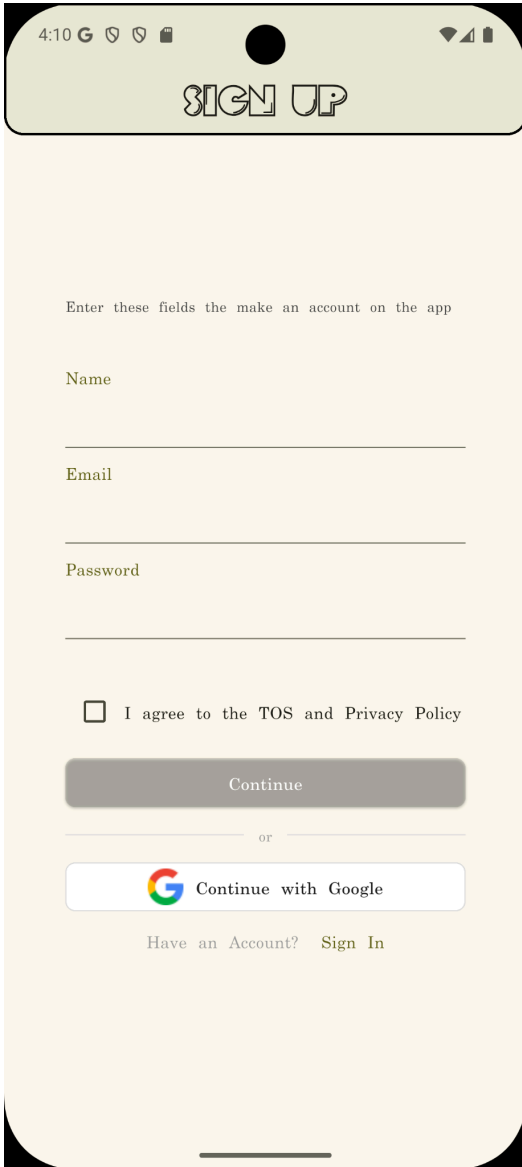
כניסה דרך חשבון גוגל — לחיצה על הכפתור המתאים תפתח חלון של גוגל לבחירת חשבון ולאחר אישור תתבצע הכניסה אוטומטית.

כניסה אנונימית — מאפשרת להיכנס כאורח מבלי ליצור חשבון. חלק מהפעולות (כגון שמירת שיא בלוח הדירוגים) לא יהיו זמינות במצב זה.

בתחתית המסך מופיע קישור למסך ההרשמה עבור משתמשים חדשים.

הודעות למשתמש: אם האימייל או הסיסמא שגויים תופיע הודעת שגיאה. אם שדה האימייל ריק תופיע הודעה בהתאם. האימייל חייב להיות בתבנית תקינה (לדוגמה: name@example.com).

מסך ההרשמה (Sign Up)



מסך זה מאפשר יצירת חשבון חדש באפליקציה. יש למלא:

שם משתמש — שם שיוצג בלוח הדירוגים ובמסכים השונים.

אימייל — חייב להיות בתבנית תקינה.

סיסמא — חייבת להכיל לפחות 6 תווים.

בנוסף יש לסמן את תיבת הסימון המאשרת את תנאי השימוש לפני שניתן יהיה להירשם. ללא סימון התיבה לא ניתן להמשיך.

קיימת גם אפשרות הרשמה דרך חשבון גוגל. בתחתית המסך מופיע קישור חזרה למסך הכניסה אם למשתמש כבר יש חשבון.

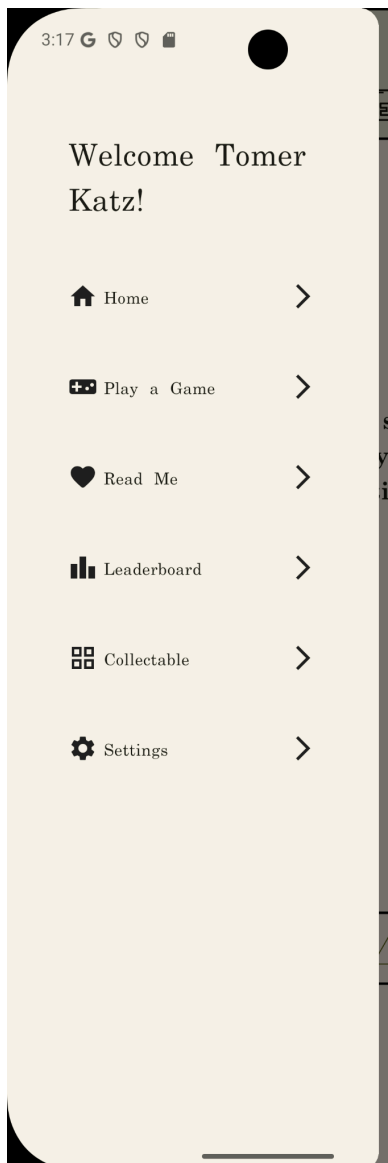
הודעות למשתמש: אם אחד מהשדות החובה לא מולא תופיע הודעת שגיאה מתאימה. אם הסיסמא קצרה מ-6 תווים תופיע הודעה. אם האימייל כבר קיים במערכת תופיע הודעת שגיאה.

תפריט עליון (App Bar)



התפריט העליון מופיע בכל מסכי האפליקציה. הוא מציג את שם המסך הנוכחי. בצד שמאל נמצא כפתור שמפתח את תפריט הצד לניווט בין המסכים. בצד ימין נמצא כפתור התנתקות — לחיצה עליו תנתק את המשתמש ותחזיר אותו למסך הכניסה.

תפריט צד (Sidebar)



לחיצה על כפתור תפריט הצד בתפריט העליון פותח תפריט ניווט. בראש התפריט מופיע שלום אישי עם שם המשתמש המחובר. מתחת מופיעה רשימת כל המסכים באפליקציה עם אייקון לכל אחד. לחיצה על שורה כלשהי תעביר את המשתמש ישירות לאותו מסך.

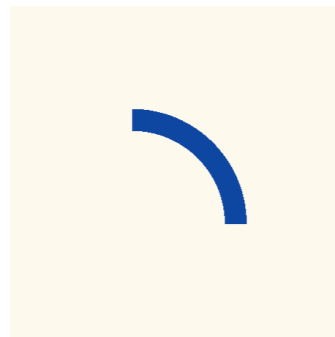
מסך הבית (Home Screen)

לאחר הכניסה, זהו המסך הראשון שמוצג. המסך מציג:

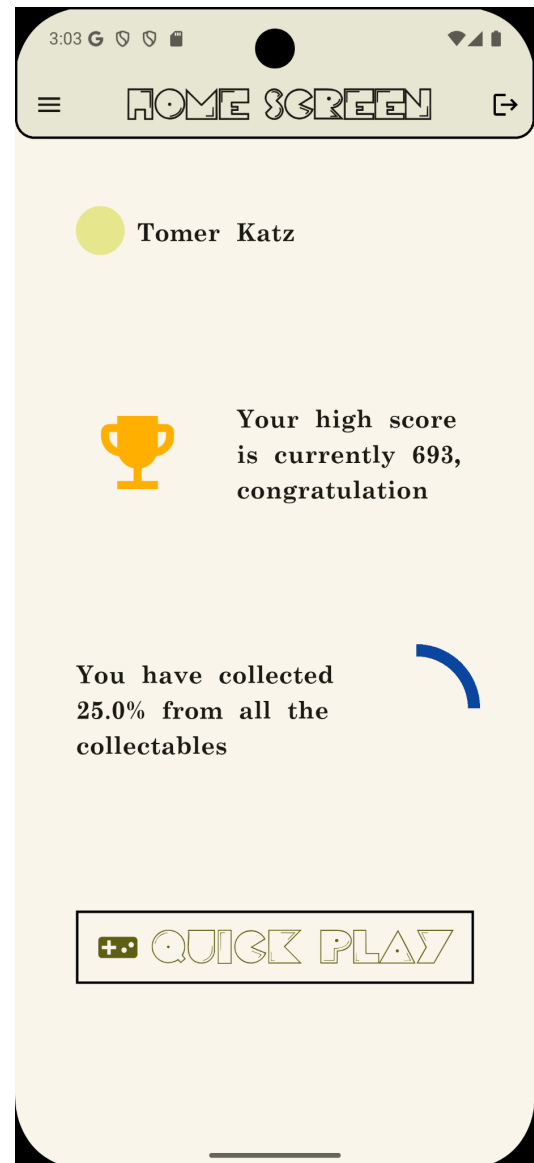
שם המשתמש המחובר.

ניקוד השיא האישי — הציון הגבוה ביותר שהמשתמש השיג אי פעם.

התקדמות בבונסים — אחוז הבונסים שנאספו עד כה מתוך שמונה הבונסים הקיימים, המוצג גם בגרף עגול.



כפתור משחק מהיר — לחיצה על הכפתור מעבירה ישירות למסך המשחק.



מסך המשחק (Game Screen)

זהו לב האפליקציה. המסך מחולק לשניים: חלק עליון ובו המשחק עצמו, וחלק תחתון ובו השלט.

מפת המבוך

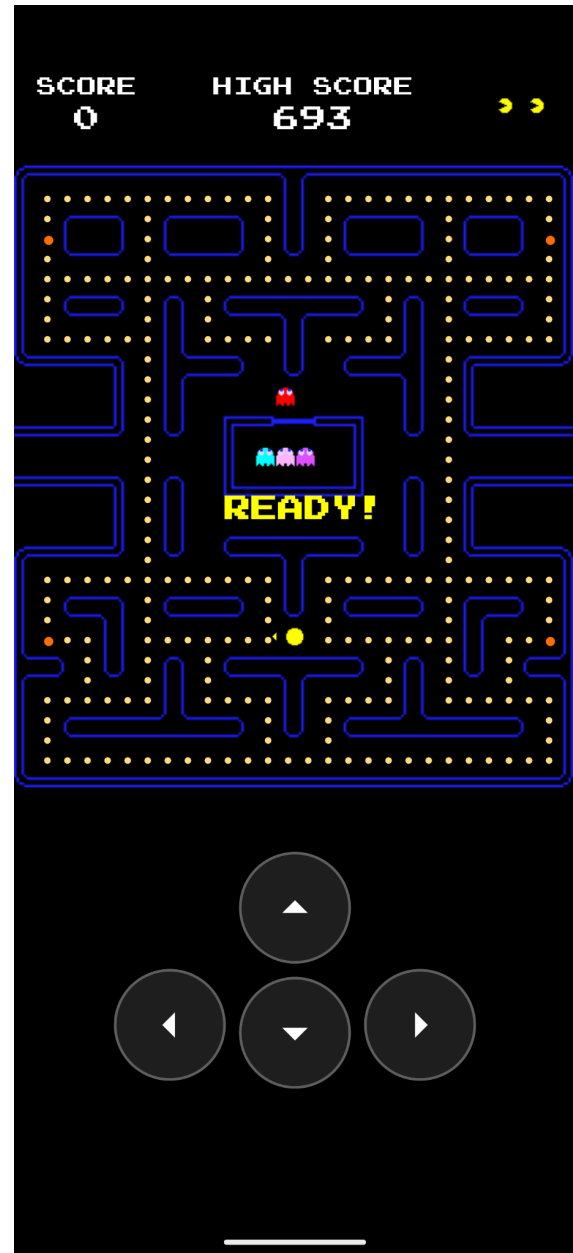
המשחק מתנהל בתוך מבוך קלאסי. ישנם שני סוגי נקודות שפקמן צריך לאסוף:

נקודות קטנות — ממלאות את המבוך. כל נקודה שנאכלת מוסיפה 1 לניקוד.

נקודות גדולות (סופר-דוט) — ארבע נקודות גדולות יותר ממוקמות בפינות המבוך. אכילת נקודה גדולה מוסיפה 10 לניקוד ומעניקה לפקמן 8 שניות של אי-פגיעות שבמהלכן הרוחות הופכות לכחולות וניתן לאכול אותן. הנקודה הגדולה מסומנת בלבן ומהבהבת.

המבוך מאפשר מעבר מצד ימין לצד שמאל ולהפך דרך המנהרות.

כאשר כל הנקודות נאכלות — המבוך מתאפס וממשיך להתגר הבא.



מידע עליון

בחלק העליון של המסך מוצגים:

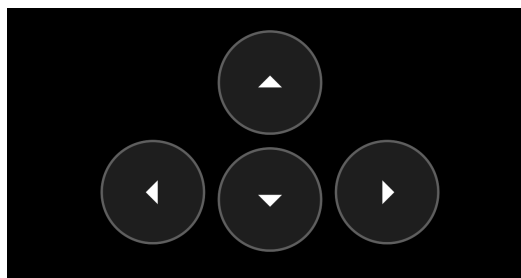
ניקוד נוכחי (Score) — הניקוד שנצבר במשחק הנוכחי.

ניקוד שיא (High Score) — השיא האישי הגבוה ביותר. אם הניקוד הנוכחי עובר את השיא, השיא מתעדכן בזמן אמת.

חיים — מספר החיים שנותרו מוצג כאייקונים של פקמן. אפשר לצבור עד תשעה חיים.



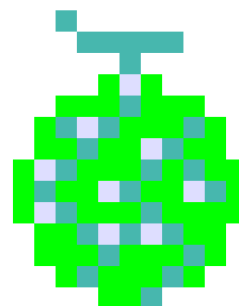
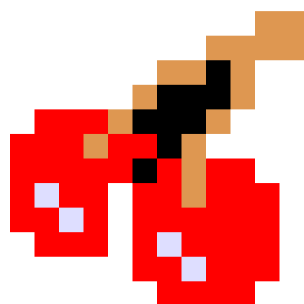
השלט



בתחתית המסך מופיע שלט עם ארבעה כפתורי כיוון: למעלה, למטה, שמאל, ימין.

אופן הפעולה של השלט: האפליקציה עובדת בסגנון Ms. PAC-MAN — הקשה על כפתור כלשהו מגדירה את הכיוון הבא הרצוי. פקמן ישמור את הכיוון הזה בזיכרון וכשיגיע לצומת שמאפשרת לפנות לאותו כיוון — הוא יפנה. אין צורך לתזמן את הלחיצה בצורה מדויקת. ניתן ללחוץ על הכיוון הבא מראש ופקמן יפנה ברגע המתאים.

בונוסים



בכל פרק זמן מסויים מופיע פריט בונוס אחד באזור המרכזי של המבוך. אכילת בונוס מוסיפה 10 לניקוד. קיימים 8 סוגי בונוסים שונים, וכל אחד מהם נחשב לפריט אספנות. הבונוסים שנאספו מוצגים בצד המסך.

חיים ובונוס חיים

פקמן מתחיל עם 3 חיים. בכל פעם שרוח נוגעת בפקמן (מחוץ למצב הפחד) הוא מאבד חיים אחד ומחזור המשחק מתחיל מחדש. כאשר כל החיים אוזלים — המשחק מסתיים.

כל 100 נקודות שנצברות מוענקת לפקמן חיים נוספים אוטומטית.

כאשר המשחק מסתיים, אם הניקוד הנוכחי גבוה מהשיא השמור — השיא מתעדכן במסד הנתונים.

הרוחות



ישנן ארבע רוחות שכל אחת פועלת לפי אלגוריתם שונה:

בלינקי (אדום) — הרוח המסוכנת ביותר. מכוונת תמיד לאחר מיקומו המדויק של פקמן. לא תפסיק לרדוף.

פינקי (ורוד) — מכוונת לנקודה שנמצאת קדימה בכיוון התנועה של פקמן, כדי לחסום אותו.

אינקי (ציאן) — אלגוריתם מורכב. אינקי מכוון לנקודת האמצע בין מיקום בלינקי לבין נקודת המטרה של פינקי, מה שיוצר הקפות בלתי צפויות.

קלייד (כתום) — מכוון לפקמן כמו בלינקי, אך אם הוא מגיע לטווח של 10 משבצות מפקמן — הוא נסוג לפינת הפיזור שלו. ניתן לנצל זאת כדי לצמצם את הסכנה ממנו.

הרוחות מחליפות בין מצב מרדף (Chase) למצב פיזור (Scatter) כל כמה שניות, כאשר במצב פיזור כל רוח נסוגה לפינת המבוך שלה.

אכילת רוחות

כאשר פקמן אוכל נקודה גדולה, הרוחות הופכות לכחולות וניתן לאכול אותן. ניקוד אכילת הרוחות מוכפל עבור כל רוח נוספת שנאכלת בפרק הזמן של אותה הנקודה הגדולה:

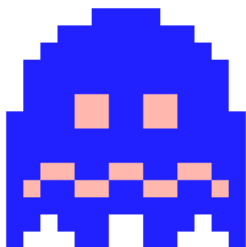
רוח ראשונה — 10 נקודות.

רוח שנייה — 20 נקודות.

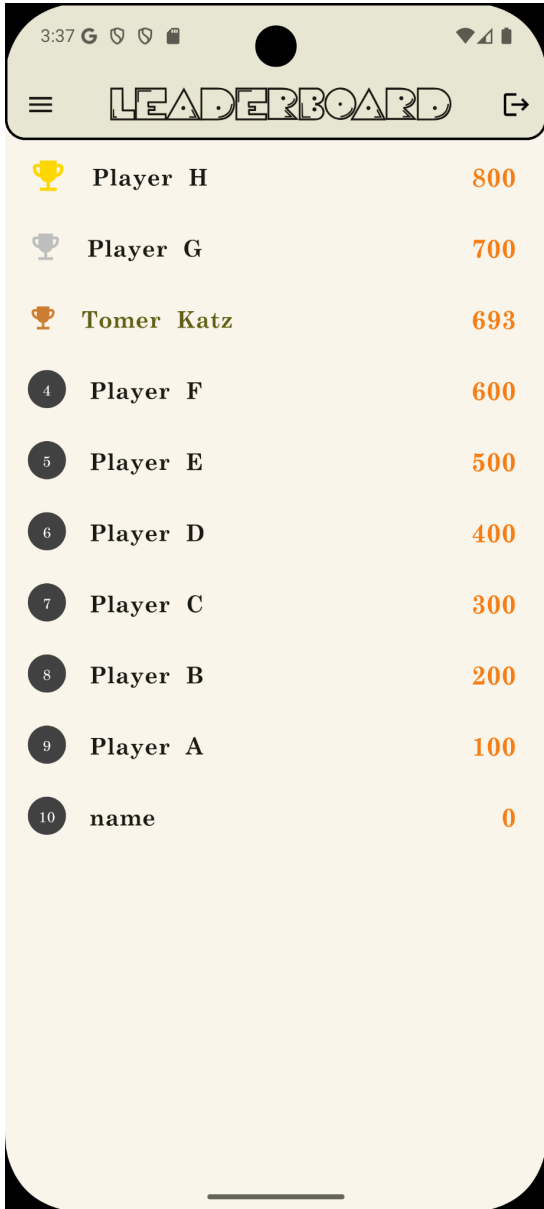
רוח שלישית — 40 נקודות.

רוח רביעית — 80 נקודות.

הרצף מתאפס כאשר 8 השניות נגמרות.



מסך הדירוגים (Leaderboard)



	LEADERBOARD	
1	Player H	800
2	Player G	700
3	Tomer Katz	693
4	Player F	600
5	Player E	500
6	Player D	400
7	Player C	300
8	Player B	200
9	Player A	100
10	name	0

מסך זה מציג את כל השחקנים הרשומים באפליקציה ממוינים לפי ניקוד השיא הגבוה ביותר שלהם.

שלושת המקומות הראשונים מסומנים עם אייקון מיוחד (זהב, כסף, ארד). המשתמש המחובר כרגע מוצג בצבע מיוחד כך שיוכל לאתר בקלות את מיקומו האישי ברשימה.

מסך הבונוסים (Collectables)

מסך זה עוקב אחרי אתגר האספנות. קיימים 8 בונוסים שונים במשחק. בונוס שעדיין לא נאסף מוצג באפור ומוסתר. בונוס שנאסף לפחות פעם אחת מוצג בצבעיו המלאים ואפשר ללחוץ עליו.

לחיצה על בונוס שנאסף פותחת את דיאלוג הבונוס.

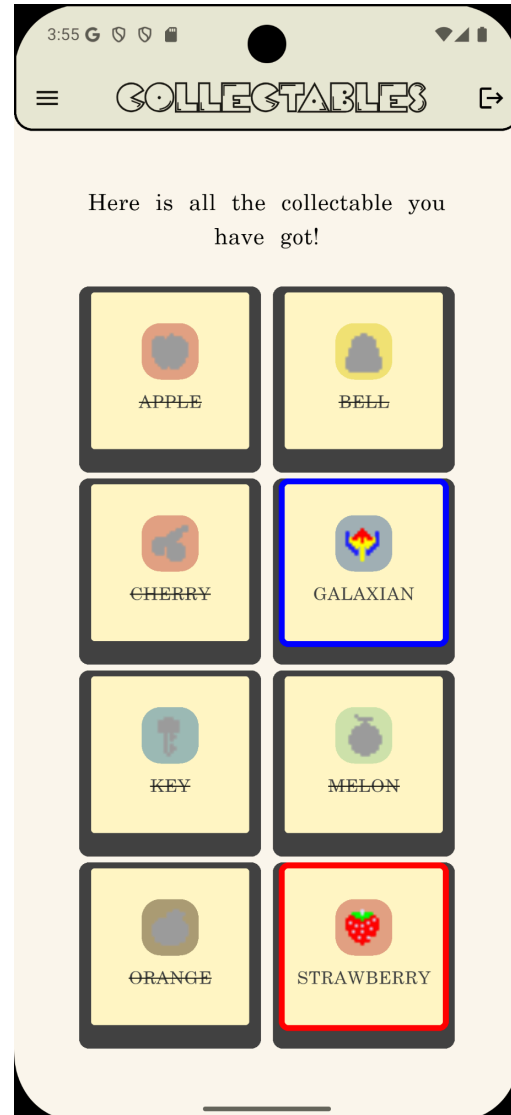
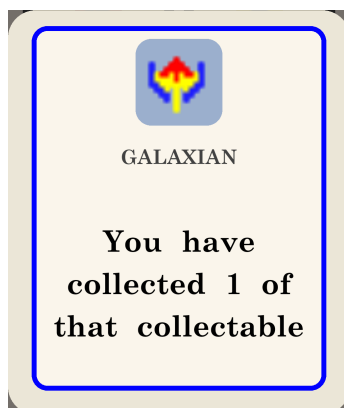
דיאלוג הבונוס

לחיצה על בונוס שנאסף מציגה חלונית קטנה עם:

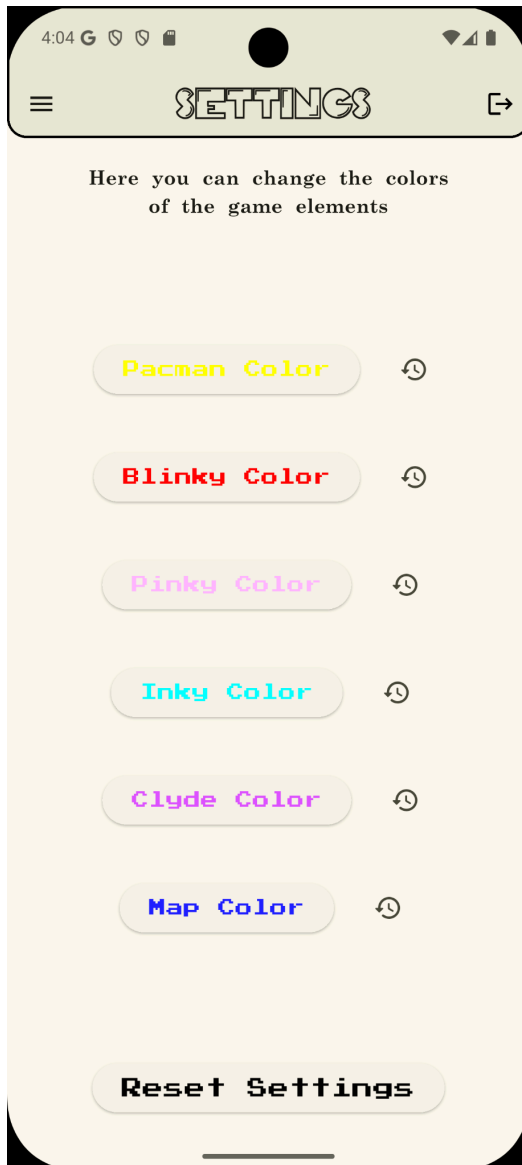
תמונה של הבונוס.

שם הבונוס.

כמות הפעמים שהבונוס נאסף בסך הכל.



מסך ההגדרות (Settings)



מסך ההגדרות מאפשר לשנות את צבעי אלמנטי המשחק. ניתן לשנות את הצבע של:

פקמן עצמו.

כל אחת מארבע הרוחות (בלינקי, פינקי, אינקי, קלייד) — בנפרד.

קירות המבוך.

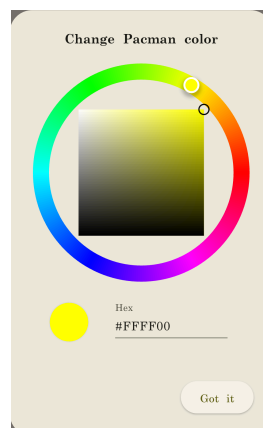
לצד כל כפתור שינוי צבע מופיע כפתור איפוס קטן שמחזיר את האלמנט הספציפי לצבעו המקורי. בתחתית הדף מופיע כפתור "שחזור כללי" שמחזיר את כל הצבעים לברירות המחדל בבת אחת.

שינויי הצבע קוסמטיים בלבד — הם לא משפיעים על התנהגות הרוחות או על מכניקות המשחק.

דיאלוג שינוי צבע

לחיצה על כפתור שינוי צבע של אלמנט כלשהו פותחת חלונית עם גלגל צבעים (HueRingPicker). גרירת הסמן על הגלגל משנה את הצבע של האלמנט הנבחר בזמן אמת. לחיצה על כפתור האישור שומרת את הבחירה וסוגרת את החלונית.

הצבע הנבחר נשמר במכשיר ויישמר גם לאחר יציאה מהאפליקציה.



מסך הסבר (ReadMe)



מדריך למשתמש

PAC-MAN – OFFICIAL INSTRUCTION MANUAL

Mobile Edition - Version 1.0



פרטי הגרסה ודרישות מערכת

הפרויקט נבדק על מכשיר אנדרואיד מסוג Samsung SM-A73.

האפליקציה פותחה בסביבת Flutter ומיועדת למכשירי אנדרואיד בלבד. לא ניתן לשחק במשחק ללא חיבור לאינטרנט, מפני שיש פעולות כגון שמירת שיא ועדכון לוח הדירוגים הדורשות חיבור פעיל.

מסך ההסבר מאפשר לראות את המסמך הזה ולראות איך אפשר לשחק במשחק.

אפשר לגלול ברחבי העמוד בתוך ה Google Docs ולראות את כל המסמך.

מגבלות ואילוצים

שמירת שיאים ועדכון לוח הדירוגים מצריכים חיבור לאינטרנט. משתמש אנונימי (אורח) לא יוכל לשמור שיא בלוח הדירוגים הגלובלי. לא ניתן למחוק רשומות מלוח הדירוגים. ניקוד השיא מתעדכן אוטומטית בסיום המשחק אם הניקוד עלה על השיא הקודם.

תומר כץ — יוצר האפליקציה בית ספר בן יהודה · מאי 2026

סיכום אישי ורפלקציה

במהלך העבודה על פרויקט ה-Pacman למדתי הרבה מעבר לכתיבת קוד בלבד. בתחילת הדרך המטרה שלי הייתה ליצור משחק מהנה שעובד בצורה טובה בטלפון, אך תוך כדי הפיתוח הבנתי כמה תכנון, סדר וחשיבה מקדימה חשובים בפרויקט גדול. נדרשתי להתמודד עם אתגרים שלא הכרתי לפני כן, במיוחד בתחום האנימציות, ניהול המשחק והבינה המלאכותית של הרוחות.

אחד הדברים המשמעותיים ביותר שלמדתי הוא איך לפרק בעיה גדולה לחלקים קטנים. בתחילת העבודה הכול הרגיש מסובך מאוד, אבל כאשר חילקתי את הפרויקט למחלקות ולמערכות נפרדות, העבודה נהייתה ברורה ומסודרת יותר. בנוסף, למדתי לעבוד בצורה מסודרת עם Flutter ו-Firebase, כולל שמירת נתונים, התחברות משתמשים וניהול מידע בזמן אמת.

האתגר המרכזי מבחינתי היה יצירת מערכת האנימציה וההתנהגות של הרוחות. Flutter לא סיפקה פתרון שמתאים בדיוק למה שרציתי, ולכן הייתי צריך ליצור פתרון עצמאי. היו הרבה רגעים שבהם דברים לא עבדו כמו שצריך, במיוחד בתזמון האנימציות ובחישובי התנועה, אבל דרך הניסיונות והטעויות הצלחתי להבין טוב יותר איך המערכת פועלת ואיך לפתור בעיות מורכבות.

אני חושב שהחלק שהכי נהניתי ממנו היה לראות את המשחק מתקדם לאט-לאט מרעיון קטן למערכת אמיתית שאפשר לשחק בה. בכל פעם שהוספתי פיצ'ר חדש, כמו בונסים, דירוגים או צבעים מותאמים אישית, הרגשתי שהפרויקט נהיה מקצועי יותר. בנוסף, נהניתי מכך שהצלחתי לקחת משחק מוכר ולהוסיף לו רעיונות אישיים שלי מבלי לאבד את התחושה המקורית של Pacman.

אם הייתי ממשיך את הפרויקט בעתיד, הייתי רוצה להוסיף עוד שלבים, מצבי משחק חדשים ואולי גם אפשרות למשחק מול שחקנים אחרים דרך האינטרנט. בנוסף הייתי משפר את מבנה הקוד עוד יותר כדי שיהיה קל יותר להוסיף תכונות חדשות בעתיד.

לסיכום, הפרויקט תרם לי מאוד גם מבחינה מקצועית וגם מבחינה אישית. למדתי להתמודד עם בעיות בצורה עצמאית, לחקור נושאים חדשים לעומק ולא לוותר גם כאשר משהו לא עבד מיד. אני מרגיש שהפרויקט שיפר משמעותית את היכולות שלי בתכנות ובחשיבה לוגית, והוא אחד הפרויקטים שאני הכי גאה בהם.

ביבליוגרפיה

- Flutter.dev -
- Pub.dev -
- Claude -
- Gemini -
- ChatGPT -
- The Spriters Resource -
- Firebase -
- Google -
- StackOverFlow -
- Gimp -
- Pacman fandom wiki -
- Youtube -
- Github -
- Video Game Music -

נספחים

מערכת האנימציה המותאמת (Custom Sprite Engine)

רקע והחלטה הנדסית

במהלך הפיתוח עלה הצורך במערכת אנימציה התומכת בשינוי פריימים מורכב עבור דמויות המשחק. בעוד ש-Flutter מציעה כלים מצוינים לאנימציות "Tween", היא פחות מותאמת כברירת מחדל לאנימציות פריימים קלאסית (Frame-by-frame). החלופה המקובלת, ספריית **Flame**, היא עוצמתית אך מחייבת שינוי ארכיטקטוני רחב בקוד. לכן, בחרתי לממש מנוע אנימציה עצמאי שמעניק שליטה מלאה ואינטגרציה חלקה עם הלוגיקה הקיימת.

ארכיטקטורת המערכת

המערכת מבוססת על שתי מחלקות מרכזיות המפרידות בין הלוגיקה של הדמות לבין ניהול התצוגה:

1. מחלקת **Animated Character** (התשתית הלוגית)

זוהי מחלקה מופשטת (Abstract Class) מהווה את הבסיס לכל דמות נעה במפה.

- **ניהול מצב (State):** היא מחזיקה את המיקום בלוח (x, y) , הכיוון הנוכחי והכיוון הבא.
- **ניהול אנימציות:** המחלקה מנהלת רשימה של אובייקטי **Sprite Animation** ואחראית על בחירת האנימציה הפעילה.
- **מחזור חיים (Update Loop):** בכל פריים, הפונקציה **update(dt)** נקראת ומקבלת את ה-**(Delta Time (dt)** – הזמן שעבר מהפריים הקודם – כדי להבטיח תנועה חלקה ללא תלות בקצב הרענון של המכשיר.

2. מחלקת **Sprite Animation** (מנוע הרינדור)

יחידה האחראית על הצגת התמונות הנכונות בזמן הנכון.

- **טעינה דינמית:** המערכת משתמשת במוסכמת שמות (Convention over Configuration). לפי שם הדמות והמצב, היא בונה אוטומטית רשימת נתיבים לקבצי ה-PNG.
- **מנגנון ה-Clock וטיפול בנפילות פריימים:** המחלקה מחזיקה צובר זמן (**clock_**). בכל עדכון, היא בודקת אם הצטבר מספיק זמן כדי לעבור לפריים הבא.
- **דיוק זמן (Frame Skip):** שימוש בלולאת **while** (במקום **if**) לבדיקת הזמן מאפשר למנוע "לדלג" על פריימים אם המכשיר חווה לאג (Frame Drop). כך, גם אם עבר זמן השווה לשני פריימים, המערכת תעדכן את האינדקס פעמים ותשמור על סנכרון עם זמן אמת.

- **ורסטיליות:** תומכת בלוגיקה של לולאה (`is Loop`), עצירה (`is Paused`) ואיפוס מצב (`idle`) כדי למנוע מצב שבו אנימציה מתחילה מהאמצע כאשר מחליפים כיוון.

פיצ'רים מתקדמים במימוש

- **Color Tinting:** המערכת תומכת בהעברת `Color` לפונקציית הרינדור. זה מאפשר להשתמש באותם נכסים גרפיים ולצבוע אותם דינמית, למשל עבור רוחות במצבים שונים (כמו מצב "Frightened").
- **ניתוק הצימוד (Decoupling):** הדמות לא צריכה לדעת "איך" לצייר, אלא רק "מה" המצב שלה. האנימציה היא רכיב עצמאי, מה שמקל על תחזוקה והוספת דמויות חדשות.
- **יעילות בזיכרון:** שימוש ב-`Image.asset` של Flutter מבטיח ניהול מטמון (Caching) יעיל של התמונות בזיכרון המערכת.

הסבר אלגוריתם רוחות

בניגוד לתנועה פשוטה שבה דמות מנסה רק לצמצם מרחק אווירי (מה שעלול לגרום לה להיתקע בפינות או קירות), ה-`complexChase` מאפשר לרוח להבין את המבנה המורכב של המבוך ולמצוא את הנתיב האופטימלי.

1. אלגוריתם ה-BFS (Breadth-First Search)

הפונקציה משתמשת באלגוריתם סריקה שסורק את המבוך בשכבות, כמו אדוות שמתפשטות במים.

- **הסריקה:** הרוח בוחנת את כל המשבצות השכנות שלה, לאחר מכן את השכנות שלהן, וכן הלאה, עד שהיא "מוצאת" את משבצת המטרה.
- **מציאת הקיצור:** מכיוון שהסריקה מתבצעת בשכבות שוות, הפעם הראשונה שהאלגוריתם יפגוש את המטרה מובטח שזה יהיה המסלול הקצר ביותר האפשרי מבחינת מספר צעדים.

2. טכניקת ה-"First Direction" (ההחלטה המיידית)

זהו החלק ה"חכם" ביותר במימוש: הרוח צריכה להחליט רק על הצעד **הקרוב** (למעלה, למטה, ימינה או שמאלה), אבל ההחלטה הזו מבוססת על מסלול שלם קדימה.

- בזמן הסריקה, המערכת שומרת בזיכרון (`firstDir`) עבור כל משבצת שנסרקה: "מה היה הכיוון הראשון שבו יצאנו מהבית כדי להגיע לכאן?".
- ברגע שהסריקה מגיעה ליעד (`tx, ty`), היא פשוט בודקת מה היה הכיוון המקורי שהוביל לשם ומחזירה אותו כ-`nextDirection`.

3. התמודדות עם "מנהרות" (Map Wrap-around)

הקוד כולל לוגיקה של שארית חלוקה (Modulo) המאפשרת לרוחות להבין שהמבוך אינו סופי. אם רוח נמצאת בקצה הימני, ה-`complexChase` יודע שהדרך הקצרה ביותר ליעד שנמצא משמאל עשויה להיות דווקא דרך המנהרה, והוא יחשב זאת כחלק מהמסלול האופטימלי.

סיכום התהליך בתוך ה-Update Loop

בכל פריים (`dt`), הרוח מבצעת את הרצף הבא:

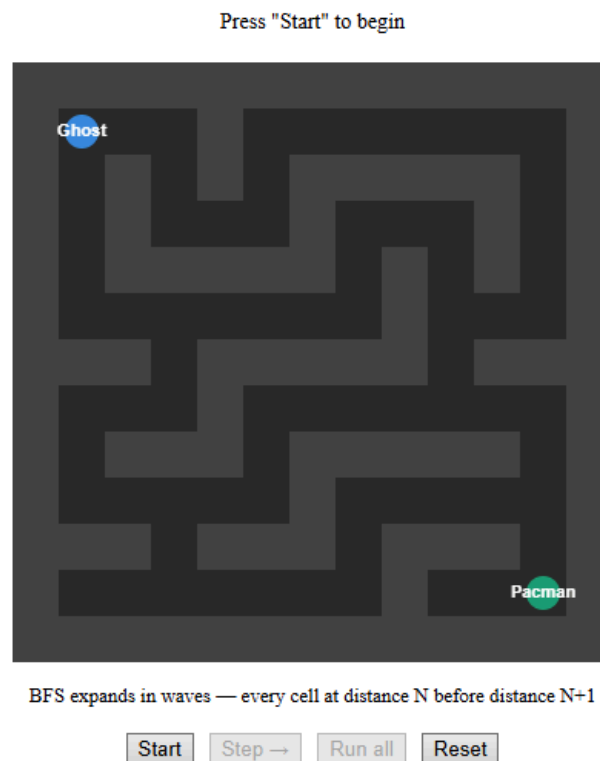
1. **זיהוי יעד:** לפי המצב שלה (מדרף, התפזרות או חזרה לבית), נקבעות קואורדינטות המטרה.
2. **חישוב מסלול:** קריאה ל-`complexChase` שמריץ BFS ומחזיר את הכיוון הנכון לצאת אליו.
3. **ביצוע תנועה:** עדכון ה-`current Direction` והזזת הדמות במרחב הלוח.

השילוב הזה גורם לרוחות להתנהג בצורה מאיימת ומאתגרת, שכן הן תמיד יבחרו בנתיב היעיל ביותר כדי לחסום את דרכו של פקמן.

דוגמא

העלתי לאינטרנט דוגמה לאיך הרוח עוברים במבוך עד שמגיעים ל Pacman.

[קישור לאתר](#)



באתר אפשר לראות מיקום של הרוח ומיקום של pacman.

אפשר ללחוץ על start ואז להתחיל את האלגוריתם.
בשביל לראות את הסימולציה אפשר ללחוץ על step. ולהתקדם צעד צעד באלגוריתם לראות חשיבה חשיבה מה הרוח חושבת לעשות.
אפשר לעשות Run All ואז הסימולציה תמשיך עד הסוף, בודקת כל פינה עד שהגיע Pacman וחושבת את המסלול הכי טוב למעקב.
לבסוף יש כפתור Reset בשביל איפוס הסימולציה.

קוד המקור+javadoc

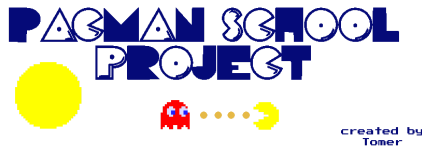
=====

Directory Tree (selected)

=====

```
lib/
├── database/
│   ├── firebase_options.dart
│   ├── game_user.dart
│   └── session.dart
├── game/
│   ├── animation/
│   │   ├── animated_character.dart
│   │   └── sprite_animation.dart
│   ├── controller/
│   │   ├── controller.dart
│   │   ├── dpad_button.dart
│   │   └── end_game_buttons.dart
│   ├── ghosts/
│   │   ├── blinky.dart
│   │   ├── clyde.dart
│   │   ├── ghost.dart
│   │   ├── inky.dart
│   │   └── pinky.dart
│   ├── map/
│   │   ├── game_map.dart
│   │   ├── map_paller.dart
│   │   └── mazePainter.dart
│   ├── bonus.dart
│   ├── direction.dart
│   ├── dot.dart
│   ├── game_manager.dart
│   ├── game_screen.dart
│   └── game_timer.dart
```

Pacman Android Edition



תומר כץ

```
|   ├── pacman.dart
|   └── super_dot.dart
└── screens/
    ├── collectables_screen.dart
    ├── home_screen.dart
    ├── leaderboard_screen.dart
    ├── login.dart
    ├── read_me.dart
    ├── settings.dart
    └── signup.dart
└── utils/
    ├── appbar.dart
    ├── bonus_card.dart
    ├── bonus_type.dart
    ├── constants.dart
    ├── game_displayer.dart
    ├── loading_screen.dart
    ├── sidebar.dart
    └── util.dart
└── main.dart
```

File: /lib/main.dart

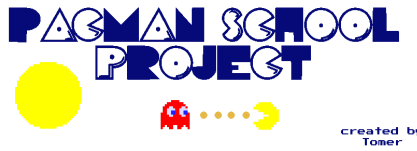
```
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'package:pacman_application/utils/constants.dart';
import 'package:pacman_application/screens/login.dart';
import
'package:pacman_application/database/firebase_options.dart';
import 'package:shared_preferences/shared_preferences.dart';

/// Bootstraps the application:
/// 1. Ensures Flutter widget binding is initialized.
/// 2. Initializes Firebase with the platform-specific
[DefaultFirebaseOptions].
/// 3. Warms up the [SharedPreferences] singleton.
/// 4. Runs [MyApp] as the root widget.
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options:
DefaultFirebaseOptions.currentPlatform);
  await SharedPreferences.getInstance();

  runApp(const MyApp());
}

/// Root widget of the Pac-Man application.
///
/// Configures the global [MaterialApp] with the app title,
theme (including
/// the custom "BJCree" font and a Pac-Man-yellow seed color),
and sets
/// [Login] as the initial route.
```


Pacman Android Edition



תומר כץ

```
class MyApp extends StatelessWidget {  
  /// Creates the root application widget.  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Pac-Man',  
      debugShowCheckedModeBanner: false,  
      theme: ThemeData(  
        fontFamily: 'BJCree',  
        colorScheme: ColorScheme.fromSeed(seedColor:  
pacmanColor),  
      ),  
      home: const Login(),  
    );  
  }  
}
```

File: /lib/utils/appbar.dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:pacman_application/database/game_user.dart';
import 'package:pacman_application/screens/login.dart';
import 'package:pacman_application/database/session.dart';
import 'package:shared_preferences/shared_preferences.dart';

/// A custom [AppBar] used on every screen in the application.
///
/// Renders a centred [header] title and, when [canLogout] is
`true`,
/// a logout icon button that signs the user out of Firebase,
clears the
/// stored session preferences, resets [currentUser] to the
anonymous
/// placeholder, and navigates back to [Login].
class AppBar extends AppBar {
  // — Constructor

  /// Creates an [AppBar].
  ///
  /// - [context] – the [BuildContext] used for navigation on
logout.
  /// - [header] – the title widget displayed in the centre of
the bar.
  /// - [canLogout] – when `false` the logout button is hidden
(useful on
  /// the [Login] and [Signup] screens).
  AppBar({
```

```
super.key,  
required this.context,  
this.header = const Text(''),  
this.canLogout = true,  
});
```

```
// — Fields
```

```
/// The [BuildContext] used when navigating to [Login] after  
sign-out.
```

```
final BuildContext context;
```

```
/// The widget rendered as the centred app-bar title.
```

```
final Text header;
```

```
/// Controls visibility of the logout icon button.
```

```
///
```

```
/// Set to `false` on screens where the user is not yet  
authenticated
```

```
/// (e.g. [Login], [Signup]).
```

```
final bool canLogout;
```

```
// — AppBar overrides
```

```
@override
```

```
Widget? get title => Center(child: header);
```

```
@override
```

```
List<Widget>? get actions => canLogout
```

```
? [
```

```
    IconButton(
```

```
        onPressed: () {
```

```
        FirebaseAuth.instance.signOut();
        currentUser = GameUser(uid: '0');
        _clearPrefs();
        Navigator.pushReplacement(
            context,
            MaterialPageRoute(
                builder: (final BuildContext context) =>
Login(),
            ),
        );
    },
    icon: const Icon(Icons.logout, color: Colors.black),
),
]
: [];

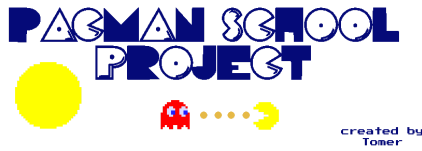
@override
double? get elevation => 8;

@override
ShapeBorder? get shape => const RoundedRectangleBorder(
    side: BorderSide(width: 2),
    borderRadius: BorderRadiusGeometry.vertical(bottom:
Radius.circular(16)),
);

// — Private methods

/// Removes the persisted "last user" keys from
[SharedPreferences] so the
/// app does not auto-login on next launch.
void _clearPrefs() async {
    final prefs = await SharedPreferences.getInstance();
```

Pacman Android Edition



תומר כץ

```
await prefs.remove('lastUser/uid');  
await prefs.remove('lastUser/name');  
await prefs.remove('lastUser/highScore');  
await prefs.remove('lastUser/email');  
}  
}
```

File: /lib/utils/bonus_card.dart

```
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:pacman_application/database/session.dart';
import 'package:pacman_application/utils/bonus_type.dart';

/// A card widget shown in the [CollectablesScreen] grid for a
single
/// [BonusType].
///
/// Cards for items the player has not yet collected are
rendered in
/// greyscale with a strikethrough label. Tapping a collected
card opens a
/// detail [AlertDialog] showing the bonus image and how many
times it has
/// been collected.
///
/// The card also features a subtle press animation: it sinks
slightly on
/// tap-down and rises back on tap-up, mimicking a physical
button.
class BonusCard extends StatefulWidget {
  const BonusCard(this.i, {super.key});

  /// Index into [BonusType.values] that identifies which bonus
this card
  /// represents.
  final int i;
```

```
@override
State<BonusCard> createState() => BonusCardState();
}

class BonusCardState extends State<BonusCard> {
  // — Constants

  /// Asset directory that contains bonus PNG images.
  static const String _assetDir = 'assets/game/bonuses/';

  /// File extension for bonus images.
  static const String _assetExt = '.png';

  // — Fields

  /// Whether the card is currently being pressed.
  bool _isPressed = false;

  /// Set to `true` 200 ms after tap-down; prevents a race where
  tap-up
  /// fires before the minimum press animation duration has
  elapsed.
  bool _isTimerFinished = false;

  /// The [BonusType] this card represents.
  late final BonusType _bonus;

  /// Whether the player has collected this bonus at least once.
  bool _hasCollected = false;

  /// The total number of times the player has collected this
  bonus.
```

```
int _collectedCount = 0;
```

```
// — Lifecycle
```

```
@override  
void initState() {  
    super.initState();  
  
    _bonus = BonusType.values[widget.i];  
    _collectedCount = currentUser.bonusTable[_bonus]!;  
    _hasCollected = _collectedCount > 0;  
}
```

```
// — Build
```

```
@override  
Widget build(BuildContext context) {  
    return GestureDetector(  
        onTapDown: (_) => setState(() {  
            _isPressed = true;  
            Timer(  
                const Duration(milliseconds: 200),  
                () => _isTimerFinished = true,  
            );  
            if (_hasCollected) _showBonusDialog(context);  
        }),  
        onTapUp: (_) {  
            if (!_isTimerFinished) {  
                Timer(const Duration(milliseconds: 200), () {  
                    setState(() {  
                        _isTimerFinished = false;  
                        _isPressed = false;  
                    });  
                });  
            }  
        })  
    );  
}
```



```
        });  
    });  
} else {  
    setState(() {  
        _isTimerFinished = false;  
        _isPressed = false;  
    });  
}  
,  
onTapCancel: () {  
    if (!_isTimerFinished) {  
        Timer(const Duration(milliseconds: 300), () {  
            setState(() {  
                _isTimerFinished = false;  
                _isPressed = false;  
            });  
        });  
    } else {  
        setState(() {  
            _isTimerFinished = false;  
            _isPressed = false;  
        });  
    }  
},  
child: AnimatedContainer(  
    duration: const Duration(milliseconds: 300),  
    curve: Curves.easeInOut,  
    decoration: BoxDecoration(  
        boxShadow: [  
            BoxShadow(  
                color: Colors.grey[800]!,  
                offset: _isPressed ? const Offset(0, 2) : const  
Offset(0, 10),  
                spreadRadius: _isPressed ? 1 : 5,
```

```
    ),  
    BoxShadow(  
      color: Colors.grey[800]!,  
      offset: _isPressed ? Offset.zero : const  
Offset(-4, 1),  
      spreadRadius: _isPressed ? 0 : 1,  
    ),  
    BoxShadow(  
      color: Colors.grey[800]!,  
      offset: _isPressed ? Offset.zero : const Offset(4,  
1),  
      spreadRadius: _isPressed ? 0 : 1,  
    ),  
  ],  
  border: BoxBorder.all(  
    color: _hasCollected ? _bonus.borderColor :  
Colors.grey[800]!,  
    width: 5,  
  ),  
  borderRadius: const  
BorderRadius.all(Radius.circular(8)),  
  color: Colors.yellow[100],  
),  
margin: const EdgeInsets.all(10),  
padding: const EdgeInsets.all(5),  
child: AnimatedSlide(  
  duration: const Duration(milliseconds: 100),  
  offset: _isPressed ? const Offset(0, 0.05) :  
Offset.zero,  
  curve: Curves.easeInOut,  
  child: Column(  
    spacing: 10,  
    mainAxisAlignment: MainAxisAlignment.center,  
    children: [  

```

```
        _bonusImageWithGlow(scale: 0.4),
        Text(
          _bonus.name.toUpperCase(),
          style: TextStyle(
            fontSize: 16,
            color: Colors.grey[800],
            decoration:
              _hasCollected ? null :
TextDecoration.lineThrough,
          ),
        ),
      ],
    ),
  ),
);
}

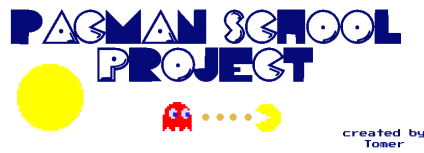
// — Private helpers

/// Opens the detail dialog for this bonus, showing a larger
image and
/// the collected count.
void _showBonusDialog(BuildContext context) => showDialog(
  context: context,
  builder: (_) => AlertDialog(
    content: SizedBox(
      height: (MediaQuery.of(context).size.height /
3).clamp(370, 600),
      child: Container(
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
```

```
border: BoxBorder.all(color: _bonus.borderColor,
width: 5),
color: Theme.of(context).canvasColor,
borderRadius: const
BorderRadius.all(Radius.circular(16)),
),
child: Column(
spacing: 20,
mainAxisAlignment: MainAxisAlignment.spaceEvenly,
children: [
_bonusImageWithGlow(scale: 0.2),
Text(
_bonus.name.toUpperCase(),
style: TextStyle(
fontWeight: FontWeight.bold,
fontSize: 20,
color: Colors.grey[800],
),
),
const Spacer(),
Text(
'You have collected $_collectedCount of that
collectable',
style: const TextStyle(
fontSize: 30,
fontWeight: FontWeight.bold,
color: Colors.black,
),
textAlign: TextAlign.center,
),
const Spacer(),
],
),
),
```

```
    ),  
  ),  
);  
  
/// Returns the bonus [Image] asset at the given [scale],  
tinted grey when  
/// the player has not yet collected this bonus.  
Image _bonusImage(double scale) => Image.asset(  
  '$_assetDir${_bonus.name}$ _assetExt',  
  scale: scale,  
  color: _hasCollected ? null : Colors.grey,  
);  
  
/// Stacks [_bonusImage] on top of a coloured glow shadow to  
create a  
/// neon-light effect.  
Widget _bonusImageWithGlow({required double scale}) => Stack(  
  alignment: AlignmentGeometry.center,  
  children: [  
    Container(  
      decoration: BoxDecoration(  
        borderRadius: const  
BorderRadius.all(Radius.circular(16)),  
        boxShadow: [  
          BoxShadow(  
            color: _bonus.bgColor.withAlpha(100),  
            spreadRadius: 4,  
            offset: Offset.zero,  
          ),  
        ],  
      ),  
    ],  
  ),  
  child: ClipRect(  
    clipBehavior: Clip.antiAlias,  
    child: _bonusImage(scale),  
  ),  
);
```

Pacman Android Edition



תומר כץ

```
        ),  
    ),  
    _bonusImage(scale),  
],  
);  
}
```

File: /lib/utils/bonus_type.dart

```
import 'dart:ui';

/// Represents the different collectible bonus items that can
/// appear on the
/// Pac-Man game board (tile value `4` in [defaultMap]).
///
/// Each variant carries two colours used by [BonusCard] to
/// style the card:
/// - [bgColor]      - background glow / shadow colour.
/// - [borderColor] - card border colour.
///
/// The [name] of each variant (e.g. `"apple"`) is also used as
/// the asset
/// filename and as the Firebase Realtime Database key inside
/// `bonusTable`.
enum BonusType {
  /// Classic red apple. Appears early in the game.
  apple(Color(0xFFB71C1C), Color(0xFFFF0000)),

  /// Golden bell. Mid-game bonus item.
  bell(Color(0xFFDBC000), Color(0xFFFFF000)),

  /// Red cherry. The first bonus item in the original arcade
  game.
  cherry(Color(0xFFB71C1C), Color(0xFFFF0000)),

  /// Galaxian flagship. A special high-value bonus.
  galaxian(Color(0xFF0D47A1), Color(0xFF0000FF)),

  /// Blue key. Late-game rare bonus item.
```

```
key(Color(0xFF01579B), Color(0xFF47B7FF)),

/// Green melon. Mid-game fruit bonus.
melon(Color(0xFF81C784), Color(0xFF00FF00)),

/// Orange fruit bonus item.
orange(Color.fromARGB(255, 46, 16, 0), Color(0xFFFFB751)),

/// Red strawberry. Appears after the first level completion.
strawberry(Color(0xFFB71C1C), Color(0xFFFF0000));

// — Fields



---



/// Background glow colour shown behind the bonus image.
final Color bgColor;

/// Border colour used on [BonusCard] and the detail dialog.
final Color borderColor;

// — Constructor



---



const BonusType(this.bgColor, this.borderColor);
}
```

File: /lib/utils/constants.dart

```
import 'package:flutter/material.dart';
import 'package:pacman_application/utils/bonus_type.dart';

// — Game map



---



/// The default 31 × 28 tile layout of the Pac-Man level.
///
/// Cell values:
/// | Value | Meaning |
/// |-----|-----|
/// | 0     | Empty space |
/// | 1     | Wall |
/// | 2     | Small dot |
/// | 3     | Super dot (power-up) |
/// | 4     | Bonus item spawn |
/// | 5     | Tunnel |
/// | 6     | Ghost house door |
/// | 7     | Ghost house interior |
/// | 8     | Player starting pos |
const List<List<int>> defaultMap = [
  [1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1],
  [1,2,2,2,2,2,2,2,2,2,2,2,1,1,2,2,2,2,2,2,2,2,2,2,2,2,2,2,1],
  [1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,2,1,1,1,1,2,1,1,1,1,2,1,1,1,2,1],
  [1,3,1,1,1,1,2,1,1,1,1,1,2,1,1,2,1,1,1,1,2,1,1,1,1,3,1,1,1,3,1],
  [1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,2,1,1,1,1,2,1,1,1,1,2,1,1,1,2,1],
  [1,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,1],
  [1,2,1,1,1,1,2,1,1,2,1,1,1,1,1,1,1,1,2,1,1,2,1,1,1,1,2,1,1,2,1],
  [1,2,1,1,1,1,2,1,1,2,1,1,1,1,1,1,1,1,2,1,1,2,1,1,1,1,2,1,1,2,1],
  [1,2,2,2,2,2,2,1,1,2,2,2,2,1,1,2,2,2,2,1,1,2,2,2,2,2,2,2,2,2,1],

```

```
[1,1,1,1,1,1,2,1,1,1,1,1,0,1,1,0,1,1,1,1,1,2,1,1,1,1,1,1],
[5,0,0,0,0,1,2,1,1,1,1,1,0,1,1,0,1,1,1,1,1,2,1,0,0,0,0,5],
[5,0,0,0,0,1,2,1,1,0,0,0,0,0,0,0,0,0,0,1,1,2,1,0,0,0,0,5],
[5,0,0,0,0,1,2,1,1,0,1,1,1,6,6,1,1,1,0,1,1,2,1,0,0,0,0,5],
[1,1,1,1,1,1,2,1,1,0,1,7,7,7,7,7,7,1,0,1,1,2,1,1,1,1,1,1],
[5,0,0,0,0,0,2,0,0,0,1,7,7,7,7,7,7,1,0,0,0,2,0,0,0,0,0,5],
[1,1,1,1,1,1,2,1,1,0,1,7,7,7,7,7,7,1,0,1,1,2,1,1,1,1,1,1],
[5,0,0,0,0,1,2,1,1,0,1,1,1,1,1,1,1,1,0,1,1,2,1,0,0,0,0,5],
[5,0,0,0,0,1,2,1,1,0,0,0,0,4,4,0,0,0,0,1,1,2,1,0,0,0,0,5],
[5,0,0,0,0,1,2,1,1,0,1,1,1,1,1,1,1,1,0,1,1,2,1,0,0,0,0,5],
[1,1,1,1,1,1,2,1,1,0,1,1,1,1,1,1,1,1,0,1,1,2,1,1,1,1,1,1],
[1,2,2,2,2,2,2,2,2,2,2,2,2,1,1,2,2,2,2,2,2,2,2,2,2,2,2,1],
[1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1],
[1,2,1,1,1,1,2,1,1,1,1,1,2,1,1,2,1,1,1,1,1,2,1,1,1,1,2,1],
[1,3,2,2,1,1,2,2,2,2,2,2,2,8,8,2,2,2,2,2,2,2,2,1,1,2,2,3,1],
[1,1,1,2,1,1,2,1,1,2,1,1,1,1,1,1,1,1,2,1,1,2,1,1,2,1,1,1],
[1,1,1,2,1,1,2,1,1,2,1,1,1,1,1,1,1,1,2,1,1,2,1,1,2,1,1,1],
[1,2,2,2,2,2,2,1,1,2,2,2,2,1,1,2,2,2,2,1,1,2,2,2,2,2,2,1],
[1,2,1,1,1,1,1,1,1,1,1,1,2,1,1,2,1,1,1,1,1,1,1,1,1,1,2,1],
[1,2,1,1,1,1,1,1,1,1,1,1,2,1,1,2,1,1,1,1,1,1,1,1,1,1,2,1],
[1,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,1],
[1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1],
];
```

```
// — Character colours
```

```
/// Default colour of Pac-Man. Can be overridden in [Settings].
const Color pacmanColor = Color(0xFFFFF000);
```

```
/// Default colour of Blinky (red ghost). Can be overridden in
[Settings].
const Color blinkyColor = Color(0xFFFF0000);
```

```
/// Default colour of Pinky (pink ghost). Can be overridden in
[Settings].
const Color pinkyColor = Color(0xFFFFB7FF);

/// Default colour of Inky (cyan ghost). Can be overridden in
[Settings].
const Color inkyColor = Color(0xFF00FFFF);

/// Default colour of Clyde (orange ghost). Can be overridden in
[Settings].
const Color clydeColor = Color(0xFFFFB751);

/// Default colour of the maze walls. Can be overridden in
[Settings].
const Color mapColor = Color(0xFF2121FF);

// — Text styles


---



/// Bold Pac-Font style used for all screen headings in the app
bar.
const TextStyle headerTextStyle = TextStyle(
  fontFamily: 'Pacfont',
  fontSize: 25,
  fontWeight: FontWeight.bold,
);

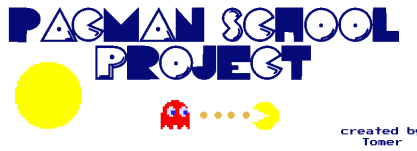
// — Anonymous / guest defaults


---



/// Firebase UID reserved for the anonymous guest account stored
in the DB.
const String anonymousId = '0';
```

Pacman Android Edition



תומר כץ

```
/// Display name shown when no user is signed in.
const String anonymousName = 'Guest';

/// High-score placeholder for a guest session.
const int anonymousHighScore = 0;

/// E-mail placeholder for a guest session.
const String anonymousEmail = 'name@gmail.com';

/// Bonus-table initial values for a guest session - all bonuses
at zero.
const Map<BonusType, int> anonymousBonusTable = {
  BonusType.apple: 0,
  BonusType.bell: 0,
  BonusType.cherry: 0,
  BonusType.galaxian: 0,
  BonusType.key: 0,
  BonusType.melon: 0,
  BonusType.orange: 0,
  BonusType.strawberry: 0,
};
```

File: /lib/utils/game_displayer.dart

```
import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:pacman_application/database/session.dart';
import 'package:pacman_application/utils/bonus_type.dart';
import 'package:pacman_application/game/game_manager.dart';
import 'package:pacman_application/screens/home_screen.dart';

/// Hosts and manages a live Pac-Man game session.
///
/// Responsibilities:
/// - Creates and initialises a [GameManager] for the current
  game.
/// - Pre-creates the next [GameManager] so a retry starts
  instantly.
/// - Wires up game-event callbacks: [_onGameOver], [_onRetry],
  [_onExit],
///   and [_onBonusCollected].
/// - Persists high-score and collectable updates back to
  [currentUser] for
///   authenticated players.
class GameDisplayer extends StatefulWidget {
  const GameDisplayer({super.key});

  @override
  State<GameDisplayer> createState() => GameDisplayerState();
}

class GameDisplayerState extends State<GameDisplayer> {
```

// — Fields

/// The [GameManager] running the current game session.

late GameManager _gameManager;

/// A fully-constructed [GameManager] ready to start the
moment the player

/// requests a retry, eliminating any initialisation delay.

late GameManager _nextGame;

/// Audio player used by the game (currently stopped on entry
so the home

/// screen music does not overlap).

final AudioPlayer _audioPlayer = AudioPlayer();

// — Lifecycle

@override

void initState() {
 super.initState();

_audioPlayer.setReleaseMode(ReleaseMode.loop);

_audioPlayer.stop();

_gameManager = _buildGameManager();

_nextGame = _buildGameManager();

_gameManager.initGame();

}

@override

void dispose() {

SystemChrome.setPreferredOrientations([

```
        DeviceOrientation.portraitDown,  
        DeviceOrientation.portraitUp,  
    );  
    super.dispose();  
}
```

```
// — Build
```

```
@override  
Widget build(BuildContext context) =>  
    Scaffold(body: _gameManager.currentScreen);
```

```
// — Private helpers
```

```
    /// Factory that creates a [GameManager] with all callbacks  
    bound to this
```

```
    /// state. Extracted to avoid duplicating the constructor  
    call.
```

```
    GameManager _buildGameManager() => GameManager(  
        highScore: () => currentUser.highScore,  
        onGameOver: _onGameOver,  
        onRetry: _onRetry,  
        onExit: _onExit,  
        onGettingBonus: _onBonusCollected,  
    );
```

```
    /// Called by [GameManager] when the game ends with [score].  
    ///
```

```
    /// Updates [currentUser]'s high score if [score] is a new  
    personal best,
```

```
    /// and the player is not in a guest session.
```

```
    void _onGameOver(int score) {
```

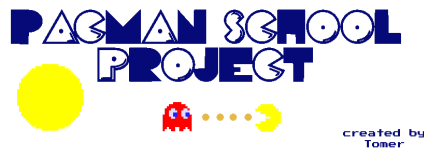
```
    if (score > currentUser.highScore && !isAnonymous) {
        currentUser.highScore = score;
    }
}

/// Called by [GameManager] when the player taps "Retry".
///
/// Swaps in the pre-built [_nextGame] as the active session
and creates a
/// fresh replacement ready for the next potential retry.
void _onRetry() {
    setState(() {
        _gameManager = _nextGame;
        _gameManager.initGame();
        _nextGame = _buildGameManager();
    });
}

/// Called by [GameManager] when the player taps "Exit".
///
/// Navigates back to [HomeScreen].
void _onExit() {
    Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (_) => const HomeScreen()),
    );
}

/// Called by [GameManager] when the player collects a bonus
of [bonusType].
///
/// Increments the bonus count in [currentUser] for
authenticated players.
void _onBonusCollected(BonusType bonusType) {
```


Pacman Android Edition



תומר כץ

```
if (!isAnonymous) {  
    currentUser.addBonus(bonusType);  
}  
}  
}
```

File: /lib/utils/loading_screen.dart

```
import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:pacman_application/utils/appbar.dart';
import 'package:pacman_application/screens/home_screen.dart';

/// A brief transitional screen displayed while the app
navigates to
/// [HomeScreen] after a successful sign-in.
///
/// After 500 ms it automatically replaces itself with
[HomeScreen],
/// giving any in-flight data fetches a moment to begin before
the
/// home screen is rendered.
class LoadingScreen extends StatefulWidget {
  const LoadingScreen({super.key});

  @override
  State<LoadingScreen> createState() => LoadingScreenState();
}

class LoadingScreenState extends State<LoadingScreen> {
  // — Fields

  /// Timer that triggers navigation to [HomeScreen] after a
short delay.
  late final Timer _timer;
```

```
// — Lifecycle
```

```
@override
void initState() {
  super.initState();

  _timer = Timer(const Duration(milliseconds: 500), () {
    Navigator.of(context).pushReplacement(
      MaterialPageRoute(builder: (_) => const HomeScreen()),
    );
  });
}
```

```
@override
void dispose() {
  _timer.cancel();

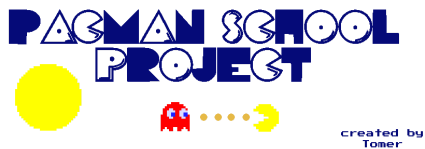
  SystemChrome.setPreferredOrientations([
    DeviceOrientation.portraitDown,
    DeviceOrientation.portraitUp,
  ]);

  super.dispose();
}
```

```
// — Build
```

```
@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(context: context),
  body: const Center(child: Text('Loading...')),
);
```

Pacman Android Edition



תומר כץ

```
);  
}
```

File: /lib/utils/sidebar.dart

```
import 'package:flutter/material.dart';
import 'package:pacman_application/database/session.dart';
import 'package:pacman_application/screens/read_me.dart';
import
'package:pacman_application/screens/collectables_screen.dart';
import 'package:pacman_application/utils/game_displayer.dart';
import 'package:pacman_application/screens/home_screen.dart';
import
'package:pacman_application/screens/leaderboard_screen.dart';
import 'package:pacman_application/screens/settings.dart';

/// A navigation [Drawer] shown when the user swipes from the
left edge or
/// taps the hamburger icon in any screen's [AppBar].
///
/// Displays the current user's name at the top, followed by
tappable rows
/// that navigate (via `pushReplacement`) to each major section
of the app:
/// Home, Play, Read Me, Leaderboard, Collectables, and
Settings.
class Sidebar extends StatefulWidget {
  const Sidebar({super.key});

  @override
  State<Sidebar> createState() => SidebarState();
}

class SidebarState extends State<Sidebar> {
```

// — Build

```
@override
Widget build(BuildContext context) => Drawer(
  child: Container(
    padding: const EdgeInsets.symmetric(vertical: 20,
horizontal: 50),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      spacing: 50,
      children: [
        const SizedBox(height: 30),
```

```
// — Header
```

```
Text(
  'Welcome ${currentUser.name}!',
  style: const TextStyle(fontSize: 26),
),
```

```
// — Home
```

```
_navRow(
  context,
  icon: Icons.home,
  label: 'Home',
  destination: const HomeScreen(),
),
```

```
// — Play
```

```
_navRow(
  context,
```

```
        icon: Icons.videogame_asset,  
        label: 'Play a Game',  
        destination: const GameDisplayer(),  
    ),
```

```
// — Read Me
```

```
_navRow(  
    context,  
    icon: Icons.favorite_rounded,  
    label: 'Read Me',  
    destination: const ReadMe(),  
),
```

```
// — Leaderboard
```

```
_navRow(  
    context,  
    icon: Icons.leaderboard,  
    label: 'Leaderboard',  
    destination: const LeaderboardScreen(),  
),
```

```
// — Collectables
```

```
_navRow(  
    context,  
    icon: Icons.grid_view,  
    label: 'Collectables',  
    destination: const CollectablesScreen(),  
),
```

```
// — Settings
```

```
        _navRow(  
            context,  
            icon: Icons.settings,  
            label: 'Settings',  
            destination: const Settings(),  
        ),  
    ],  
),  
),  
);
```

```
// — Private helpers
```

```
/// Builds a single navigation row with an [icon], a [label],  
and a
```

```
/// right-pointing chevron. Tapping the row replaces the  
current route
```

```
/// with [destination].
```

```
Widget _navRow(  
    BuildContext context, {  
    required IconData icon,  
    required String label,  
    required Widget destination,  
}) {  
    return GestureDetector(  
        onTap: () => Navigator.pushReplacement(  
            context,  
            MaterialPageRoute(builder: (_) => destination),  
        ),  
        child: Row(  
            spacing: 5,  
            children: [  
                Icon(icon),
```



```
Text(label),  
Expanded(  
  child: Row(  
    mainAxisAlignment: MainAxisAlignment.end,  
    children: const [Icon(Icons.arrow_forward_ios)],  
  ),  
,  
],  
,  
);  
}  
}
```

File: /lib/utils/util.dart

```
import 'dart:math';

/// Utility functions shared across the application.

/// Returns the Euclidean distance from the origin to the point
/// ([x], [y]).
///
/// Equivalent to `sqrt(x2 + y2)`. Useful for collision
/// detection and
/// proximity checks inside the game engine.
double dist(num x, num y) {
  return sqrt((x * x + y * y).abs());
}

/// Casts [object] to [T], throwing a [TypeError] if the cast
/// fails.
///
/// Use this when you are certain of the runtime type and want
/// an explicit
/// error on mismatch rather than a silent `null`.
T cast<T extends Object>(dynamic object) => object as T;

/// Attempts to cast [object] to [T], returning `null` instead
/// of throwing
/// if [object] is not an instance of [T].
///
/// Prefer this when reading values from Firebase snapshots
/// whose type is
/// not guaranteed at compile time.
```

```
T? tryCast<T extends Object>(dynamic object) => object is T ?  
object : null;
```

```
-----  
File: /lib/screens/collectables_screen.dart  
-----
```

```
import 'package:flutter/material.dart';  
import 'package:flutter/services.dart';  
import 'package:pacman_application/utils/constants.dart';  
import 'package:pacman_application/utils/bonus_type.dart';  
import 'package:pacman_application/utils/appbar.dart';  
import 'package:pacman_application/utils/sidebar.dart';  
import 'package:pacman_application/utils/bonus_card.dart';  
  
/// Displays the player's full collectable item collection in a  
2-column grid.  
///  
/// Each grid cell is rendered by a [BonusCard] widget. Cards  
for items the  
/// player has not yet obtained are shown in greyscale with a  
strikethrough  
/// label; collected items are shown in full colour and open a  
detail dialog  
/// on tap.  
class CollectablesScreen extends StatefulWidget {  
  const CollectablesScreen({super.key});  
  
  @override  
  State<CollectablesScreen> createState() =>  
    CollectablesScreenState();  
}  
  
class CollectablesScreenState extends State<CollectablesScreen>  
{
```

// — Lifecycle

```
@override
void dispose() {
  // Restore portrait orientation when leaving the screen.
  SystemChrome.setPreferredOrientations([
    DeviceOrientation.portraitDown,
    DeviceOrientation.portraitUp,
  ]);
  super.dispose();
}
```

// — Build

```
@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(
    context: context,
    header: Text('COLLECTABLES', style: headerTextStyle),
  ),
  drawer: const Sidebar(),
  body: SafeArea(
    child: Container(
      padding: const EdgeInsets.symmetric(vertical: 20,
horizontal: 50),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Text(
            'Here is all the collectables you have got!',
            style: TextStyle(fontSize: 20, color:
Colors.black),
```

```
        textAlign: TextAlign.center,  
    ),  
  
    const SizedBox(height: 20),  
  
    // 2-column grid of one BonusCard per BonusType.  
    GridView.builder(  
        shrinkWrap: true,  
        itemCount: BonusType.values.length,  
        gridDelegate: const  
SliverGridDelegateWithFixedCrossAxisCount(  
            crossAxisCount: 2,  
        ),  
        itemBuilder: (_, i) => BonusCard(i),  
    ),  
    ],  
    ),  
    ),  
    ),  
);  
}
```

File: /lib/screens/home_screen.dart

```
import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:pacman_application/utils/constants.dart';
import 'package:pacman_application/database/session.dart';
import 'package:pacman_application/utils/game_displayer.dart';
import 'package:pacman_application/utils/appbar.dart';
import 'package:pacman_application/utils/sidebar.dart';

/// The main landing screen shown after a successful sign-in.
///
/// Displays the player's avatar and username, their current
high score, a
/// circular progress indicator summarising collectable
completion, and a
/// "QUICK PLAY" button that launches the game directly.
///
/// Background music loops while this screen is active and is
stopped when
/// the player navigates away.
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => HomeScreenState();
}

class HomeScreenState extends State<HomeScreen> {
  // — Fields
```

```
    /// Fraction of bonus types that the player has collected at
    least once,
    /// in the range [0, 1]. Drives the
    [CircularProgressIndicator].
    double _progress = 0;

    /// Audio player that loops the home-screen background track.
    final AudioPlayer _audioPlayer = AudioPlayer();

    // — Lifecycle



---



    @override
    void initState() {
        super.initState();

        _audioPlayer.setReleaseMode(ReleaseMode.loop);
        _audioPlayer.play(AssetSource('audio/Home_Music.mp3'));

        _computeBonusProgress();
    }

    @override
    void dispose() {
        // Restore portrait orientation when leaving the screen.
        SystemChrome.setPreferredOrientations([
            DeviceOrientation.portraitDown,
            DeviceOrientation.portraitUp,
        ]);
        super.dispose();
    }
```

// — Build

```
@override
Widget build(BuildContext context) {
  // If the user data has not finished loading yet, trigger a
  rebuild once
  // the async fetch in GameUser.fromUid completes.
  if (!currentUser.haveInitialized()) {
    setState(() {});
  }

  return Scaffold(
    appBar: AppBar(
      context: context,
      header: Text('HOME SCREEN', style: headerTextStyle),
    ),
    drawer: const Sidebar(),
    body: SafeArea(
      child: Container(
        padding: const EdgeInsets.symmetric(vertical: 20,
horizontal: 50),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisAlignment: MainAxisAlignment.spaceAround,
          spacing: 10,
          children: [
            const SizedBox(height: 25),

            // — Player identity row

            Row(
              mainAxisAlignment: MainAxisAlignment.start,
              spacing: 10,
```

```
        children: [
            const CircleAvatar(),
            Text(
                currentUser.name,
                style: const TextStyle(
                    fontSize: 20,
                    fontWeight: FontWeight.bold,
                ),
            ),
        ],
    ),

    const Spacer(),

    // — High-score row

    Row(
        mainAxisAlignment:
MainAxisAlignment.spaceAround,
        spacing: 20,
        children: [
            Icon(Icons.emoji_events, color:
Colors.amber[600], size: 80),
            Text(
                'Your high score\nis currently
${currentUser.highScore},\ncongratulations!',
                style: const TextStyle(
                    fontWeight: FontWeight.bold,
                    fontSize: 20,
                ),
            ),
        ],
    ),
```

```
const Spacer(),

// — Collectable progress row

```

```
    Row(
      mainAxisAlignment:
MainAxisAlignment.spaceAround,
      spacing: 20,
      children: [
        Text(
          'You have collected\n${(_progress *
100).toStringAsFixed(0)}% of all\ncollectables',
          style: const TextStyle(
            fontWeight: FontWeight.bold,
            fontSize: 20,
          ),
        ),
      ],
    ),
    Expanded(
      child: AspectRatio(
        aspectRatio: 1,
        child: CircularProgressIndicator(
          value: _progress,
          color: Colors.blue[900],
          strokeWidth: 10,
        ),
      ),
    ),
  ],
),

const Spacer(),

// — Quick Play button

```

```
GestureDetector(  
  onTap: () => Navigator.pushReplacement(  
    context,  
    MaterialPageRoute(builder: (_) => const  
GameDisplay()),  
  ),  
  child: Container(  
    decoration: BoxDecoration(  
      border: BoxBorder.all(color: Colors.black,  
width: 2),  
    ),  
    child: Padding(  
      padding: const EdgeInsets.all(8.0),  
      child: Row(  
        mainAxisAlignment:  
MainAxisAlignment.center,  
        spacing: 10,  
        children: [  
          Icon(  
            Icons.videogame_asset,  
            size: 35,  
            color: Theme.of(context).primaryColor,  
          ),  
          Text(  
            'QUICK PLAY',  
            style: TextStyle(  
              fontSize: 28,  
              fontFamily: 'PacFont',  
              color:  
Theme.of(context).primaryColor,  
            ),  
          ),  
        ],  
      ),  
    ),  
  ),  
),
```

```
        ),
    ),
),

    const Spacer(),
],
),
),
),
);
}

// — Private helpers

/// Calculates [_progress] as the fraction of distinct bonus
types that the
/// player has collected at least once.
void _computeBonusProgress() {
    int collectedCount = 0;
    for (final entry in currentUser.bonusTable.entries) {
        if (entry.value > 0) collectedCount++;
    }
    _progress = collectedCount / currentUser.bonusTable.length;
}
}
```

File: /lib/screens/leaderboard_screen.dart

```
import 'dart:async';

import 'package:firebase_database/firebase_database.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:pacman_application/utils/constants.dart';
import 'package:pacman_application/database/game_user.dart';
import 'package:pacman_application/database/session.dart';
import 'package:pacman_application/utils/appbar.dart';
import 'package:pacman_application/utils/sidebar.dart';

/// Shows all registered players ranked by their all-time high
/// score.
///
/// Data is fetched from Firebase Realtime Database once when
/// the screen
/// mounts. The top three entries are highlighted with gold,
/// silver, and
/// bronze trophy icons respectively; remaining entries show a
/// numbered
/// circular badge. The currently signed-in player's row is
/// highlighted.
class LeaderboardScreen extends StatefulWidget {
  const LeaderboardScreen({super.key});

  @override
  State<LeaderboardScreen> createState() =>
    LeaderboardScreenState();
}
```

```
class LeaderboardScreenState extends State<LeaderboardScreen> {  
    // — Fields  
  
    /// All users fetched from the database. Sorted by high score  
    at build time.  
    final List<GameUser> _users = [];  
  
    // — Lifecycle  
  
    @override  
    void initState() {  
        super.initState();  
  
        _loadUsers();  
  
        // Trigger a rebuild after 500 ms to display any users whose  
        async data  
        // fetch has completed by then.  
        Timer(const Duration(milliseconds: 500), () {  
            if (mounted) setState(() {});  
        });  
    }  
  
    @override  
    void dispose() {  
        SystemChrome.setPreferredOrientations([  
            DeviceOrientation.portraitDown,  
            DeviceOrientation.portraitUp,  
        ]);  
        super.dispose();  
    }  
}
```

// — Build

```
@override
Widget build(BuildContext context) {
  // Sort descending by high score each time the widget
  rebuilds.
  final sortedUsers = List<GameUser>.from(_users)
    ..sort((a, b) => b.highScore.compareTo(a.highScore));

  return Scaffold(
    appBar: AppBar(
      context: context,
      header: Text('LEADERBOARD', style: headerTextStyle),
    ),
    drawer: const Sidebar(),
    body: SafeArea(
      child: ListView.builder(
        itemCount: sortedUsers.length,
        itemBuilder: (_, index) {
          final user = sortedUsers[index];
          final rank = index + 1;

          return ListTile(
            // Highlight the current user's row.
            selected: user == currentUser,
            leading: _rankBadge(rank),
            title: Text(
              user.name,
              style: const TextStyle(
                fontWeight: FontWeight.bold,
                fontSize: 18,
              ),
            ),
          ),
        ),
      ),
    ),
  );
}
```

```
        trailing: Text(  
            '${user.highScore}',  
            style: TextStyle(  
                color: Colors.yellow[900],  
                fontSize: 20,  
                fontWeight: FontWeight.bold,  
            ),  
        ),  
    ),  
);  
},  
),  
),  
);  
}  

```

```
// — Private helpers
```

```
    /// Fetches all user records from `users/` in Firebase and  
    populates  
    /// [_users], skipping the anonymous-guest placeholder (uid  
    `""0"``).  
    void _loadUsers() async {  
        final ref = FirebaseDatabase.instance.ref('users');  
        final snapshot = await ref.get();  
  
        if (snapshot.exists) {  
            for (final child in snapshot.children) {  
                if (child.key != '0') {  
                    _users.add(GameUser.fromUid(uid: child.key!));  
                }  
            }  
        }  
    }  
}
```



```
    if (mounted) setState(() {});
  }

  /// Returns a rank indicator widget.
  ///
  /// Ranks 1-3 receive a trophy icon in gold, silver, or
  bronze. All other
  /// ranks receive a circular avatar with the rank number.
  Widget _rankBadge(int rank) => Container(
    margin: const EdgeInsets.all(2),
    child: switch (rank) {
      1 => const Icon(Icons.emoji_events, color:
Color(0xFFFFD700), size: 32),
      2 => const Icon(Icons.emoji_events, color:
Color(0xFFC0C0C0), size: 28),
      3 => const Icon(Icons.emoji_events, color:
Color(0xFFCD7F32), size: 24),
      _ => CircleAvatar(
        backgroundColor: Colors.grey[800],
        radius: 15,
        child: Text(
          '$rank',
          style: const TextStyle(color: Colors.white, fontSize:
12),
        ),
      ),
    },
  );
}
```

File: /lib/screens/login.dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:google_sign_in/google_sign_in.dart';
import 'package:pacman_application/utils/constants.dart';
import 'package:pacman_application/database/game_user.dart';
import 'package:pacman_application/database/session.dart';
import 'package:pacman_application/utils/bonus_type.dart';
import 'package:pacman_application/utils/appbar.dart';
import 'package:pacman_application/utils/loading_screen.dart';
import 'package:pacman_application/screens/signup.dart';
import 'package:pacman_application/screens/home_screen.dart';
import 'package:shared_preferences/shared_preferences.dart';

/// The sign-in screen shown on first launch and after sign-out.
///
/// Supports three authentication paths:
/// 1. **Email + password** – via
[FirebaseAuth.signInWithEmailAndPassword].
/// 2. **Google Sign-In** – via the `google_sign_in` package.
/// 3. **Guest / anonymous** – skips authentication entirely;
score and
/// collectables are not persisted.
///
/// On a successful sign-in the screen saves the user's details
to
/// [SharedPreferences] (enabling fast re-launch) and navigates
to
/// [LoadingScreen] → [HomeScreen].
```

```
class Login extends StatefulWidget {
    const Login({super.key});

    @override
    State<Login> createState() => LoginState();
}

class LoginState extends State<Login> {
    // — Fields

    /// Controller for the email input field.
    final TextEditingController _emailController =
    TextEditingController();

    /// Controller for the password input field.
    final TextEditingController _passwordController =
    TextEditingController();

    /// Whether the Google Sign-In flow is currently in progress.
    bool _googleLoading = false;

    // — Lifecycle

    @override
    void initState() {
        super.initState();
        // Uncomment to re-enable automatic sign-in from cached
        credentials:
        // _checkPrefs();
    }

    @override
```

```
void dispose() {  
  _emailController.dispose();  
  _passwordController.dispose();  
  
  SystemChrome.setPreferredOrientations([  
    DeviceOrientation.portraitDown,  
    DeviceOrientation.portraitUp,  
  ]);  
  
  super.dispose();  
}
```

```
// — Build
```

```
@override  
Widget build(BuildContext context) => StreamBuilder(  
  stream: FirebaseAuth.instance.authStateChanges(),  
  builder: (context, snapshot) {  
    // Already initialised (e.g. hot-reload) – go straight to  
loading.  
    if (currentUser.haveInitialized()) {  
      return const LoadingScreen();  
    }  
  
    // Firebase reports a signed-in user; reuse their session.  
    if (snapshot.hasData) {  
      final user = snapshot.data!;  
      isAnonymous = false;  
      currentUser = GameUser.fromUid(uid: user.uid);  
      _savePrefs(currentUser);  
      return const LoadingScreen();  
    }  
  }  
);
```

```
return Scaffold(  
  appBar: AppBar(  
    context: context,  
    header: Text('LOGIN', style: headerTextStyle),  
    canLogout: false,  
  ),  
  body: SafeArea(  
    child: Container(  
      padding: const EdgeInsets.symmetric(vertical: 20,  
horizontal: 50),  
      child: SingleChildScrollView(  
        child: Column(  
          children: [  
            const SizedBox(height: 150),  
            Column(  
              spacing: 10,  
              children: [  
                Text(  
                  'Hi there! Nice to see you again.',  
                  style: TextStyle(fontSize: 12, color:  
Colors.grey[800]),  
                ),  
  
                const SizedBox(height: 15),  
  
                // — Email field  
  
                Column(  
                  crossAxisAlignment:  
CrossAxisAlignment.start,  
                  spacing: 2,  
                  children: [  
                    Text(  
                      'Email',
```

```
                style: TextStyle(
                  color:
Theme.of(context).primaryColor,
                ),
            ),
            TextField(
                controller: _emailController,
                onSubmitted: (_) => setState(() {}),
            ),
        ],
    ),

    // — Password field
    Column(
        crossAxisAlignment:
CrossAxisAlignment.start,
        children: [
            Text(
                'Password',
                style: TextStyle(
                    color:
Theme.of(context).primaryColor,
                ),
            ),
            TextField(
                controller: _passwordController,
                obscureText: true,
                onSubmitted: (_) => setState(() {}),
            ),
        ],
    ),

    const SizedBox(height: 15),
```

```
// — Sign In button

ElevatedButton(
  onPressed: () async {
    if (_emailController.text.isNotEmpty
    &&
    _passwordController.text.isNotEmpty) {
      try {
        final credentials = await
        FirebaseAuth.instance
          .signInWithEmailAndPassword(
            email:
            _emailController.text,
            password:
            _passwordController.text,
          );

        isAnonymous = false;
        currentUser = GameUser.fromUid(
          uid: credentials.user!.uid,
        );
        _savePrefs(currentUser);

        if (context.mounted) {
          Navigator.pushReplacement(
            context,
            MaterialPageRoute(
              builder: (_) => const
              HomeScreen(),
            ),
          );
        }
      }
    }
  }
);
```

```
        } on FirebaseAuthException catch (e)
    {
        if (kDebugMode) print('error:
    ${e.code}');

        } catch (e) {
            if (kDebugMode) print(e);
        }
    },
    style: ElevatedButton.styleFrom(
        minimumSize: const Size(1000, 40),
        shape: RoundedRectangleBorder(
            borderRadius:
    BorderRadiusGeometry.circular(8),
        ),
        backgroundColor:
            _emailController.text.isEmpty ||

    _passwordController.text.isEmpty
            ? Theme.of(context).focusColor
            : Theme.of(context).primaryColor,
    ),
    child: const Text(
        'Sign In',
        style: TextStyle(color: Colors.white),
    ),
),

// — Divider

Row(
    children: [
        Expanded(
```



```
        child: Divider(color:
Colors.grey.shade300),
      ),
      Padding(
        padding: const
EdgeInsets.symmetric(horizontal: 12),
        child: Text(
          'or',
          style: TextStyle(
            fontSize: 12,
            color: Colors.grey,
          ),
        ),
      ),
      Expanded(
        child: Divider(color:
Colors.grey.shade300),
      ),
    ],
  ),

  // — Google Sign-In button
  OutlinedButton(
    onPressed:
      _googleLoading ? null :
      _signInWithGoogle,
    style: OutlinedButton.styleFrom(
      minimumSize: const Size(1000, 40),
      shape: RoundedRectangleBorder(
        borderRadius:
BorderRadiusGeometry.circular(8),
      ),
    ),
```

```
        side: BorderSide(color:
Colors.grey.shade300),
        backgroundColor: Colors.white,
      ),
      child: _googleLoading
        ? SizedBox(
            height: 18,
            width: 18,
            child:
CircularProgressIndicator(
                strokeWidth: 2,
                color:
Theme.of(context).primaryColor,
            ),
        )
        : Row(
            mainAxisAlignment:
MainAxisAlignment.center,
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
Image.asset('assets/GoogleG.png'),
            const SizedBox(width: 10),
            const Text(
                'Continue with Google',
                style: TextStyle(
                    color: Colors.black87,
                    fontSize: 14,
                ),
            ),
        ],
    ),
),
```

```
const SizedBox(height: 10),

// — Sign Up link

```

```
GestureDetector(
  onTap: () => Navigator.pushReplacement(
    context,
    MaterialPageRoute(builder: (_) =>
const Signup()),
  ),
  child: Text(
    'Sign Up',
    style: TextStyle(
      color:
Theme.of(context).primaryColor,
    ),
  ),
),

const SizedBox(height: 10),

// — Guest entry

```

```
GestureDetector(
  onTap: () {
    isAnonymous = true;
    _removePrefs();
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (_) => const
HomeScreen(),
      ),
    );
  },
);
```

```
        },  
        child: const Text('Enter as a guest'),  
      ),  
    ],  
  ),  
],  
,  
},  
);
```

```
// — Private methods
```

```
    /// Checks [SharedPreferences] for a previously saved user  
session and  
    /// signs in automatically if one exists, bypassing the login  
form.  
    ///  
    /// Currently unused – uncomment the call in [initState] to  
enable.  
    // ignore: unused_element  
void _checkPrefs() async {  
    final prefs = await SharedPreferences.getInstance();  
  
    final lastUserId = prefs.getString('lastUser/uid');  
    final lastUserName = prefs.getString('lastUser/name');  
    final lastUserHighScore =  
prefs.getInt('lastUser/highScore');  
    final lastUserEmail = prefs.getString('lastUser/email');
```

```
final entries = await Future.wait(
    BonusType.values.map((bonus) =>
        _bonusEntryFromPrefs(bonus, prefs)),
    );

if (lastUserId != null && mounted) {
    isAnonymous = false;
    currentUser = GameUser(
        uid: lastUserId,
        name: lastUserName ?? currentUser.name,
        highScore: lastUserHighScore ?? currentUser.highScore,
        email: lastUserEmail ?? currentUser.email,
        bonusTable: <BonusType, int>{}..addEntries(entries),
    );
    Navigator.of(context).pushReplacement(
        MaterialPageRoute(builder: (_) => const
LoadingScreen()),
    );
}

/// Reads the stored count for [bonus] from [prefs] and
returns it as a
/// [MapEntry] for reconstructing the bonus table.
Future<MapEntry<BonusType, int>> _bonusEntryFromPrefs(
    BonusType bonus,
    SharedPreferences prefs,
) async {
    return MapEntry(
        bonus,
        prefs.getInt('lastUser/bonusTable/${bonus.name}') ?? 0,
    );
}
```

```
/// Persists all relevant fields of [user] to
[SharedPreferences] so the
/// player can be automatically signed in on the next launch.
void _savePrefs(GameUser user) async {
    final prefs = await SharedPreferences.getInstance();

    await prefs.setString('lastUser/uid', user.uid);
    await prefs.setString('lastUser/name', user.name);
    await prefs.setInt('lastUser/highScore', user.highScore);
    await prefs.setString('lastUser/email', user.email);

    for (final entry in user.bonusTable.entries) {
        await prefs.setInt(
            'lastUser/bonusTable/${entry.key.name}',
            entry.value,
        );
    }
}

/// Removes all "lastUser/*" keys from [SharedPreferences],
preventing
/// automatic sign-in on the next launch (used when the player
signs out).
void _removePrefs() async {
    final prefs = await SharedPreferences.getInstance();

    await prefs.remove('lastUser/uid');
    await prefs.remove('lastUser/name');
    await prefs.remove('lastUser/highScore');
    await prefs.remove('lastUser/email');

    for (final bonus in BonusType.values) {
        await prefs.remove('lastUser/bonusTable/${bonus.name}');
    }
}
```

```
}

    /// Initiates the Google Sign-In flow and signs the resulting
    credential
    /// into Firebase Auth.
    ///
    /// On success, sets [currentUser] and navigates to
    [LoadingScreen].
    void _signInWithGoogle() async {
        setState(() => _googleLoading = true);

        try {
            final GoogleSignInAccount? googleUser = await
GoogleSignIn().signIn();

            // User cancelled the account picker.
            if (googleUser == null) {
                setState(() => _googleLoading = false);
                return;
            }

            final GoogleSignInAuthentication googleAuth =
                await googleUser.authentication;

            final AuthCredential credential =
GoogleAuthProvider.credential(
                accessToken: googleAuth.accessToken,
                idToken: googleAuth.idToken,
            );

            final UserCredential userCredential =
                await
FirebaseAuth.instance.signInWithCredential(credential);
```

```
isAnonymous = false;
currentUser = GameUser.fromUid(uid:
userCredential.user!.uid);
_savePrefs(currentUser);

if (mounted) {
  Navigator.pushReplacement(
    context,
    MaterialPageRoute(builder: (_) => const
LoadingScreen()),
  );
}
} on FirebaseAuthException catch (e) {
  if (kDebugMode) print('Google sign-in Firebase error:
${e.code}');
  } catch (e) {
    if (kDebugMode) print('Google sign-in error: $e');
  } finally {
    if (mounted) setState(() => _googleLoading = false);
  }
}
}
```

File: /lib/screens/read_me.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:pacman_application/utils/constants.dart';
import 'package:pacman_application/utils/appbar.dart';
import 'package:pacman_application/utils/sidebar.dart';
import 'package:webview_flutter/webview_flutter.dart';

/// A screen that will contain game instructions, credits, or
other
/// supplementary information for the player.
///
/// Currently displays a placeholder; replace the body content
with the
/// actual readme / help text before shipping.
class ReadMe extends StatefulWidget {
  const ReadMe({super.key});

  @override
  State<ReadMe> createState() => ReadMeState();
}

class ReadMeState extends State<ReadMe> {

  /// controller to view the project wiki from the web
  final WebViewController _controller = WebViewController()
    ..setJavaScriptMode(JavaScriptMode.unrestricted)

  ..loadRequest(Uri.parse("https://docs.google.com/document/d/1h2Z
_hUe-yG7tg1wrDSrGjNImLIGe7AQLBSUyIven2Po/edit?usp=sharing"));
```

// — Lifecycle

```
@override
void dispose() {
  SystemChrome.setPreferredOrientations([
    DeviceOrientation.portraitDown,
    DeviceOrientation.portraitUp,
  ]);
  super.dispose();
}
```

// — Build

```
@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(
    context: context,
    header: Text('READ ME', style: headerTextStyle),
  ),
  drawer: const Sidebar(),
  body: SafeArea(
    child: Center(
      child: WebViewWidget(controller: _controller),
    ),
  ),
);
}
```

File: /lib/screens/settings.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_colorpicker/flutter_colorpicker.dart';
import 'package:pacman_application/utils/constants.dart';
import 'package:pacman_application/utils/appbar.dart';
import 'package:pacman_application/utils/sidebar.dart';
import 'package:shared_preferences/shared_preferences.dart';

/// Allows the player to customise the colours of Pac-Man, each
/// ghost, and
/// the maze walls.
///
/// Each element has its own colour picker (a [HueRingPicker]
/// shown in an
/// [AlertDialog]) and a reset button that restores the game's
/// original colour.
/// A global "Reset Settings" button at the bottom reverts every
/// element at
/// once. Colour choices are persisted via [SharedPreferences]
/// so they survive
/// app restarts.
class Settings extends StatefulWidget {
  const Settings({super.key});

  @override
  State<Settings> createState() => SettingsState();
}

class SettingsState extends State<Settings> {
```

// — Fields

```
/// Currently selected colour for Pac-Man.  
Color _pacmanColor = pacmanColor;  
  
/// Currently selected colour for Blinky (red ghost).  
Color _blinkyColor = blinkyColor;  
  
/// Currently selected colour for Pinky (pink ghost).  
Color _pinkyColor = pinkyColor;  
  
/// Currently selected colour for Inky (cyan ghost).  
Color _inkyColor = inkyColor;  
  
/// Currently selected colour for Clyde (orange ghost).  
Color _clydeColor = clydeColor;  
  
/// Currently selected colour for the maze walls.  
Color _mapColor = mapColor;
```

// — Lifecycle

```
@override  
void initState() {  
    super.initState();  
    _loadColors();  
}  
  
@override  
void dispose() {  
    SystemChrome.setPreferredOrientations([  
        DeviceOrientation.portraitDown,
```

```
        DeviceOrientation.portraitUp,  
    ]);  
    super.dispose();  
}
```

```
// — Build
```

```
@override  
Widget build(BuildContext context) => Scaffold(  
  appBar: AppBar(  
    context: context,  
    header: Text('SETTINGS', style: headerTextStyle),  
  ),  
  drawer: const Sidebar(),  
  body: SafeArea(  
    child: Container(  
      padding: const EdgeInsets.symmetric(vertical: 20,  
horizontal: 50),  
      child: Center(  
        child: Column(  
          spacing: 40,  
          mainAxisAlignment: MainAxisAlignment.center,  
          children: [  
            const Row(  
              mainAxisAlignment: MainAxisAlignment.center,  
              children: [  
                Text(  
                  'Here you can change the colors\nof the game  
elements',  
                  style: TextStyle(fontSize: 16, fontWeight:  
FontWeight.bold),  
                  textAlign: TextAlign.center,  
                ),  
              ],  
            ),  
          ],  
        ),  
      ),  
    ),  
  ),  
);
```

```
    ],  
  ),  
  
  const Spacer(),  
  
  _colorPickerRow('pacman'),  
  _colorPickerRow('blinky'),  
  _colorPickerRow('pinky'),  
  _colorPickerRow('inky'),  
  _colorPickerRow('clyde'),  
  _colorPickerRow('map'),  
  
  const Spacer(),  
  
  Row(  
    mainAxisAlignment: MainAxisAlignment.center,  
    children: [  
      ElevatedButton(  
        onPressed: () {  
          setState(() {  
            _resetPacmanColor();  
            _resetBlinkyColor();  
            _resetPinkyColor();  
            _resetInkyColor();  
            _resetClydeColor();  
            _resetMapColor();  
          });  
        },  
        child: const Text(  
          'Reset Settings',  
          style: TextStyle(  
            fontFamily: 'PressStart',  
            fontSize: 17,  
            color: Colors.black,
```

```
        ),  
    ),  
    ),  
    ],  
    ),  
    ],  
    ),  
    ),  
    ),  
    ),  
    ),  
);
```

```
// — Private helpers
```

```
/// Loads all persisted colour preferences from  
[SharedPreferences] and  
/// updates the corresponding state fields, triggering a  
rebuild.
```

```
void _loadColors() async {  
    final prefs = await SharedPreferences.getInstance();
```

```
    Color? loaded;
```

```
    loaded = _readColorPrefs(prefs, 'pacman');  
    if (loaded != null) _pacmanColor = loaded;
```

```
    loaded = _readColorPrefs(prefs, 'blinky');  
    if (loaded != null) _blinkyColor = loaded;
```

```
    loaded = _readColorPrefs(prefs, 'pinky');  
    if (loaded != null) _pinkyColor = loaded;
```

```
    loaded = _readColorPrefs(prefs, 'inky');
```

```
if (loaded != null) _inkyColor = loaded;

loaded = _readColorPrefs(prefs, 'clyde');
if (loaded != null) _clydeColor = loaded;

loaded = _readColorPrefs(prefs, 'map');
if (loaded != null) _mapColor = loaded;

setState(() {});
}

/// Reads the ARGB components for [character] from [prefs] and
returns the
/// reconstructed [Color], or `null` if any component is
missing.
Color? _readColorPrefs(SharedPreferences prefs, String
character) {
    final a = prefs.getInt('${character}Color/a');
    final r = prefs.getInt('${character}Color/r');
    final g = prefs.getInt('${character}Color/g');
    final b = prefs.getInt('${character}Color/b');
    if (a == null || r == null || g == null || b == null) return
null;
    return Color.fromARGB(a, r, g, b);
}

/// Converts a double colour channel value in [0, 1] to an
8-bit integer
/// in [0, 255].
int _floatToInt8(double x) => (x * 255.0).round().clamp(0,
255);

/// Returns the currently selected [Color] for [character].
Color _getCharacterColor(String character) {
```



```
return switch (character) {
    'pacman' => _pacmanColor,
    'blinky' => _blinkyColor,
    'pinky'  => _pinkyColor,
    'inky'   => _inkyColor,
    'clyde'  => _clydeColor,
    'map'    => _mapColor,
    _        => Colors.transparent,
};
}

/// Persists the current colour for [character] to
[SharedPreferences].
void _saveColor(String character) async {
    final prefs = await SharedPreferences.getInstance();
    final color = _getCharacterColor(character);

    await prefs.setInt('${character}Color/a',
        _floatToInt8(color.a));
    await prefs.setInt('${character}Color/r',
        _floatToInt8(color.r));
    await prefs.setInt('${character}Color/g',
        _floatToInt8(color.g));
    await prefs.setInt('${character}Color/b',
        _floatToInt8(color.b));
}

/// Opens a colour-picker dialog for [character]. The selected
colour is
/// applied in real time and saved when the user confirms.
void _showColorDialog(BuildContext context, String character)
{
    showDialog(
        context: context,
```

```
builder: (_) => AlertDialog(  
  title: Center(  
    child: Text(  
      'Change  
${character[0].toUpperCase()}${character.substring(1)} color',  
      style: const TextStyle(fontSize: 16, fontWeight:  
FontWeight.bold),  
      textAlign: TextAlign.center,  
    ),  
  ),  
  content: SizedBox(  
    height: (MediaQuery.of(context).size.height /  
3).clamp(370, 600),  
    child: HueRingPicker(  
      pickerColor: _getCharacterColor(character),  
      onColorChanged: (Color color) {  
        setState(() {  
          switch (character) {  
            case 'pacman': _pacmanColor = color;  
            case 'blinky': _blinkyColor = color;  
            case 'pinky': _pinkyColor = color;  
            case 'inky': _inkyColor = color;  
            case 'clyde': _clydeColor = color;  
            case 'map': _mapColor = color;  
          }  
        });  
      },  
    ),  
  ),  
  actions: [  
    ElevatedButton(  
      onPressed: () {  
        _saveColor(character);  
        Navigator.of(context).pop();  
      },  
    ),  
  ],  
);
```

```
    },  
    child: const Text('Got it'),  
  ),  
],  
),  
);  
}
```

```
// — Reset helpers
```

```
void _resetPacmanColor() { _pacmanColor = pacmanColor;  
_saveColor('pacman'); }  
void _resetBlinkyColor() { _blinkyColor = blinkyColor;  
_saveColor('blinky'); }  
void _resetPinkyColor() { _pinkyColor = pinkyColor;  
_saveColor('pinky'); }  
void _resetInkyColor() { _inkyColor = inkyColor;  
_saveColor('inky'); }  
void _resetClydeColor() { _clydeColor = clydeColor;  
_saveColor('clyde'); }  
void _resetMapColor() { _mapColor = mapColor;  
_saveColor('map'); }
```

```
/// Resets the colour of [character] to its default value and  
saves it.
```

```
void _resetColor(String character) {  
  setState(() {  
    switch (character) {  
      case 'pacman': _resetPacmanColor();  
      case 'blinky': _resetBlinkyColor();  
      case 'pinky': _resetPinkyColor();  
      case 'inky': _resetInkyColor();  
      case 'clyde': _resetClydeColor();
```

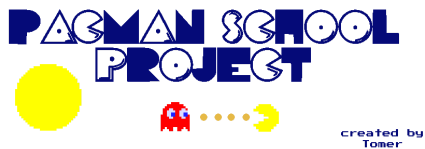
```
        case 'map':      _resetMapColor();
    }
    });
}
```

```
// — Widget builders
```

```
/// Builds a row containing a colour-picker button (labelled
with
/// [character]'s name in its current colour) and a reset icon
button.
```

```
Widget _colorPickerRow(String character) {
    return Row(
        crossAxisAlignment: CrossAxisAlignment.center,
        mainAxisAlignment: MainAxisAlignment.center,
        spacing: 20,
        children: [
            ElevatedButton(
                onPressed: () => _showColorDialog(context, character),
                child: Text(
                    '${character[0].toUpperCase()}${character.substring(1)} Color',
                    style: TextStyle(
                        fontFamily: 'PressStart',
                        color: _getCharacterColor(character),
                    ),
                ),
            ),
            IconButton(
                onPressed: () => _resetColor(character),
                icon: const Icon(Icons.restore_rounded),
            ),
        ],
    );
}
```

Pacman Android Edition



תומר כץ

```
};  
}  
}
```

File: /lib/screens/signup.dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_database/firebase_database.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:google_sign_in/google_sign_in.dart';
import 'package:pacman_application/database/game_user.dart';
import 'package:pacman_application/database/session.dart';
import 'package:pacman_application/screens/login.dart';
import 'package:pacman_application/utils/appbar.dart';
import 'package:pacman_application/utils/bonus_type.dart';
import 'package:pacman_application/utils/constants.dart';
import 'package:shared_preferences/shared_preferences.dart';

/// Signup screen that allows users to create an account using:
///
/// - Email and password.
/// - Google authentication.
///
/// The screen also stores user information locally using
/// [SharedPreferences].
class Signup extends StatefulWidget {
  /// Creates the signup screen.
  const Signup({super.key});

  @override
  State<Signup> createState() => _SignupState();
}

/// State class for the [Signup] screen.
```

```
class _SignupState extends State<Signup> {  
    /// Indicates whether the user accepted the Terms of Service.  
    bool _acceptedTerms = false;  
  
    /// Indicates whether the Google sign-in process is running.  
    bool _isGoogleLoading = false;  
  
    /// Message displayed to the user.  
    String _message = '';  
  
    /// Controller for the username field.  
    final TextEditingController _nameController =  
TextEditingController();  
  
    /// Controller for the email field.  
    final TextEditingController _emailController =  
TextEditingController();  
  
    /// Controller for the password field.  
    final TextEditingController _passwordController =  
        TextEditingController();  
  
    @override  
    void dispose() {  
        _nameController.dispose();  
        _emailController.dispose();  
        _passwordController.dispose();  
  
        SystemChrome.setPreferredOrientations([  
            DeviceOrientation.portraitDown,  
            DeviceOrientation.portraitUp,  
        ]);  
  
        super.dispose();  
    }  
}
```

```
}

/// Returns whether all required fields are filled.
bool get _canContinue {
    return _nameController.text.isNotEmpty &&
        _emailController.text.isNotEmpty &&
        _passwordController.text.isNotEmpty &&
        _acceptedTerms;
}

/// Saves the currently logged-in user locally.
Future<void> _saveUserPreferences(GameUser user) async {
    final SharedPreferences preferences =
        await SharedPreferences.getInstance();

    await preferences.setString('lastUser/uid', user.uid);
    await preferences.setString('lastUser/name', user.name);
    await preferences.setInt('lastUser/highScore',
user.highScore);
    await preferences.setString('lastUser/email', user.email);

    for (final MapEntry<BonusType, int> bonusEntry
        in user.bonusTable.entries) {
        await preferences.setInt(
            'lastUser/bonusTable/${bonusEntry.key.name}',
            bonusEntry.value,
        );
    }
}

/// Returns whether the Firebase user is new.
Future<bool> _isNewUser(String userId) async {
    final DatabaseReference userReference =
        FirebaseDatabase.instance.ref('users/$userId');
```



```
    return !(await userReference.get()).exists;
}

/// Creates the default bonus table.
Map<String, int> _createDefaultBonusTable() {
  return {
    'apple': 0,
    'bell': 0,
    'cherry': 0,
    'galaxian': 0,
    'key': 0,
    'melon': 0,
    'orange': 0,
    'strawberry': 0,
  };
}

/// Creates a database record for a new user.
Future<void> _createDatabaseUser({
  required String userId,
  required String name,
  required String email,
}) async {
  final DatabaseReference userReference =
    FirebaseDatabase.instance.ref('users/$userId');

  await userReference.set({
    'name': name,
    'email': email,
    'highScore': anonymousHighScore,
    'bonusTable': _createDefaultBonusTable(),
  });
}
```

```
/// Updates the global session user.
Future<void> _setCurrentUser({
  required String userId,
  required String name,
  required String email,
}) async {
  isAnonymous = false;

  currentUser = GameUser(
    uid: userId,
    name: name,
    highScore: anonymousHighScore,
    email: email,
    bonusTable: anonymousBonusTable,
  );

  await _saveUserPreferences(currentUser);
}

/// Navigates to the login screen.
void _goToLoginScreen() {
  Navigator.pushReplacement(
    context,
    MaterialPageRoute(builder: (context) => Login()),
  );
}

/// Signs up the user using Google authentication.
Future<void> _signUpWithGoogle() async {
  if (!_acceptedTerms) {
    setState(() {
      _message = 'Please accept the TOS before continuing.';
    });
  }
}
```

```
        return;
    }

    setState(() {
        _isGoogleLoading = true;
        _message = '';
    });

    try {
        final GoogleSignInAccount? googleAccount =
            await GoogleSignIn().signIn();

        if (googleAccount == null) {
            setState(() {
                _isGoogleLoading = false;
            });

            return;
        }

        final GoogleSignInAuthentication googleAuthentication =
            await googleAccount.authentication;

        final AuthCredential credential =
        GoogleAuthProvider.credential(
            accessToken: googleAuthentication.accessToken,
            idToken: googleAuthentication.idToken,
        );

        final UserCredential userCredential = await
        FirebaseAuth.instance
            .signInWithCredential(credential);
    }
```

```
final User firebaseUser = userCredential.user!;

final String userName =
    firebaseUser.displayName ?? googleAccount.displayName
?? '';

final String userEmail =
    firebaseUser.email ?? googleAccount.email;

if (await _isNewUser(firebaseUser.uid)) {
    await _createDatabaseUser(
        userId: firebaseUser.uid,
        name: userName,
        email: userEmail,
    );
}

await _setCurrentUser(
    userId: firebaseUser.uid,
    name: userName,
    email: userEmail,
);

if (mounted) {
    _goToLoginScreen();
}
} on FirebaseAuthException catch (exception) {
    if (kDebugMode) {
        print('Google sign-up Firebase error:
${exception.code}');
    }

    setState(() {
        _message = exception.code;
    });
}
```

```
    });  
    } catch (exception) {  
        if (kDebugMode) {  
            print('Google sign-up error: $exception');  
        }  
  
        setState(() {  
            _message = 'something-went-wrong';  
        });  
    } finally {  
        if (mounted) {  
            setState(() {  
                _isGoogleLoading = false;  
            });  
        }  
    }  
}  
  
/// Signs up the user using email and password.  
Future<void> _signUpWithEmail() async {  
    if (!_acceptedTerms) {  
        return;  
    }  
  
    try {  
        final UserCredential userCredential = await  
        FirebaseAuth.instance  
            .createUserWithEmailAndPassword(  
                email: _emailController.text,  
                password: _passwordController.text,  
            );  
  
        await userCredential.user!.updateDisplayName(  
            _nameController.text,  
        );  
    }  
}
```

```
);

await _createDatabaseUser(
  userId: userCredential.user!.uid,
  name: _nameController.text,
  email: _emailController.text,
);

await _setCurrentUser(
  userId: userCredential.user!.uid,
  name: _nameController.text,
  email: _emailController.text,
);

if (mounted) {
  _goToLoginScreen();
}
} on FirebaseAuthException catch (exception) {
  if (kDebugMode) {
    print(exception.code);
  }

  setState(() {
    _message = exception.code;
  });
} on FirebaseException catch (exception) {
  if (kDebugMode) {
    print(exception.code);
  }

  setState(() {
    _message = exception.code;
  });
} catch (exception) {
```

```
        if (kDebugMode) {
            print(exception);
        }
    }
}

/// Builds a text input section.
Widget _buildInputField({
    required String label,
    required TextEditingController controller,
    bool obscureText = false,
    TextInputType keyboardType = TextInputType.text,
}) {
    return Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
            Text(
                label,
                style: TextStyle(color:
Theme.of(context).primaryColor),
            ),
            TextField(
                controller: controller,
                obscureText: obscureText,
                keyboardType: keyboardType,
                onSubmitted: (_) => setState(() {}),
            ),
        ],
    );
}

/// Builds the Google sign-up button.
Widget _buildGoogleButton() {
    return OutlinedButton(
```

```
onPressed: _isGoogleLoading ? null : _signUpWithGoogle,
style: OutlinedButton.styleFrom(
  minimumSize: const Size(1000, 40),
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(8),
  ),
  side: BorderSide(color: Colors.grey.shade300),
  backgroundColor: Colors.white,
),
child: _isGoogleLoading
  ? SizedBox(
    height: 18,
    width: 18,
    child: CircularProgressIndicator(
      strokeWidth: 2,
      color: Theme.of(context).primaryColor,
    ),
  )
  : Row(
    mainAxisAlignment: MainAxisAlignment.center,
    mainAxisSize: MainAxisSize.min,
    children: [
      Image.asset('assets/GoogleG.png'),
      const SizedBox(width: 10),
      const Text(
        'Continue with Google',
        style: TextStyle(
          color: Colors.black87,
          fontSize: 14,
        ),
      ),
    ],
  ),
);
```



```
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      context: context,
      header: Text('SIGN UP', style: headerTextStyle),
      canLogout: false,
    ),
    body: SafeArea(
      child: Container(
        padding: const EdgeInsets.symmetric(
          vertical: 20,
          horizontal: 50,
        ),
      ),
      child: SingleChildScrollView(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 100),
            Text(
              'Enter these fields to make an account on the
app',

              style: TextStyle(
                fontSize: 12,
                color: Colors.grey[800],
              ),
            ),
            const SizedBox(height: 20),
            _buildInputField(
              label: 'Name',
              controller: _nameController,
            ),
          ],
        ),
      ),
    ),
  );
}
```

```
const SizedBox(height: 10),
_buildInputField(
  label: 'Email',
  controller: _emailController,
  keyboardType: TextInputType.emailAddress,
),
const SizedBox(height: 10),
_buildInputField(
  label: 'Password',
  controller: _passwordController,
  obscureText: true,
),
const SizedBox(height: 15),
Row(
  crossAxisAlignment: CrossAxisAlignment.center,
  children: [
    Checkbox(
      value: _acceptedTerms,
      onChanged: (bool? value) {
        setState(() {
          _acceptedTerms = value ?? false;
        });
      },
    ),
    const Expanded(
      child: Text(
        'I agree to the TOS and Privacy Policy',
      ),
    ),
  ],
),
ElevatedButton(
  onPressed: _canContinue ? _signUpWithEmail :
null,
```

```
style: ElevatedButton.styleFrom(  
    minimumSize: const Size(1000, 40),  
    shape: RoundedRectangleBorder(  
        borderRadius: BorderRadius.circular(8),  
    ),  
),  
child: const Text(  
    'Continue',  
    style: TextStyle(color: Colors.white),  
),  
),  
const SizedBox(height: 20),  
Row(  
    children: [  
        Expanded(  
            child: Divider(color:  
Colors.grey.shade300),  
        ),  
        const Padding(  
            padding: EdgeInsets.symmetric(horizontal:  
12),  
            child: Text(  
                'or',  
                style: TextStyle(  
                    fontSize: 12,  
                    color: Colors.grey,  
                ),  
            ),  
        ),  
        Expanded(  
            child: Divider(color:  
Colors.grey.shade300),  
        ),  
    ],  
),
```

```
),
const SizedBox(height: 20),
_buildGoogleButton(),
const SizedBox(height: 20),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    const Text(
      'Have an Account? ',
      style: TextStyle(color: Colors.grey),
    ),
    GestureDetector(
      onTap: _goToLoginScreen,
      child: Text(
        'Sign In',
        style: TextStyle(
          color: Theme.of(context).primaryColor,
        ),
      ),
    ),
  ],
),
const SizedBox(height: 15),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Expanded(
      child: Text(
        _message.toUpperCase(),
        textAlign: TextAlign.center,
        style: const TextStyle(
          color: Colors.black,
          fontFamily: 'PressStart',
          fontSize: 20,
```

created by
Tomer

created by
Tomer

257

File: /lib/game/bonus.dart

```
import 'dart:math';

import 'package:flutter/material.dart';
import 'package:pacman_application/utils/bonus_type.dart';
import 'package:pacman_application/game/game_manager.dart';
import 'package:pacman_application/game/pacman.dart';
import 'package:pacman_application/utils/util.dart';

/// The bonus item that periodically appears in the centre of
the maze.
///
/// [GameManager] calls [setVisible] via a [GameTimer] to show a
randomly
/// chosen [BonusType]. Each [update] tick checks whether
Pac-Man has
/// collected it. The item is hidden again after collection.
class Bonus {
  /// Creates a [Bonus] positioned at the map's bonus tile and
hidden by default.
  Bonus({required this.gameManager, this.onEaten}) {
    x = gameManager.gameMap.bonusX;
    y = gameManager.gameMap.bonusY;
  }

  // — Dependencies

  /// Reference to the central [GameManager].
  final GameManager gameManager;
```

```
// — Position
```

```
/// Horizontal tile position (set from [GameMap.bonusX] in  
constructor).
```

```
late double x;
```

```
/// Vertical tile position (set from [GameMap.bonusY] in  
constructor).
```

```
late double y;
```

```
// — State
```

—

```
/// Whether the bonus item is currently shown on screen.  
bool isVisible = false;
```

```
/// The [BonusType] currently displayed (randomised each time  
[setVisible] is called).
```

```
BonusType currentType = BonusType.cherry;
```

```
// — Callbacks
```

```
/// Called when Pac-Man collects the bonus item.
```

```
final void Function(BonusType bonusType)? onEaten;
```

```
// — Asset path constants
```

```
/// Directory under which bonus-item sprites are stored.
```

```
static const String bonusAssetsPath = "assets/game/bonuses/";
```

```
/// File extension shared by all bonus sprites.
static const String bonusesAssetsEndFile = ".png";

// — Public API



---

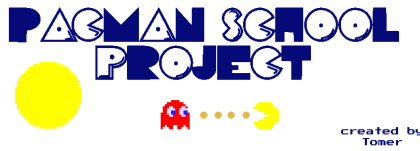


/// Shows the bonus item with a freshly randomised
[BonusType].
///
/// If the item is already visible, only its type is
re-randomised.
void setVisible() {
    currentType =
BonusType.values[_random.nextInt(BonusType.values.length)];
    isVisible = true;
}

/// Advances bonus logic by [dt] seconds.
///
/// Calls [onEaten] and hides the item when Pac-Man is within
0.5 tiles.
void update(double dt) {
    if (isVisible && dist(_getPacman().x - x, _getPacman().y -
y) <= 0.5) {
        onEaten?.call(currentType);
        isVisible = false;
    }
}

/// Returns a [Positioned] widget showing the current bonus
sprite, or an
/// empty [Container] when the item is hidden.
Widget getSprite(double tileSize) => Positioned(
    left: x * tileSize,
```


Pacman Android Edition



תומר כץ

```
        top: y * tileSize,
        child: isVisible
            ? Image.asset(

"$bonusAssetsPath${currentType.name}$bonusesAssetsEndFile",
        )
        : Container(),
    );

// — Private helpers

final Random _random = Random();

Pacman _getPacman() => gameManager.pacman;
}
```

File: /lib/game/direction.dart

```
import 'dart:math';

/// Represents the four cardinal movement directions used
throughout the game.
enum Direction {
  up,
  down,
  left,
  right;

  /// Returns the direction that is directly opposite to this
  one.
  Direction opposite() {
    switch (this) {
      case Direction.up:
        return Direction.down;
      case Direction.down:
        return Direction.up;
      case Direction.left:
        return Direction.right;
      case Direction.right:
        return Direction.left;
    }
  }
}

/// Returns the (dx, dy) grid delta for one step in this
direction.
(int, int) toDelta() {
  switch (this) {
    case Direction.up:
```

```
        return (0, -1);
    case Direction.down:
        return (0, 1);
    case Direction.left:
        return (-1, 0);
    case Direction.right:
        return (1, 0);
    }
}

/// Returns the rotation angle in radians for rendering a
sprite facing this direction.
///
/// Assumes the base sprite points upward (0 radians).
double toAngle() => switch (this) {
    Direction.up => 0,
    Direction.down => pi,
    Direction.left => -pi / 2,
    Direction.right => pi / 2,
};
}
```

File: /lib/game/dot.dart

```
import 'package:flutter/material.dart';

/// A small collectible dot placed on the game map.
///
/// Dots are created by [GameMap] during initialisation. Each
dot knows its
/// grid position and fires [onEaten] the first time Pac-Man
passes over it.
/// Call [reset] to restore the dot for a new round.
class Dot extends StatelessWidget {
  /// Creates a [Dot] at grid position ([x], [y]).
  ///
  /// [onEaten] is called exactly once per life/round when the
dot is collected.
  Dot(this.x, this.y, {super.key, required this.onEaten}) :
_isEaten = false;

  // — Identity / position

  /// Horizontal grid column of this dot.
  final int x;

  /// Vertical grid row of this dot.
  final int y;

  // — State
```

```
/// Whether this dot has already been collected in the current round.
```

```
bool _isEaten;
```

```
// — Callbacks
```

```
/// Invoked once when Pac-Man eats this dot.
```

```
final void Function() onEaten;
```

```
// — Public API
```

```
/// Checks whether Pac-Man (at [pacmanX], [pacmanY]) is close enough to eat
```

```
/// this dot. Tolerance is  $\pm 0.5$  tiles on both axes.
```

```
void checkIfEaten(double pacmanX, double pacmanY) {
```

```
    if (!_isEaten && (pacmanX - x).abs() < 0.5 && (pacmanY - y).abs() < 0.5) {
```

```
        _eat();
```

```
    }
```

```
}
```

```
/// Restores the dot so it is visible and collectable again.
```

```
void reset() {
```

```
    _isEaten = false;
```

```
}
```

```
/// Returns a [Positioned] widget that renders the dot inside a [Stack].
```

```
///
```

```
/// The dot is centred inside its tile using a 0.5-tile offset.
```

```
/// Renders an empty [Container] when already eaten.
```

```
Widget showOnStack(double tileSize) {  
    return Positioned(  
        left: (x + 0.5) * tileSize,  
        top: (y + 0.5) * tileSize,  
        child: _isEaten ? Container() : this,  
    );  
}
```

```
// — Private helpers
```

```
/// Marks the dot as eaten and fires [onEaten].  
void _eat() {  
    onEaten();  
    _isEaten = true;  
}
```

```
// — Widget
```

```
@override  
Widget build(BuildContext context) => Container(  
    constraints: BoxConstraints.tight(const Size.square(5)),  
    decoration: BoxDecoration(  
        color: Colors.amber[200],  
        shape: BoxShape.circle,  
    ),  
);  
}
```

File: /lib/game/game_manager.dart

```
import 'dart:async';
import 'package:audioplayers/audioplayers.dart';
import 'package:flutter/material.dart';
import 'package:pacman_application/utils/constants.dart';
import 'package:pacman_application/game/bonus.dart';
import 'package:pacman_application/utils/bonus_type.dart';
import
'package:pacman_application/game/controller/controller.dart';
import
'package:pacman_application/game/controller/end_game_buttons.dar
t';
import 'package:pacman_application/game/map/game_map.dart';
import 'package:pacman_application/game/game_screen.dart';
import 'package:pacman_application/game/game_timer.dart';
import 'package:pacman_application/game/ghosts/blinky.dart';
import 'package:pacman_application/game/ghosts/clyde.dart';
import 'package:pacman_application/game/ghosts/ghost.dart';
import 'package:pacman_application/game/ghosts/inky.dart';
import 'package:pacman_application/game/ghosts/pinky.dart';
import 'package:pacman_application/game/pacman.dart';
```

```
// — Audio state
```

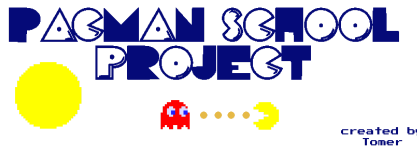
```
/// Describes which audio loop the game should currently be
playing.
```

```
enum _AudioState { idle, normal, blue, eye }
```

```
/// Central controller for the Pac-Man game.
```

```
///
```

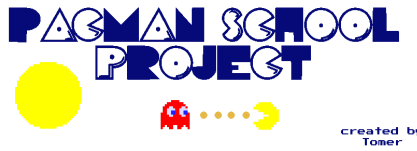
Pacman Android Edition



תומר כץ

```
/// [GameManager] owns all game objects ([gameMap], [pacman],  
[ghosts],  
/// [bonus]) and drives the main game loop via a  
[Timer.periodic]. It handles:  
/// - Score / lives / high score tracking.  
/// - Ghost state transitions (chase ↔ scatter toggle every 5  
s).  
/// - Frightened-mode timers after a super-dot is eaten.  
/// - Bonus item scheduling.  
/// - Death / game-over flow.  
/// - Audio state machine.  
///  
/// Pass `start: true` to [GameManager] to begin the game  
immediately, or call  
/// [initGame] manually once the widget tree is ready.  
class GameManager {  
    /// Creates a [GameManager], optionally starting the game loop  
    immediately.  
    ///  
    /// All callback parameters are optional; pass them to wire up  
    your UI.  
    GameManager({  
        bool start = false,  
        int Function()? highScore,  
        void Function(int score)? onGameOver,  
        void Function()? onRetry,  
        void Function()? onExit,  
        void Function(BonusType bonus)? onGettingBonus,  
    }) {  
        this.onGameOver      = onGameOver      ?? (_) {};  
        this.onRetry         = onRetry         ?? () {};  
        this.onExit          = onExit          ?? () {};  
        this.onGettingBonus  = onGettingBonus  ?? (_) {};
```


Pacman Android Edition



תומר כץ

```
if (highScore != null) {
    this.highScore = highScore();
    _getHighScoreCallback = highScore;
}

if (start) initGame();
}

// — Scoring & progression



---



/// Current score for this session.
int score = 0;

/// All-time high score; updated live if [score] exceeds it.
int highScore = 0;

/// Index of the current maze screen (increments when all dots
are cleared).
int screenNum = 0;

/// Points awarded for the next ghost eaten in the current
frightened chain.
///
/// Starts at 10 and doubles with each consecutive ghost
eaten.
int nextGhostEatenScore = 10;

/// Remaining lives. Game over when this reaches 0.
int lives = 3;

// — Flags
```

```
/// Whether the game is currently paused (death animation,  
level transition).  
bool isPaused = false;  
  
/// Whether the game has ended.  
bool isGameOver = false;  
  
/// Prevents triggering the next-screen transition more than  
once per clear.  
bool _hasTriggeredNextScreen = false;  
  
// — Game objects
```

```
/// The game map (tiles, dots, super-dots, layout constants).  
late final GameMap gameMap = GameMap(  
    map: defaultMap,  
    onDotEaten: _onDotEaten,  
    onSuperDotEaten: _onSuperDotEaten,  
);  
  
/// The player-controlled Pac-Man character.  
late final Pacman pacman = Pacman(gameManager: this);  
  
/// All four ghosts in order: Blinky, Pinky, Inky, Clyde.  
late final List<Ghost> ghosts = [  
    Blinky(gameManager: this, onDeath: _onDeath, onEaten:  
_onGhostEaten),  
    Pinky (gameManager: this, onDeath: _onDeath, onEaten:  
_onGhostEaten),  
    Inky  (gameManager: this, onDeath: _onDeath, onEaten:  
_onGhostEaten),
```

```
Clyde (gameManager: this, onDeath: _onDeath, onEaten:
_onGhostEaten),
];

/// The bonus item that periodically appears in the centre of
the maze.
late final Bonus bonus = Bonus(gameManager: this, onEaten:
_onBonusEaten);

/// Bonuses collected this session (shown as icons on screen).
late final List<BonusType> bonusesTaken = [];

// — Timers

/// Game-loop delta-time in seconds.
final double dt = 0.001;

/// All active [GameTimer] instances updated each tick.
late final List<GameTimer> _gameTimers = [_bonusTimer];

/// Periodically makes the bonus item visible (every 10 s,
looping).
late final GameTimer _bonusTimer = GameTimer(
    10,
    onEnd: () => bonus.setVisible(),
    isLoop: true,
);

/// Main [Timer.periodic] that drives the game loop.
late Timer _periodicTimer;

/// High-precision stopwatch used to calculate real-elapsed
delta-time.
```

```
late final Stopwatch _stopwatch = Stopwatch();

/// Timestamp of the last tick in seconds (used to compute
real dt).
double _lastTickTime = 0;

// — UI components



---



/// The on-screen D-pad; wired to update
[pacman.nextDirection].
late final Controller controller = Controller(
    onDirectionChanged: (direction) => pacman.nextDirection =
direction,
);

/// Retry / Exit buttons shown after game over.
late final EndGameButtons endGameButtons = EndGameButtons(
    size: 120,
    onRetry: onRetry,
    onExit: onExit,
);

/// The main game screen widget.
late final GameScreen currentScreen = GameScreen(
    gameMessage: (x) => _gameMessage(x),
    gameMap: gameMap,
    controller: controller,
    pacman: pacman,
    ghosts: ghosts,
    getScore: () => score,
    getLives: () => lives,
    dt: dt,
    bonus: bonus,
```

```
bonusesTaken: bonusesTaken,  
getHighScore: () => highScore,  
isGameOver: () => isGameOver,  
endGameButtons: endGameButtons,  
);
```

```
// — In-game message overlay
```

```
/// Builder for the overlay text (e.g. "READY!", "GAME OVER",  
empty widget).
```

```
Widget Function(double tileSize) _gameMessage = (double  
tileSize) =>
```

```
  Positioned(  
    top: 16.2 * tileSize,  
    left: 10.5 * tileSize,  
    child: Center(  
      child: Text(  
        "READY!",  
        style: TextStyle(  
          color: pacmanColor,  
          fontSize: 20,  
          fontWeight: FontWeight.bold,  
          fontFamily: "PressStart",  
        ),  
      ),  
    ),  
  ),  
);
```

```
// — External callbacks
```

```
/// Called with the final score when the game ends.  
late final void Function(int score) onGameOver;
```

```
/// Called when the player chooses to retry.
late final void Function() onRetry;

/// Called when the player chooses to exit.
late final void Function() onExit;

/// Called each time Pac-Man collects a bonus item.
late final void Function(BonusType) onGettingBonus;

/// Optional callback that returns the persisted high score
from the UI layer.
int Function()? _getHighScoreCallback;

// — Audio

—

final AudioPlayer _audioPlayer = AudioPlayer();

_AudioState _currentAudioState = _AudioState.idle;

// — Public API

/// Starts the game: plays the start jingle, then begins the
game loop after
/// a 5-second countdown (matching the "READY!" display
duration).
void initGame() {
    _audioPlayer.play(AssetSource("audio/Start_Music.mp3"));
    highScore = _getHighScoreCallback?.call() ?? highScore;

    Timer(const Duration(seconds: 5), () {
```

```
_gameMessage = (x) => Container();
_bonusTimer.start();
_stopwatch.start();

_periodicTimer = Timer.periodic(
    Duration(milliseconds: (dt * 1000).toInt()),
    (timer) {
        final realDt =
            (_stopwatch.elapsedMicroseconds / 1_000_000.0) -
            _lastTickTime;
        _lastTickTime = _stopwatch.elapsedMicroseconds /
            1_000_000.0;

        if (isPaused) {
            pacman.updateDeathAnimation(realDt);
        } else {
            for (final timer in _gameTimers) {
                timer.update(realDt);
            }
            _update(realDt);
        }
    },
);
});

// Toggle ghosts between chase and scatter every 5 seconds.
Timer.periodic(const Duration(seconds: 5), (_) {
    for (final ghost in ghosts) {
        if (ghost.state == GhostState.chase) {
            ghost.state = GhostState.scatter;
        } else if (ghost.state == GhostState.scatter) {
            ghost.state = GhostState.chase;
        }
    }
})
```

```
});  
}  
  
// — Private game-loop  
  
/// Called every tick while the game is not paused.  
void _update(double dt) {  
    pacman.update(dt);  
  
    for (final dot in gameMap.dots) {  
        dot.checkIfEaten(pacman.x, pacman.y);  
    }  
    for (final superDot in gameMap.superDots) {  
        superDot.checkIfEaten(pacman.x, pacman.y);  
    }  
  
    // Keep Inky's static Blinky-position reference up to date.  
    Inky.setBlinkyPosition(ghosts[0].x, ghosts[0].y);  
  
    bonus.update(dt);  
  
    if (score > highScore) highScore = score;  
  
    // Update ghosts and determine audio state.  
    final previousAudioState = _currentAudioState;  
    _currentAudioState = _AudioState.normal;  
  
    for (final ghost in ghosts) {  
        ghost.update(dt);  
  
        switch (ghost.state) {  
            case GhostState.frightened0 || GhostState.frightened1:  
                if (_currentAudioState == _AudioState.normal) {
```



```
        _currentAudioState = _AudioState.blue;
    }
    case GhostState.eaten:
        _currentAudioState = _AudioState.eye;
    default:
        break;
    }
}

// Switch audio track when state changes.
_audioPlayer.setReleaseMode(ReleaseMode.loop);
if (_currentAudioState != previousAudioState) {
    switch (_currentAudioState) {
        case _AudioState.idle:
            break;
        case _AudioState.normal:

_audioPlayer.play(AssetSource("audio/Pacman_Eating_Dots.mp3"));
        case _AudioState.blue:

_audioPlayer.play(AssetSource("audio/Ghost_Turn_To_Blue.mp3"));
        case _AudioState.eye:

_audioPlayer.play(AssetSource("audio/Ghost_Return_To_Home.mp3"))
;
    }
}

// Level clear - all dots collected.
if (!_hasTriggeredNextScreen && gameMap.remainingDots == 0)
{
    isPaused = true;
    _hasTriggeredNextScreen = true;
}
```

```
Timer(const Duration(microseconds: 750), () {
    screenNum++;
    for (final ghost in ghosts) {
        ghost.reset();
    }
    pacman.reset();
    gameMap.reset();

    Timer(const Duration(seconds: 2), () {
        isPaused = false;
        _hasTriggeredNextScreen = false;
    });
});
}
```

```
// — Collectible callbacks
```

```
void _onDotEaten() {
    if ((score % 100) >= 99) lives++;
    score += 1;
}
```

```
void _onSuperDotEaten() {
    if ((score % 100) >= 90) lives++;
    score += 10;

    for (final ghost in ghosts) {
        ghost.state = GhostState.frightened0;
    }
}
```

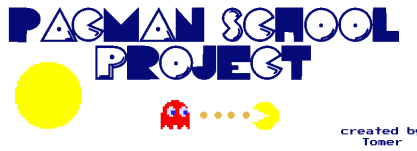
```
// After 8 s, switch frightened0 → frightened1 (warning
flash).
```

```
_gameTimers.add(
    GameTimer(
        8,
        onEnd: () {
            for (final ghost in ghosts) {
                if (ghost.state == GhostState.frightened0) {
                    ghost.state = GhostState.frightened1;
                }
            }
        },
        isOn: true,
    ),
);

// After 10 s, end frightened mode entirely.
_gameTimers.add(
    GameTimer(
        10,
        onEnd: () {
            for (final ghost in ghosts) {
                if (ghost.state == GhostState.frightened1) {
                    ghost.state = GhostState.chase;
                    nextGhostEatenScore = 10;
                }
            }
        },
        isOn: true,
    ),
);
}

void _onBonusEaten(BonusType type) {
    bonusesTaken.add(type);
    if (score % 100 >= 90) lives++;
}
```

Pacman Android Edition



תומר כץ

```
score += 10;  
onGettingBonus(type);  
}
```

```
// — Death / game-over flow
```

```
/// Handles Pac-Man's death: plays the death animation, resets  
characters,
```

```
/// decrements lives, and checks for game over.
```

```
void _onDeath() {  
    isPaused = true;
```

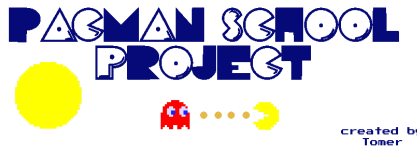
```
    _audioPlayer.setReleaseMode(ReleaseMode.release);  
    _audioPlayer.play(AssetSource("audio/Fail.mp3"));  
    _currentAudioState = _AudioState.idle;  
    pacman.playDeath();
```

```
    Timer(const Duration(seconds: 3), () {  
        isPaused = false;  
        for (final ghost in ghosts) {  
            ghost.reset();  
        }  
        pacman.reset();  
        lives--;
```

```
        if (lives <= 0) {  
            _triggerGameOver();  
        }  
    });  
}
```

```
/// Briefly pauses the game and awards score when Pac-Man eats  
a ghost.
```

Pacman Android Edition



תומר כץ

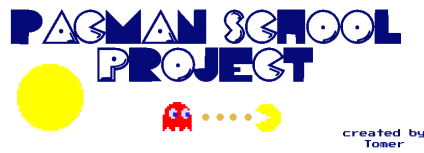
```
void _onGhostEaten() {
    isPaused = true;
    if (100 - (score % 100) <= nextGhostEatenScore) lives++;
    score += nextGhostEatenScore;
    nextGhostEatenScore *= 2;
    Timer(const Duration(milliseconds: 500), () => isPaused =
false);
}

/// Stops the game loop, shows "GAME OVER", and fires
[onGameOver].
void _triggerGameOver() {
    isGameOver = true;

    _gameMessage = (double tileSize) => Positioned(
        top: tileSize * 16.3,
        left: tileSize * 9.3,
        child: Center(
            child: const Text(
                "GAME OVER",
                style: TextStyle(
                    color: Colors.red,
                    fontSize: 16,
                    fontWeight: FontWeight.bold,
                    fontFamily: "PressStart",
                ),
            ),
        ),
    );

    for (final ghost in ghosts) {
        ghost.state = GhostState.idle;
    }
}
```

Pacman Android Edition



תומר כץ

```
_periodicTimer.cancel();  
_audioPlayer.stop();  
_audioPlayer.dispose();  
onGameOver(score);  
}  
}
```

File: /lib/game/game_screen.dart

```
import 'dart:async';
import 'dart:math';

import 'package:flutter/material.dart';
import 'package:pacman_application/game/bonus.dart';
import 'package:pacman_application/utils/bonus_type.dart';
import
'package:pacman_application/game/controller/controller.dart';
import
'package:pacman_application/game/controller/end_game_buttons.dar
t';
import 'package:pacman_application/game/map/game_map.dart';
import 'package:pacman_application/game/ghosts/ghost.dart';
import 'package:pacman_application/game/pacman.dart';
import 'package:shared_preferences/shared_preferences.dart';

/// The main in-game screen widget.
///
/// Renders the maze image, all dots, power pellets, ghosts,
Pac-Man, the
/// HUD (score / high score / lives), the bonus item, the
collected-bonuses
/// strip, and either the D-pad [Controller] or the
[EndGameButtons].
///
/// A [Timer] drives `setState` at the same rate as the game's
[dt] so the
/// widget rebuilds every tick.
class GameScreen extends StatefulWidget {
  const GameScreen({
```

```
super.key,  
required this.gameMessage,  
required this.controller,  
required this.gameMap,  
required this.pacman,  
required this.getScore,  
required this.getLives,  
required this.getHighScore,  
required this.ghosts,  
required this.bonus,  
required this.dt,  
required this.bonusesTaken,  
required this.endGameButtons,  
required this.isGameOver,  
});
```

```
// — Game objects
```

```
/// The shared game map (tiles, dots, super-dots).  
final GameMap gameMap;
```

```
/// The player-controlled Pac-Man character.  
final Pacman pacman;
```

```
/// All four ghost instances.  
final List<Ghost> ghosts;
```

```
/// The bonus item that appears periodically.  
final Bonus bonus;
```

```
/// Bonuses collected so far – shown as icons at the bottom of  
the screen.  
final List<BonusType> bonusesTaken;
```



```
// — UI components
```

```
/// In-game overlay text (e.g. "READY!", "GAME OVER").  
final Widget Function(double tileSize) gameMessage;
```

```
/// The D-pad controller shown during normal play.  
final Controller controller;
```

```
/// Retry / Exit buttons shown when [isGameOver] returns  
`true`.  
final EndGameButtons endGameButtons;
```

```
// — Callbacks
```

```
/// Returns the current score.  
final int Function() getScore;
```

```
/// Returns the current number of lives.  
final int Function() getLives;
```

```
/// Returns the current high score.  
final int Function() getHighScore;
```

```
/// Returns whether the game is in a game-over state.  
final bool Function() isGameOver;
```

```
// — Timing
```

```
/// Game tick delta-time in seconds; used to set the rebuild  
timer rate.
```

```
final double dt;

@override
State<GameScreen> createState() => _GameScreenState();
}

class _GameScreenState extends State<GameScreen> {
  // — State


---


  —

  Timer? _uiTimer;

  /// Custom colour overrides loaded from [SharedPreferences].
  Color? _pacmanColor;
  Color? _blinkColor;
  Color? _pinkyColor;
  Color? _inkyColor;
  Color? _clydeColor;
  Color? _mapColor;

  // — Lifecycle


---



  @override
  void initState() {
    super.initState();
    _loadCustomColors();

    // Rebuild the widget at the same cadence as the game loop.
    _uiTimer = Timer.periodic(
      Duration(milliseconds: (widget.dt * 1000).round()),
      (_) => setState(() {}),
    );
  }
}
```

```
}
```

```
@override  
void dispose() {  
    _uiTimer?.cancel();  
    super.dispose();  
}
```

```
// — Widget
```

```
@override  
Widget build(BuildContext context) {  
    final screenSize = MediaQuery.of(context).size;  
  
    // Calculate the largest tile size that fits both  
dimensions.  
    double tileSize = screenSize.width / widget.gameMap.kCols;  
    if (tileSize * widget.gameMap.kRows > screenSize.height) {  
        tileSize = screenSize.height / widget.gameMap.kRows;  
    }  
    final mazeWidth = tileSize * widget.gameMap.kCols;  
    final mazeHeight = tileSize * widget.gameMap.kRows;  
  
    return Scaffold(  
        backgroundColor: Colors.black,  
        body: SafeArea(  
            child: Column(  
                mainAxisAlignment: MainAxisAlignment.start,  
                crossAxisAlignment: CrossAxisAlignment.center,  
                spacing: 20,  
                children: [  
                    _buildHud(),
```

// — Maze

```
Stack(
  children: [
    Center(
      child: Image.asset(
        widget.gameMap.mapAssetPath,
        width: mazeWidth,
        height: mazeHeight,
        fit: BoxFit.fill,
        color: _mapColor,
      ),
    ),
    widget.gameMessage(tileSize),
    ...widget.gameMap.dots.map((dot) =>
dot.showOnStack(tileSize)),
    ...widget.gameMap.superDots.map(
      (superDot) => superDot.showOnStack(tileSize),
    ),
    ...widget.ghosts.map(
      (ghost) => ghost.getSprite(
        tileSize,
        color: _getGhostColor(ghost.name),
      ),
    ),
    widget.pacman.getSprite(tileSize, color:
_pacmanColor),
    widget.pacman.nextDirectionArrow(tileSize,
color: _pacmanColor),
    widget.bonus.getSprite(tileSize),
  ],
),

_buildBonusStrip(widget.bonusesTaken),
```

```
        widget.isGameOver()
            ? widget.endGameButtons
            : widget.controller,
    ],
  ),
);
}

// — HUD



---



/// Builds the top bar showing score, high score, and
remaining lives.
Widget _buildHud() {
  return Container(
    height: 50,
    color: Colors.black,
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      spacing: 40,
      children: [
        _buildScoreColumn("SCORE", widget.getScore()),
        _buildScoreColumn("HIGH SCORE",
widget.getHighScore()),
        _buildLivesIndicator(widget.getLives() - 1),
      ],
    ),
  );
}

/// A two-line label + number column for the score display.
```

```
Widget _buildScoreColumn(String label, int value) {  
  return Column(  
    crossAxisAlignment: CrossAxisAlignment.center,  
    children: [  
      Text(  
        label,  
        style: const TextStyle(  
          fontFamily: "PressStart",  
          color: Colors.white,  
          fontSize: 15,  
        ),  
      ),  
      Text(  
        value.toString(),  
        style: const TextStyle(  
          fontFamily: "PressStart",  
          color: Colors.white,  
          fontSize: 20,  
        ),  
      ),  
    ],  
  );  
}
```

```
/// Renders up to 9 small Pac-Man icons representing remaining  
lives.
```

```
///
```

```
/// Icons are arranged in rows of 3, with a maximum of 3 rows.
```

```
Widget _buildLivesIndicator(int lives) {  
  if (lives <= 0) return const SizedBox(width: 50);
```

```
  final image = Image.asset("assets/game/pacman.png");  
  return Column(  
    spacing: 1,
```

```
mainAxisSize: MainAxisSize.min,
mainAxisAlignment: MainAxisAlignment.end,
children: List.generate(
  min((lives / 3).ceil(), 3),
  (row) => Row(
    mainAxisSize: MainAxisSize.min,
    children: List.generate(
      min(lives - row * 3, 3),
      (_) => Padding(
        padding: const EdgeInsets.symmetric(horizontal:
4),
        child: image,
      ),
    ),
  ),
),
);
}

// — Bonus strip
```

/// Renders up to 8 collected-bonus icons in a right-aligned row.

```
Widget _buildBonusStrip(List<BonusType> bonuses) => Padding(
  padding: const EdgeInsets.all(2.0),
  child: bonuses.isNotEmpty
    ? Row(
      mainAxisAlignment: MainAxisAlignment.end,
      crossAxisAlignment: CrossAxisAlignment.end,
      mainAxisSize: MainAxisSize.min,
      children: bonuses
        .sublist(0, min(8, bonuses.length))
        .map(
```

```
(bonusType) => Image.asset(

"${Bonus.bonusAssetsPath}${bonusType.name}${Bonus.bonusesAssetsE
ndFile}",

    ),
    )
    .toList(),
    )
    : const SizedBox(height: 5),
);

// — Private helpers

/// Loads custom character/map colour overrides from
[SharedPreferences].
Future<void> _loadCustomColors() async {
    final prefs = await SharedPreferences.getInstance();

    Color? readColor(String key) {
        final a = prefs.getInt("$key/a");
        final r = prefs.getInt("$key/r");
        final g = prefs.getInt("$key/g");
        final b = prefs.getInt("$key/b");
        if (a == null || r == null || g == null || b == null)
return null;
        return Color.fromARGB(a, r, g, b);
    }

    final pacman = readColor("pacmanColor");
    final blinky = readColor("blinkyColor");
    final pinky = readColor("pinkyColor");
    final inky = readColor("inkyColor");
    final clyde = readColor("clydeColor");
```



```
final map      = readColor("mapColor");

// Only call setState if at least one colour was loaded.
if ([pacman, blinky, pinky, inky, clyde, map].any((c) => c
!= null)) {
    setState(() {
        _pacmanColor = pacman ?? _pacmanColor;
        _blinkyColor = blinky ?? _blinkyColor;
        _pinkyColor  = pinky  ?? _pinkyColor;
        _inkyColor   = inky   ?? _inkyColor;
        _clydeColor  = clyde  ?? _clydeColor;
        _mapColor    = map    ?? _mapColor;
    });
}

}

/// Returns the custom tint colour for the ghost with [name],
or `null`.
Color? _getGhostColor(String name) {
    return switch (name) {
        "Blinky" => _blinkyColor,
        "Pinky"  => _pinkyColor,
        "Inky"   => _inkyColor,
        "Clyde"  => _clydeColor,
        _        => null,
    };
}

}
```

File: /lib/game/game_timer.dart

```
/// A simple countdown timer used to schedule in-game events.
///
/// Create a [GameTimer] with a duration in seconds and an
optional [onEnd]
/// callback. Call [start] to activate it, then call [update]
every game tick
/// with the real elapsed delta-time. When the accumulated time
reaches
/// [duration], [onEnd] is fired and the timer either stops or
restarts
/// depending on [isLoop].
class GameTimer {
  /// Creates a [GameTimer].
  ///
  /// [duration] - seconds until [onEnd] fires.
  /// [onEnd]    - callback invoked when the timer expires.
  /// [isOn]     - whether the timer starts in an active state.
  /// [isLoop]   - if `true` the timer automatically restarts
after firing.
  GameTimer(
    this.duration, {
    this.onEnd,
    this.isOn = false,
    this.isLoop = false,
  }) : _clock = 0;

  // — Configuration
```

```
/// Total seconds that must elapse before [onEnd] is
triggered.
final double duration;

/// Callback invoked when the timer reaches [duration]. May be
`null`.
final void Function()? onEnd;

/// Whether the timer restarts automatically after firing.
final bool isLoop;

// — State



---



/// Seconds accumulated since the timer last started or reset.
double _clock;

/// Whether the timer is currently counting.
bool isOn;

// — Public API



---



/// Advances the timer by [dt] seconds and fires [onEnd] if
the duration is reached.
void update(double dt) {
    if (!isOn) return;

    _clock += dt;

    if (_clock >= duration) {
        onEnd?.call();
        _reset();
    }
}
```

```
}

/// Activates the timer so it begins counting on the next
[update] call.
void start() {
    isOn = true;
}

/// Pauses the timer without resetting its accumulated time.
void pause() {
    isOn = false;
}

// — Private helpers



---



/// Resets accumulated time and stops (or loops) the timer.
void _reset() {
    isOn = isLoop;
    _clock = 0;
}
}
```

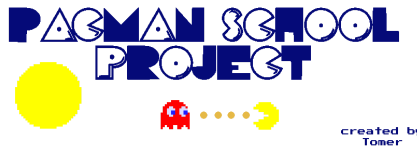
File: /lib/game/pacman.dart

```
import 'package:flutter/material.dart';
import 'package:pacman_application/utils/constants.dart';
import
'package:pacman_application/game/animation/animated_character.da
rt';
import 'package:pacman_application/game/direction.dart';
import 'package:pacman_application/game/map/game_map.dart';
import
'package:pacman_application/game/animation/sprite_animation.dart
';

/// Pac-Man: the player-controlled character.
///
/// [move] applies directional input with wall-collision checks
and tunnel
/// wrapping. [updateAnimation] selects the correct directional
cycle.
/// Call [playDeath] when a life is lost to switch to the death
animation, then
/// drive it manually with [updateDeathAnimation].
class Pacman extends AnimatedCharacter {
  /// Creates [Pacman] and loads all sprite animations.
  Pacman({required super.gameManager}) {
    x = getGameMap().startingX;
    y = getGameMap().startingY;

    _deathAnimation = SpriteAnimation(
      characterName: _kName,
      animationName: "death",
      spritesAmount: 12,
```

Pacman Android Edition



תומר כץ

```
        isLoop: false,
    );
    _idleAnimation = SpriteAnimation(
        characterName: _kName,
        animationName: "idle",
        spritesAmount: 1,
        isLoop: true,
    );
    _upAnimation = SpriteAnimation(
        characterName: _kName,
        animationName: "up",
        spritesAmount: 3,
        isLoop: true,
    );
    _rightAnimation = SpriteAnimation(
        characterName: _kName,
        animationName: "right",
        spritesAmount: 3,
        isLoop: true,
    );
    _downAnimation = SpriteAnimation(
        characterName: _kName,
        animationName: "down",
        spritesAmount: 3,
        isLoop: true,
    );
    _leftAnimation = SpriteAnimation(
        characterName: _kName,
        animationName: "left",
        spritesAmount: 3,
        isLoop: true,
    );

    currentAnimation = _deathAnimation;
```

```
}
```

```
// — Constants
```

```
static const String _kName = "pacman";
```

```
/// Movement speed in tiles per second.
```

```
static const double _kSpeed = 15;
```

```
// — AnimatedCharacter overrides (identity)
```

```
@override
```

```
final String name = _kName;
```

```
// — AnimatedCharacter overrides (mutable state)
```

```
@override
```

```
Direction currentDirection = Direction.left;
```

```
@override
```

```
Direction nextDirection = Direction.left;
```

```
@override
```

```
double x = 0;
```

```
@override
```

```
double y = 0;
```

```
// — Animations
```

```
late SpriteAnimation _deathAnimation;
late SpriteAnimation _idleAnimation;
late SpriteAnimation _upAnimation;
late SpriteAnimation _rightAnimation;
late SpriteAnimation _downAnimation;
late SpriteAnimation _leftAnimation;
```

```
@override
```

```
List<SpriteAnimation> get animations => [
    _deathAnimation,
    _idleAnimation,
    _upAnimation,
    _rightAnimation,
    _downAnimation,
    _leftAnimation,
];
```

```
// — Public API
```

```
    /// Switches to the death animation. Drive it with
[updateDeathAnimation].
    void playDeath() {
        currentAnimation = _deathAnimation;
    }

    /// Advances only the death animation by [dt] seconds.
    ///
    /// Called by [GameManager] while the game is paused after a
death.
    void updateDeathAnimation(double dt) {
        currentAnimation.update(dt);
    }
```



```
/// Returns a [Positioned] arrow icon indicating
[nextDirection].
///
/// Helps the player see their queued turn. Pass [color] to
match the
/// custom Pac-Man colour setting.
Widget nextDirectionArrow(double tileSize, {Color? color}) {
  return Positioned(
    left: (x + nextDirection.toDelta().$1) * tileSize,
    top: (y + nextDirection.toDelta().$2) * tileSize,
    child: Transform.rotate(
      angle: nextDirection.toAngle(),
      child: Icon(
        Icons.arrow_drop_up_sharp,
        color: color ?? pacmanColor,
        size: tileSize,
      ),
    ),
  );
}
```

```
// — AnimatedCharacter overrides (behaviour)
```

```
@override
void move(double dt) {
  // Apply a queued direction change if the next tile is
passable.
  if (currentDirection != nextDirection &&
    !getGameMap().isWall(x.round(), y.round(),
nextDirection, isPacman: true)) {
    x = x.roundToDouble();
    y = y.roundToDouble();
    currentDirection = nextDirection;
```

```
}

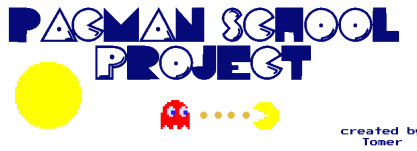
// Stop at walls.
if (getGameMap().isWall(x.round(), y.round(),
currentDirection, isPacman: true)) {
    x = x.roundToDouble();
    y = y.roundToDouble();
    return;
}

// Advance position.
switch (currentDirection) {
    case Direction.up:
        y -= _kSpeed * dt;
        break;
    case Direction.down:
        y += _kSpeed * dt;
        break;
    case Direction.left:
        x -= _kSpeed * dt;
        break;
    case Direction.right:
        x += _kSpeed * dt;
        break;
}

x = x.clamp(0, getGameMap().kCols - 1).toDouble();
y = y.clamp(0, getGameMap().kRows - 1).toDouble();

// Handle tunnel wrap-around.
if (getGameMap().map[y.round()][x.round()] ==
GameMap.kTunnel) {
    if (x < 1) {
        x = getGameMap().kCols - 2.01;
    }
}
```

Pacman Android Edition



תומר כץ

```
        } else if (x > getGameMap().kCols - 2) {
            x = 1.01;
        }
    }
}

@Override
void updateAnimation(double dt) {
    currentAnimation = switch (currentDirection) {
        Direction.up => _upAnimation,
        Direction.down => _downAnimation,
        Direction.left => _leftAnimation,
        Direction.right => _rightAnimation,
    };

    for (final animation in animations) {
        if (animation != currentAnimation) {
            animation.idle();
        } else {
            animation.update(dt);
        }
    }
}

@Override
void reset() {
    x = getGameMap().startingX;
    y = getGameMap().startingY;
    currentDirection = Direction.left;
    nextDirection = Direction.left;
}
}
```

File: /lib/game/super_dot.dart

```
import 'package:flutter/material.dart';

/// A large power pellet that temporarily puts ghosts into
/// frightened mode.
///
/// Functionally identical to [Dot] but larger and worth more
/// points. When
/// Pac-Man collects a [SuperDot] the [GameManager] switches all
/// ghosts to
/// [GhostState.frightened0].
class SuperDot extends StatelessWidget {
  /// Creates a [SuperDot] at grid position ([x], [y]).
  ///
  /// [onEaten] is called exactly once per round when the pellet
  is collected.
  SuperDot(this.x, this.y, {super.key, required this.onEaten})
    : _isEaten = false;

  // — Identity / position

  /// Horizontal grid column of this power pellet.
  final int x;

  /// Vertical grid row of this power pellet.
  final int y;

  // — State
```

```
/// Whether this pellet has already been collected in the
current round.
bool _isEaten;

// — Callbacks



---



/// Invoked once when Pac-Man eats this power pellet.
final void Function() onEaten;

// — Public API



---



/// Checks whether Pac-Man (at [pacmanX], [pacmanY]) is close
enough to eat
/// this pellet. Tolerance is  $\pm 0.5$  tiles on both axes.
void checkIfEaten(double pacmanX, double pacmanY) {
    if (!_isEaten && (pacmanX - x).abs() < 0.5 && (pacmanY -
y).abs() < 0.5) {
        _eat();
    }
}

/// Restores the pellet so it is visible and collectable
again.
void reset() {
    _isEaten = false;
}

/// Returns a [Positioned] widget that renders the pellet
inside a [Stack].
///
```

```
/// The pellet is centred inside its tile using a 0.5-tile  
offset.
```

```
/// Renders an empty [Container] when already eaten.
```

```
Widget showOnStack(double tileSize) {  
    return Positioned(  
        left: (x + 0.5) * tileSize,  
        top: (y + 0.5) * tileSize,  
        child: _isEaten ? Container() : this,  
    );  
}
```

```
// — Private helpers
```

```
/// Marks the pellet as eaten and fires [onEaten].
```

```
void _eat() {  
    onEaten();  
    _isEaten = true;  
}
```

```
// — Widget
```

```
@override
```

```
Widget build(BuildContext context) => Container(  
    constraints: BoxConstraints.tight(const Size(7, 7)),  
    decoration: BoxDecoration(  
        color: Colors.amber[900],  
        shape: BoxShape.circle,  
    ),  
);  
}
```

File: /lib/game/map/game_map.dart

```
import 'package:pacman_application/game/direction.dart';
import 'package:pacman_application/game/dot.dart';
import 'package:pacman_application/game/super_dot.dart';

/// Represents the static layout and collectible state of the
game map.
///
/// The map is a 2-D grid of integer tile codes (see the `k*`
constants).
/// [GameMap] parses the raw grid on construction, building
[dots] and
/// [superDots] lists that track which collectibles are still
available.
///
/// Call [reset] between rounds to restore all collectibles.
class GameMap {
  /// Creates a [GameMap] from [map] and registers the two
collectible callbacks.
  ///
  /// [onDotEaten]      - called by each [Dot] when Pac-Man eats
it.
  /// [onSuperDotEaten] - called by each [SuperDot] when Pac-Man
eats it.
  GameMap({
    required this.map,
    required this.onDotEaten,
    required this.onSuperDotEaten,
  }) {
    _init();
  }
}
```

```
// — Tile-type constants
```

```
/// An empty passable tile with no collectible.  
static const int kEmpty = 0;
```

```
/// A solid wall tile that blocks all movement.  
static const int kWall = 1;
```

```
/// A small dot collectible.  
static const int kSmallDot = 2;
```

```
/// A large power-pellet collectible.  
static const int kSuperDot = 3;
```

```
/// A bonus-item spawn tile.  
static const int kBonus = 4;
```

```
/// A tunnel tile: characters that reach the edge wrap to the  
other side.  
static const int kTunnel = 5;
```

```
/// The ghost-house door – impassable for Pac-Man (except  
upward exit).  
static const int kGhostHouseDoor = 6;
```

```
/// The interior of the ghost house where ghosts respawn.  
static const int kGhostHouseInterior = 7;
```

```
/// Pac-Man's starting tile (informational; not used for  
collision).  
static const int kStartingPosition = 8;
```



```
// — Dimensions & layout constants
```

```
/// Number of tile columns in the map.  
final int kCols = 28;
```

```
/// Number of tile rows in the map.  
final int kRows = 31;
```

```
/// Pac-Man's initial horizontal position in tile units.  
final double startingX = 13.5;
```

```
/// Pac-Man's initial vertical position in tile units.  
final double startingY = 23;
```

```
/// Horizontal tile coordinate of the ghost-house entrance  
used for pathfinding.  
final int ghostHouseX = 14;
```

```
/// Vertical tile coordinate of the ghost-house entrance used  
for pathfinding.  
final int ghostHouseY = 15;
```

```
/// Horizontal tile position where the bonus item appears.  
final double bonusX = 13.5;
```

```
/// Vertical tile position where the bonus item appears.  
final double bonusY = 17;
```

```
/// Asset path for the pre-rendered maze image.  
final String mapAssetPath = 'assets/game/map/map.png';
```

```
// — Map data
```

```
/// The raw 2-D grid of tile codes ([kRows] × [kCols]).  
final List<List<int>> map;
```

```
// — Collectibles
```

```
/// All small dots on the map (eaten and uneaten).  
late final List<Dot> dots = [];
```

```
/// All power pellets on the map (eaten and uneaten).  
late final List<SuperDot> superDots = [];
```

```
/// Number of collectibles that have not yet been eaten this  
round.
```

```
late int remainingDots = dots.length + superDots.length;
```

```
// — Callbacks
```

```
/// Invoked by a [Dot] when Pac-Man collects it.  
final void Function() onDotEaten;
```

```
/// Invoked by a [SuperDot] when Pac-Man collects it.  
final void Function() onSuperDotEaten;
```

```
// — Public API
```

```
/// Restores all collectibles and resets [remainingDots] for a  
new round.
```

```
void reset() {  
    for (final dot in dots) {  
        dot.reset();  
    }  
}
```

```
}  
for (final superDot in superDots) {  
    superDot.reset();  
}  
remainingDots = dots.length + superDots.length;  
}  
  
/// Returns `true` if moving one step in [direction] from  
([col], [row])  
/// would enter a wall tile.  
///  
/// Out-of-bounds targets are treated as walls.  
/// When [isPacman] is `true`, the ghost-house door also acts  
as a wall  
/// (except when moving upward, which allows exiting the ghost  
house).  
bool isWall(int col, int row, Direction direction, {bool  
isPacman = false}) {  
    int targetCol = col;  
    int targetRow = row;  
  
    switch (direction) {  
        case Direction.up:  
            targetRow -= 1;  
            break;  
        case Direction.down:  
            targetRow += 1;  
            break;  
        case Direction.left:  
            targetCol -= 1;  
            break;  
        case Direction.right:  
            targetCol += 1;  
            break;  
    }
```

```
}

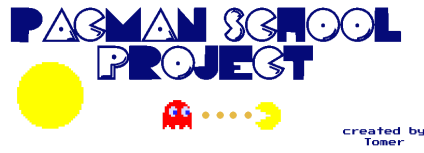
if (targetCol < 0 ||
    targetCol >= kCols ||
    targetRow < 0 ||
    targetRow >= kRows) {
    return true; // out-of-bounds counts as a wall
}

return map[targetRow][targetCol] == kWall ||
    (isPacman &&
     map[targetRow][targetCol] == kGhostHouseDoor &&
     direction != Direction.up);
}

// — Private helpers
```

```
/// Scans [map] and populates [dots] and [superDots].
void _init() {
    for (int row = 0; row < kRows; row++) {
        for (int col = 0; col < kCols; col++) {
            if (map[row][col] == kSmallDot) {
                dots.add(
                    Dot(
                        col,
                        row,
                        onEaten: () {
                            onDotEaten();
                            remainingDots--;
                        },
                    ),
                );
            } else if (map[row][col] == kSuperDot) {
```

Pacman Android Edition



תומר כץ

```
superDots.add(  
    SuperDot(  
        col,  
        row,  
        onEaten: () {  
            onSuperDotEaten();  
            remainingDots--;  
        },  
    ),  
);  
}  
}  
}  
}  
}  
}
```

File: /lib/game/map/map_paller.dart

```
import 'package:flutter/material.dart';
import 'package:pacman_application/game/map/game_map.dart';
import 'package:pacman_application/game/map/mazePainter.dart';

/// A [StatelessWidget] that lays out the Pac-Man maze.
///
/// The maze is rendered as two stacked layers:
/// 1. A static background PNG (`assets/game
sprites/map/map.png`).
/// 2. A [CustomPaint] overlay produced by [MazePainter] that
draws all
///   game-tile states (walls, dots, tunnels, ghost house,
etc.).
///
/// Both layers are sized to fill as much of the screen as
possible while
/// preserving the grid's aspect ratio (determined by
[GameMap.kCols] ×
/// [GameMap.kRows]).
class MapPalette extends StatelessWidget {
  /// Creates a [MapPalette] for the given [gameMap].
  const MapPalette({super.key, required this.gameMap});

  //
  // Fields
  //
  //
```



```
/// The game map data used to compute tile sizes and passed to
[MazePainter].
final GameMap gameMap;

//
-----

// Build
//
-----

@Override
Widget build(BuildContext context) {
    final Size screenSize = MediaQuery.of(context).size;

    // Compute the largest tile size that fits the screen in
    both dimensions.
    final double tileSize = _computeTileSize(screenSize);

    final double mazeWidth = tileSize * gameMap.kCols;
    final double mazeHeight = tileSize * gameMap.kRows;

    return Stack(
        children: [
            // Layer 1: static background image.
            Center(
                child: Image.asset(
                    'assets/game sprites/map/map.png',
                    width: mazeWidth,
                    height: mazeHeight,
                    fit: BoxFit.fill,
                ),
            ),
        ],
    );
}
```

```
),

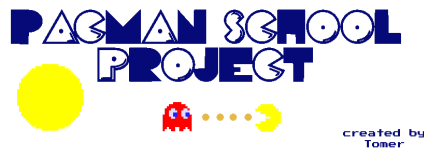
// Layer 2: dynamic tile overlay drawn by MazePainter.
Center(
  child: SizedBox(
    width: mazeWidth,
    height: mazeHeight,
    child: ClipRect(
      child: CustomPaint(
        painter: MazePainter(gameMap: gameMap),
        size: Size(mazeWidth, mazeHeight),
      ),
    ),
  ),
),
],
);
}

//
-----
-----

// Helpers
//
-----
-----

/// Calculates the tile size (in logical pixels) that keeps
the entire maze
/// visible within [screenSize].
///
/// Starts with a width-based tile size, then shrinks it if
the resulting
/// maze height would exceed the available screen height.
```


Pacman Android Edition



תומר כץ

```
double _computeTileSize(Size screenSize) {  
    double tileSize = screenSize.width / gameMap.kCols;  
    if (tileSize * gameMap.kRows > screenSize.height) {  
        tileSize = screenSize.height / gameMap.kRows;  
    }  
    return tileSize;  
}  
}
```

File: /lib/game/map/mazePainter.dart

```
import 'package:flutter/material.dart';
import 'package:pacman_application/game/map/game_map.dart';

/// A [CustomPainter] that iterates over every cell in [gameMap]
and draws the
/// appropriate shape and colour for each tile type.
///
/// Tile-type constants are defined in [GameMap] (e.g.
[GameMap.kWall],
/// [GameMap.kSmallDot]). Unknown tile values are silently
ignored.
///
/// This painter is declared `const`-constructible and never
repaints once the
/// initial frame is drawn ([shouldRepaint] always returns
`false`). If the map
/// can change at runtime, override [shouldRepaint] to compare
old and new
/// [GameMap] instances.
class MazePainter extends CustomPainter {
  /// Creates a [MazePainter] for the given [gameMap].
  const MazePainter({required this.gameMap});

  //
  -----
  -----
  // Fields
  //
  -----
  -----
```

```
/// The game map whose tiles will be painted onto the canvas.
final GameMap gameMap;

//
-----

// Paint colours (private constants for readability)
//
-----

static const Color _wallColor = Colors.blue;
static const Color _smallDotColor = Colors.yellow;
static const Color _superDotColor = Colors.red;
static const Color _bonusColor = Colors.green;
static const Color _tunnelColor = Colors.purple;
static const Color _ghostHouseDoorColor = Colors.orange;
static const Color _ghostHouseInteriorColor = Colors.grey;
static const Color _startingPositionColor = Colors.white;

//
-----

// CustomPainter overrides
//
-----

/// Paints every tile in [gameMap] onto [canvas] using the
given [size].
///
/// The canvas is divided into a uniform grid where each cell
is
```

```
/// `size.width / gameMap.kCols` logical pixels wide and tall.
@Override
void paint(Canvas canvas, Size size) {
    final double tileSize = size.width / gameMap.kCols;

    for (int row = 0; row < gameMap.kRows; row++) {
        for (int col = 0; col < gameMap.kCols; col++) {
            _paintTile(canvas, row: row, col: col, tileSize:
tileSize);
        }
    }
}

/// Returns `false` because the map is treated as immutable
after the first
/// paint. Override this if dynamic map updates are required.
@Override
bool shouldRepaint(covariant CustomPainter oldDelegate) =>
false;

//
-----
-----
// Private helpers
//
-----
-----

/// Draws the tile at ([row], [col]) according to its value in
[gameMap.map].
void _paintTile(
    Canvas canvas, {
        required int row,
        required int col,
```

```
        required double tileSize,
    }) {
        final int tileValue = gameMap.map[row][col];

        // Pixel origin of this tile.
        final double left = col * tileSize;
        final double top = row * tileSize;

        // Centre point of this tile, used for circles.
        final Offset center = Offset(left + tileSize / 2, top +
tileSize / 2);

        switch (tileValue) {
            case GameMap.kEmpty:
                // Nothing to draw.
                break;

            case GameMap.kWall:
                _drawRect(canvas, left: left, top: top, tileSize:
tileSize, color: _wallColor);

            case GameMap.kSmallDot:
                _drawCircle(canvas, center: center, radius: tileSize /
6, color: _smallDotColor);

            case GameMap.kSuperDot:
                _drawCircle(canvas, center: center, radius: tileSize /
3, color: _superDotColor);

            case GameMap.kBonus:
                _drawCircle(canvas, center: center, radius: tileSize /
4, color: _bonusColor);

            case GameMap.kTunnel:
```

```
        _drawRect(canvas, left: left, top: top, tileSize:
tileSize, color: _tunnelColor);

        case GameMap.kGhostHouseDoor:
            _drawRect(canvas, left: left, top: top, tileSize:
tileSize, color: _ghostHouseDoorColor);

        case GameMap.kGhostHouseInterior:
            _drawRect(canvas, left: left, top: top, tileSize:
tileSize, color: _ghostHouseInteriorColor);

        case GameMap.kStartingPosition:
            _drawCircle(canvas, center: center, radius: tileSize /
2, color: _startingPositionColor);

        default:
            // Unknown tile type - ignore silently.
            break;
    }
}

/// Draws a filled rectangle covering one tile at ([left],
[top]).
void _drawRect(
    Canvas canvas, {
        required double left,
        required double top,
        required double tileSize,
        required Color color,
    }) {
    canvas.drawRect(
        Rect.fromLTWH(left, top, tileSize, tileSize),
        Paint()..color = color,
    );
}
```

```
}

/// Draws a filled circle centred at [center] with the given
[radius].
void _drawCircle(
    Canvas canvas, {
    required Offset center,
    required double radius,
    required Color color,
}) {
    canvas.drawCircle(center, radius, Paint()..color = color);
}
}
```

File: /lib/game/ghosts/blinky.dart

```
import
'package:pacman_application/game/animation/sprite_animation.dart
';
import 'package:pacman_application/game/direction.dart';
import 'package:pacman_application/game/ghosts/ghost.dart';

/// Blinky (the red ghost) – the most aggressive chaser.
///
/// Chase strategy: targets Pac-Man's exact current tile.
/// Scatter corner: **top-right** of the maze.
class Blinky extends Ghost {
  /// Creates [Blinky] and registers optional death/eaten
  callbacks.
  Blinky({required super.gameManager, super.onDeath,
super.onEaten});

  // — Identity

  @override
  String name = "Blinky";

  // — Position

  @override
  double x = 13;

  @override
  double y = 11;
```



```
@override  
(int x, int y) startingPosition = (13, 11);  
  
// — Direction
```

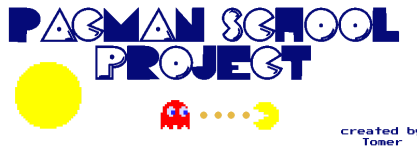
```
@override  
Direction currentDirection = Direction.left;  
  
@override  
Direction nextDirection = Direction.left;  
  
// — Animation
```

```
@override  
SpriteAnimation get idleAnimation => up;  
  
// — Scatter corner
```

```
@override  
late (int, int) scatterLocation =  
(getGameMap().map.first.length - 1, 1);  
  
// — Chase behaviour
```

```
/// Targets Pac-Man's current tile directly via BFS.  
@override  
void chase(  
    double dt,  
    double pacmanX,
```

Pacman Android Edition



תומר כץ

```
double pacmanY,  
Direction pacmanDirection,  
) {  
    hasStartedFrightenedMode = false;  
  
    final direction = ghostChase(  
        pacmanX.round(),  
        pacmanY.round(),  
        x.round(),  
        y.round(),  
    );  
    if (direction != null) nextDirection = direction;  
  
    if (nextDirection != currentDirection &&  
        !getGameMap().isWall(x.round(), y.round(),  
nextDirection)) {  
        currentDirection = nextDirection;  
        x = x.roundToDouble();  
        y = y.roundToDouble();  
    }  
  
    if (getGameMap().isWall(x.round(), y.round(),  
currentDirection)) {  
        x = x.roundToDouble();  
        y = y.roundToDouble();  
        return;  
    }  
  
    stepForward(dt);  
}  
}
```

File: /lib/game/ghosts/clyde.dart

```
import
'package:pacman_application/game/animation/sprite_animation.dart
';
import 'package:pacman_application/game/direction.dart';
import 'package:pacman_application/game/ghosts/ghost.dart';
import 'package:pacman_application/utils/util.dart';

/// Clyde (the orange ghost) – the unpredictable one.
///
/// Chase strategy:
/// - When **8 or more tiles away** from Pac-Man, chases him
directly like Blinky.
/// - When **closer than 8 tiles**, retreats to his scatter
corner instead,
///   creating an oscillating approach/retreat behaviour.
///
/// Scatter corner: **bottom-left** of the maze.
class Clyde extends Ghost {
  /// Creates [Clyde] and registers optional death/eaten
callbacks.
  Clyde({required super.gameManager, super.onDeath,
super.onEaten});

  // — Identity



---



  @override
  String name = "Clyde";
```

// — Position

```
@override
double x = 14;

@Override
double y = 14;

@Override
(int x, int y) startPosition = (14, 14);
```

// — Direction

```
@override
Direction currentDirection = Direction.left;

@Override
Direction nextDirection = Direction.left;
```

// — Animation

```
@override
SpriteAnimation get idleAnimation => right;
```

// — Scatter corner

```
@override
late (int, int) scatterLocation = (1, getGameMap().map.length
- 2);
```

// — Chase behaviour

```
    /// Chases Pac-Man when  $\geq 8$  tiles away; retreats to scatter
corner when closer.
    @override
    void chase(double dt, double pacmanX, double pacmanY,
Direction pacmanDirection) {
        // Choose target: scatter corner if too close, Pac-Man's
tile otherwise.
        final target = dist(pacmanX - x, pacmanY - y)  $\geq$  8
            ? (pacmanX.round(), pacmanY.round())
            : scatterLocation;

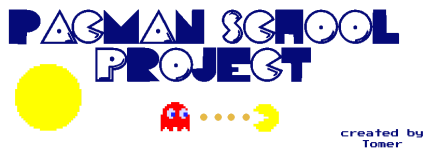
        final direction = ghostChase(target.$1, target.$2,
x.round(), y.round());
        if (direction != null) nextDirection = direction;

        if (nextDirection != currentDirection &&
            !getGameMap().isWall(x.round(), y.round(),
nextDirection)) {
            currentDirection = nextDirection;
            x = x.roundToDouble();
            y = y.roundToDouble();
        }

        if (getGameMap().isWall(x.round(), y.round(),
currentDirection)) {
            x = x.roundToDouble();
            y = y.roundToDouble();
            return;
        }

        stepForward(dt);
```

Pacman Android Edition



תומר כץ

```
}  
}
```

File: /lib/game/ghosts/ghost.dart

```
import 'dart:collection';
import 'dart:math';
import 'package:flutter/material.dart';
import
'package:pacman_application/game/animation/animated_character.da
rt';
import
'package:pacman_application/game/animation/sprite_animation.dart
';
import 'package:pacman_application/game/direction.dart';
import 'package:pacman_application/game/map/game_map.dart';
import 'package:pacman_application/utils/util.dart';

/// All possible behavioural states a ghost can be in.
enum GhostState {
  /// Pursuing Pac-Man using the ghost's unique targeting
  strategy.
  chase,

  /// Retreating to the ghost's fixed corner of the maze.
  scatter,

  /// Flashing blue – vulnerable, warning phase about to expire.
  frightened0,

  /// Flashing white – last few seconds of vulnerability.
  frightened1,

  /// Eyes-only mode: returning to the ghost house after being
  eaten.
```

```
eaten,

    /// Stationary inside the ghost house; not yet released into
the maze.
    idle,
}

/// Abstract base for all four ghosts (Blinky, Pinky, Inky,
Clyde).
///
/// Provides the shared state machine ([GhostState]), movement
modes
/// ([scatter], [frightened], [eaten]), the BFS pathfinder
([ghostChase]),
/// and animation management.
///
/// Subclasses must implement:
/// - [name], [x], [y], [currentDirection], [nextDirection]
/// - [startingPosition] and [scatterLocation]
/// - [idleAnimation] - animation shown in [GhostState.idle]
/// - [chase] - the ghost's unique targeting behaviour
abstract class Ghost extends AnimatedCharacter {
    /// Creates a [Ghost] and initialises all animation objects.
    ///
    /// [onDeath] - called when Pac-Man collides with a
non-frightened ghost.
    /// [onEaten] - called when Pac-Man eats this ghost while
frightened.
    Ghost({required super.gameManager, this.onDeath,
this.onEaten}) {
        _up = SpriteAnimation(
            characterName: name, animationName: "up",
            spritesAmount: 2, isLoop: true,
        );
    }
}
```



```
_right = SpriteAnimation(  
    characterName: name, animationName: "right",  
    spritesAmount: 2, isLoop: true,  
);  
_down = SpriteAnimation(  
    characterName: name, animationName: "down",  
    spritesAmount: 2, isLoop: true,  
);  
_left = SpriteAnimation(  
    characterName: name, animationName: "left",  
    spritesAmount: 2, isLoop: true,  
);  
_eatenUp = SpriteAnimation(  
    characterName: name, animationName: "eatenUp",  
    spritesAmount: 1, isLoop: true,  
);  
_eatenRight = SpriteAnimation(  
    characterName: name, animationName: "eatenRight",  
    spritesAmount: 1, isLoop: true,  
);  
_eatenDown = SpriteAnimation(  
    characterName: name, animationName: "eatenDown",  
    spritesAmount: 1, isLoop: true,  
);  
_eatenLeft = SpriteAnimation(  
    characterName: name, animationName: "eatenLeft",  
    spritesAmount: 1, isLoop: true,  
);  
_frightened0 = SpriteAnimation(  
    characterName: name, animationName: "frightened0",  
    spritesAmount: 4, isLoop: true,  
);  
_frightened1 = SpriteAnimation(  
    characterName: name, animationName: "frightened1",
```

```
spritesAmount: 4, isLoop: true,
);

currentAnimation = idleAnimation;
_currentEyesAnimation = _eatenUp;
}

// — Abstract members (subclass must implement)


---



/// Grid coordinates the ghost spawns at and returns to on
[reset].
abstract (int x, int y) startingPosition;

/// Tile this ghost targets while in [GhostState.scatter].
abstract (int, int) scatterLocation;

/// Animation played while the ghost is in [GhostState.idle].
SpriteAnimation get idleAnimation;

/// Ghost-specific Pac-Man targeting logic, called each tick
during [GhostState.chase].
void chase(double dt, double pacmanX, double pacmanY,
Direction pacmanDirection);

// — Public state


---



/// Current behavioural state (read and written by
[GameManager] and self).
GhostState state = GhostState.scatter;

/// Movement speed in tiles per second.
double speed = 8;
```

```
/// Guards against re-applying the one-time direction reversal
on entry to
/// frightened mode. Subclasses should reset this to `false`
at the start
/// of their [chase] override.
bool hasStartedFrightenedMode = false;

// — Callbacks



---



/// Invoked when Pac-Man collides with this non-frightened
ghost.
final void Function()? onDeath;

/// Invoked when Pac-Man eats this frightened ghost.
final void Function()? onEaten;

// — Directional animation getters (for subclass use in
idleAnimation) ——

/// Two-frame looping upward-movement animation.
SpriteAnimation get up => _up;

/// Two-frame looping rightward-movement animation.
SpriteAnimation get right => _right;

/// Two-frame looping downward-movement animation.
SpriteAnimation get down => _down;

/// Two-frame looping leftward-movement animation.
SpriteAnimation get left => _left;
```

```
// — Private animation storage
```

```
late SpriteAnimation _up;  
late SpriteAnimation _right;  
late SpriteAnimation _down;  
late SpriteAnimation _left;  
late SpriteAnimation _eatenUp;  
late SpriteAnimation _eatenRight;  
late SpriteAnimation _eatenDown;  
late SpriteAnimation _eatenLeft;  
late SpriteAnimation _frightened0;  
late SpriteAnimation _frightened1;
```

```
/// Directional eyes overlay rendered above the body sprite  
during chase/scatter.
```

```
SpriteAnimation? _currentEyesAnimation;
```

```
final Random _random = Random();
```

```
@override
```

```
List<SpriteAnimation> get animations => [  
    _up, _right, _down, _left,  
    _eatenUp, _eatenRight, _eatenDown, _eatenLeft,  
    _frightened0, _frightened1,  
];
```

```
// — AnimatedCharacter overrides
```

```
@override
```

```
Widget getSprite(double tileSize, {Color? color}) =>  
Positioned(  
    top: y * tileSize,
```

```
left: x * tileSize,
child: Stack(
  children: [
    // Body - tinted only during chase/scatter; other states
have
    // built-in sprite colours.
    currentAnimation.getImage(
      color: state == GhostState.chase || state ==
GhostState.scatter
        ? color
        : null,
    ),
    // Eyes overlay (visible during chase/scatter when a
custom colour is set).
    if (color != null && _currentEyesAnimation != null)
      _currentEyesAnimation!.getImage()
    else
      Container(),
  ],
),
);

@override
void move(double dt) {
  final pacmanX = getPacman().x;
  final pacmanY = getPacman().y;
  final pacmanDirection = getPacman().currentDirection;

  switch (state) {
    case GhostState.chase:
      chase(dt, pacmanX, pacmanY, pacmanDirection);
    case GhostState.scatter:
      scatter(dt);
    case GhostState.frightened0 || GhostState.frightened1:
```

```
        frightened(dt);
    case GhostState.eaten:
        eaten(dt);
    case GhostState.idle:
        break;
}

x = x.clamp(0, getGameMap().kCols - 1);
y = y.clamp(0, getGameMap().kRows - 1);
}

@Override
void updateAnimation(double dt) {
    currentAnimation = switch (state) {
        GhostState.chase || GhostState.scatter => switch
(currentDirection) {
            Direction.up      => _up,
            Direction.down    => _down,
            Direction.left    => _left,
            Direction.right   => _right,
        },
        GhostState.frightened0 => _frightened0,
        GhostState.frightened1 => _frightened1,
        GhostState.eaten => switch (currentDirection) {
            Direction.up      => _eatenUp,
            Direction.down    => _eatenDown,
            Direction.left    => _eatenLeft,
            Direction.right   => _eatenRight,
        },
        GhostState.idle => idleAnimation,
    };

    _currentEyesAnimation = switch (state) {
```

```
        GhostState.chase || GhostState.scatter => switch
(currentDirection) {
    Direction.up      => _eatenUp,
    Direction.down    => _eatenDown,
    Direction.left    => _eatenLeft,
    Direction.right   => _eatenRight,
},
_ => null,
};

    for (final animation in animations) {
        if (animation != currentAnimation && animation !=
_currentEyesAnimation) {
            animation.idle();
        } else {
            animation.update(dt);
        }
    }
}

@Override
void update(double dt) {
    move(dt);
    updateState();
    updateAnimation(dt);
}

@Override
void reset() {
    x = startingPosition.$1.toDouble();
    y = startingPosition.$2.toDouble();
    state = GhostState.scatter;
}
```

// — Shared movement modes

```
/// Moves the ghost toward [scatterLocation] using BFS.
void scatter(double dt) {
    hasStartedFrightenedMode = false;

    final direction = ghostChase(
        scatterLocation.$1, scatterLocation.$2, x.round(),
        y.round(),
    );
    if (direction != null) nextDirection = direction;

    if (nextDirection != currentDirection &&
        !getGameMap().isWall(x.round(), y.round(),
        nextDirection)) {
        x = x.roundToDouble();
        y = y.roundToDouble();
        currentDirection = nextDirection;
    }
    if (getGameMap().isWall(x.round(), y.round(),
        currentDirection)) {
        x = x.roundToDouble();
        y = y.roundToDouble();
        return;
    }
    stepForward(dt);
}

/// Moves the ghost randomly while in frightened mode.
///
/// Reverses direction once on entry, then re-randomises on
wall hits.
void frightened(double dt) {
```



```
if (!hasStartedFrightenedMode) {
    nextDirection = currentDirection.opposite();
    x = x.roundToDouble();
    y = y.roundToDouble();
    if (!getGameMap().isWall(x.round(), y.round(),
nextDirection)) {
        currentDirection = nextDirection;
    }
    hasStartedFrightenedMode = true;
    if (getGameMap().isWall(x.round(), y.round(),
currentDirection)) {
        return frightened(dt);
    }
} else if (dt % 1 == 0 ||
    getGameMap().isWall(x.round(), y.round(),
currentDirection)) {
    nextDirection = Direction.values[_random.nextInt(4)];
    if (getGameMap().isWall(x.round(), y.round(),
nextDirection)) {
        frightened(dt);
        return;
    }
    currentDirection = nextDirection;
    x = x.roundToDouble();
    y = y.roundToDouble();
}
stepForward(dt);
}

/// Moves the ghost back to the ghost-house entrance
(eyes-only).
void eaten(double dt) {
    hasStartedFrightenedMode = false;
```

```
final direction = ghostChase(
    getGameMap().ghostHouseX, getGameMap().ghostHouseY,
    x.round(), y.round(),
);
if (direction != null) nextDirection = direction;

if (nextDirection != currentDirection &&
    !getGameMap().isWall(x.round(), y.round(),
nextDirection)) {
    currentDirection = nextDirection;
    x = x.roundToDouble();
    y = y.roundToDouble();
}
if (getGameMap().isWall(x.round(), y.round(),
currentDirection)) {
    x = x.roundToDouble();
    y = y.roundToDouble();
    return;
}
stepForward(dt);
}

// — State transition helper



---



/// Checks proximity to Pac-Man / ghost house and triggers
state changes.
void updateState() {
    // Return to chase after re-entering the ghost house.
    if (state == GhostState.eaten &&
        getGameMap().map[y.round().clamp(0,
getGameMap().map.length)][
            x.round().clamp(0, getGameMap().map.first.length)]
    ==
```

```
        GameMap.kGhostHouseInterior) {
    state = GhostState.chase;
}

// Collide with Pac-Man.
if (dist(getPacman().x - x, getPacman().y - y) <= 0.75) {
    if (state == GhostState.frightened0 || state ==
GhostState.frightened1) {
        state = GhostState.eaten;
        onEaten?.call();
    } else if (state == GhostState.chase || state ==
GhostState.scatter) {
        onDeath?.call();
    }
}
}

// — Movement helper



---



/// Advances (x, y) by [speed] × [dt] in [currentDirection].
void stepForward(double dt) {
    switch (currentDirection) {
        case Direction.up:    y -= speed * dt;
        case Direction.down:  y += speed * dt;
        case Direction.left:  x -= speed * dt;
        case Direction.right: x += speed * dt;
    }
}

// — Pathfinding



---


```

```
/// Chooses the first [Direction] on the shortest path from
(cx, cy) to
/// (tx, ty) using BFS. Returns `null` if already at the
target.
Direction? ghostChase(int tx, int ty, int cx, int cy) {
    return complexChase(tx, ty, cx, cy);
}

/// Greedy single-step: chooses the adjacent tile closest to
(tx, ty).
///
/// Faster than BFS but may get stuck in concave walls.
Available to
/// subclasses or as a debugging fallback.
Direction? simpleChase(int tx, int ty, int cx, int cy) {
    if (tx == cx && ty == cy) return null;

    double minDistance = double.infinity;
    Direction? bestDirection;

    for (final dir in Direction.values) {
        final distance = dist(
            tx - (cx + dir.toDelta().$1),
            ty - (cy + dir.toDelta().$2),
        );
        if (!getGameMap().isWall(cx, cy, dir) && minDistance >
distance) {
            minDistance = distance;
            bestDirection = dir;
        }
    }
    return bestDirection;
}
```

```
/// Full BFS from (cx, cy) toward (tx, ty), with map
wrap-around support.
///
/// Records the *first* direction taken from the origin for
every visited
/// cell, then returns that direction when the target is
reached.
Direction? complexChase(int tx, int ty, int cx, int cy) {
    if (tx == cx && ty == cy) return null;

    final GameMap gameMap = getGameMap();
    final queue = Queue<(int, int)>();
    final firstDir = <int, Direction>{};

    int key(int x, int y) => y * gameMap.kCols + x;

    // Seed BFS with immediate neighbours of the starting cell.
    for (final dir in Direction.values) {
        final nx = (cx + dir.toDelta().$1 + gameMap.kCols) %
gameMap.kCols;
        final ny = (cy + dir.toDelta().$2 + gameMap.kRows) %
gameMap.kRows;
        if (nx < 0 || ny < 0 || nx >= gameMap.kCols || ny >=
gameMap.kRows) continue;
        final nk = key(nx, ny);
        if (firstDir.containsKey(nk) || gameMap.isWall(cx, cy,
dir)) continue;
        firstDir[nk] = dir;
        queue.add((nx, ny));
    }

    firstDir[key(cx, cy)] = Direction.up; // dummy – marks
origin as visited
```

```
while (queue.isNotEmpty()) {
    final (x, y) = queue.removeFirst();

    if (x == tx && y == ty) return firstDir[key(x, y)];

    for (final dir in Direction.values) {
        final nx = (x + dir.toDelta().$1 + gameMap.kCols) %
gameMap.kCols;
        final ny = (y + dir.toDelta().$2 + gameMap.kRows) %
gameMap.kRows;
        if (nx < 0 || ny < 0 || nx >= gameMap.kCols || ny >=
gameMap.kRows) continue;
        final nk = key(nx, ny);
        if (firstDir.containsKey(nk) || gameMap.isWall(x, y,
dir)) continue;
        firstDir[nk] = firstDir[key(x, y)]!;
        queue.add((nx, ny));
    }
}

return null;
}
```

File: /lib/game/ghosts/inky.dart

```
import
'package:pacman_application/game/animation/sprite_animation.dart
';
import 'package:pacman_application/game/direction.dart';
import 'package:pacman_application/game/ghosts/ghost.dart';

/// Inky (the cyan ghost) - the flanker.
///
/// Chase strategy: combines Blinky's position and a point ahead
of Pac-Man
/// to produce a flanking target.
///
/// Algorithm:
/// 1. Find a "pivot" up to 4 tiles ahead of Pac-Man
(wall-limited).
/// 2. Draw a vector from Blinky's position to the pivot.
/// 3. Double that vector to get Inky's target tile.
///
/// Scatter corner: **bottom-right** of the maze.
class Inky extends Ghost {
  /// Creates [Inky] and registers optional death/eaten
  callbacks.
  Inky({required super.gameManager, super.onDeath,
  super.onEaten});

  // — Identity

  @override
  String name = "Inky";
```

```
// — Position
```

```
@override  
double x = 12;
```

```
@override  
double y = 14;
```

```
@override  
(int x, int y) startingPosition = (12, 14);
```

```
// — Direction
```

```
@override  
Direction currentDirection = Direction.left;
```

```
@override  
Direction nextDirection = Direction.left;
```

```
// — Animation
```

```
@override  
SpriteAnimation get idleAnimation => left;
```

```
// — Scatter corner
```

```
@override  
late (int, int) scatterLocation = (  
    getGameMap().map.first.length - 1,
```



```
    getGameMap().map.length - 1,
);

// — Blinky position feed (static, updated by GameManager
each tick) —————

    /// Blinky's most recent tile position, injected by
    [GameManager] every tick.
    ///
    /// Kept static so [Inky] can read Blinky's position without a
    direct
    /// reference to the [Blinky] instance.
    static var blinkyPosition = (0.0, 0.0);

    /// Updates [blinkyPosition] with the current tile coordinates
    of Blinky.
    static void setBlinkyPosition(double x, double y) {
        blinkyPosition = (x, y);
    }

// — Chase behaviour



---



    /// Targets a tile computed from Blinky's position and
    Pac-Man's heading.
    @override
    void chase(
        double dt,
        double pacmanX,
        double pacmanY,
        Direction pacmanDirection,
    ) {
        // Find the pivot: up to 4 tiles ahead of Pac-Man.
        int steps;
```

```
for (steps = 0; steps < 4; steps++) {
    if (getGameMap().isWall(
        x.round() + steps * pacmanDirection.toDelta().$1,
        y.round() + steps * pacmanDirection.toDelta().$2,
        pacmanDirection,
    )) {
        break;
    }
}

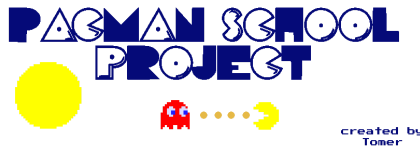
final pivot = (
    pacmanX.round() + steps * pacmanDirection.toDelta().$1,
    pacmanY.round() + steps * pacmanDirection.toDelta().$2,
);

// Target = 2 × (pivot - blinky), clamped to the map bounds.
final target = (
    (2 * pivot.$1 - blinkyPosition.$1)
        .clamp(1, getGameMap().kCols - 1)
        .toInt(),
    (2 * pivot.$2 - blinkyPosition.$2)
        .clamp(1, getGameMap().kRows - 1)
        .toInt(),
);

final direction = ghostChase(target.$1, target.$2,
x.round(), y.round());
if (direction != null) nextDirection = direction;

if (nextDirection != currentDirection &&
    !getGameMap().isWall(x.round(), y.round(),
nextDirection)) {
    currentDirection = nextDirection;
    x = x.roundToDouble();
}
```

Pacman Android Edition



תומר כץ

```
        y = y.roundToDouble();
    }

    if (getGameMap().isWall(x.round(), y.round(),
currentDirection)) {
        x = x.roundToDouble();
        y = y.roundToDouble();
        return;
    }

    stepForward(dt);
}
}
```

File: /lib/game/ghosts/pinky.dart

```
import
'package:pacman_application/game/animation/sprite_animation.dart
';
import 'package:pacman_application/game/direction.dart';
import 'package:pacman_application/game/ghosts/ghost.dart';

/// Pinky (the pink ghost) – the ambusher.
///
/// Chase strategy: targets the tile **4 steps ahead** of
Pac-Man's current
/// direction (stopping earlier if a wall is in the way),
attempting to
/// cut Pac-Man off at the front.
/// Scatter corner: **top-left** of the maze.
class Pinky extends Ghost {
  /// Creates [Pinky] and registers optional death/eaten
callbacks.
  Pinky({required super.gameManager, super.onDeath,
super.onEaten});

  // — Identity

  @override
  String name = "Pinky";

  // — Position

  @override
```

```
double x = 13;
```

```
@override
```

```
double y = 14;
```

```
@override
```

```
(int x, int y) startPosition = (13, 14);
```

```
// — Direction
```

```
@override
```

```
Direction currentDirection = Direction.left;
```

```
@override
```

```
Direction nextDirection = Direction.left;
```

```
// — Animation
```

```
@override
```

```
SpriteAnimation get idleAnimation => down;
```

```
// — Scatter corner
```

```
@override
```

```
(int, int) scatterLocation = (1, 1);
```

```
// — Chase behaviour
```

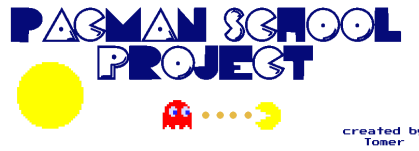
```
/// Targets the tile up to 4 steps ahead of Pac-Man's facing  
direction.
```

```
///
/// Walks forward along Pac-Man's direction until a wall is
hit or 4 tiles
/// have been counted, then uses that tile as the BFS target.
@Override
void chase(
    double dt,
    double pacmanX,
    double pacmanY,
    Direction pacmanDirection,
) {
    hasStartedFrightenedMode = false;

    // Find how far ahead Pac-Man's path is clear (up to 4
tiles).
    int steps;
    for (steps = 0; steps < 4; steps++) {
        if (getGameMap().isWall(
            pacmanX.round() + steps * pacmanDirection.toDelta().$1,
            pacmanY.round() + steps * pacmanDirection.toDelta().$2,
            pacmanDirection,
        )) {
            break;
        }
    }

    final direction = ghostChase(
        pacmanX.round() + steps * pacmanDirection.toDelta().$1,
        pacmanY.round() + steps * pacmanDirection.toDelta().$2,
        x.round(),
        y.round(),
    );
    if (direction != null) nextDirection = direction;
```

Pacman Android Edition



תומר כץ

```
        if (nextDirection != currentDirection &&
            !getGameMap().isWall(x.round(), y.round(),
nextDirection)) {
            currentDirection = nextDirection;
            x = x.roundToDouble();
            y = y.roundToDouble();
        }

        if (getGameMap().isWall(x.round(), y.round(),
currentDirection)) {
            x = x.roundToDouble();
            y = y.roundToDouble();
            return;
        }

        stepForward(dt);
    }
}
```

File: /lib/game/controller/controller.dart

```
import 'package:flutter/material.dart';
import
'package:pacman_application/game/controller/dpad_button.dart';
import 'package:pacman_application/game/direction.dart';

/// An on-screen D-pad controller rendered as four circular
[DpadButton]s.
///
/// Calls [onDirectionChanged] with the appropriate [Direction]
whenever a
/// button is pressed (and continues calling it repeatedly while
held, since
/// [DpadButton] has built-in repeat behaviour).
class Controller extends StatefulWidget {
  const Controller({
    super.key,
    required this.onDirectionChanged,
  });

  /// Called with the new [Direction] each time a D-pad button
is activated.
  final void Function(Direction direction) onDirectionChanged;

  @override
  State<Controller> createState() => _ControllerState();
}

class _ControllerState extends State<Controller> {
  @override
  Widget build(BuildContext context) {
```



```
// Button size scales with screen width, clamped to a
sensible range.
final buttonSize =
    (MediaQuery.of(context).size.width * 0.22).clamp(60,
85).toDouble();
return SizedBox(
    height: buttonSize * 3 + 8,
    child: _buildDpad(buttonSize),
);
}

/// Lays out the four direction buttons in a cross / plus
shape.
Widget _buildDpad(double buttonSize) => Stack(
    alignment: Alignment.center,
    children: [
        // Up button – top centre.
        Positioned(
            top: 0,
            child: DpadButton(
                size: buttonSize,
                icon: Icons.arrow_drop_up,
                onPressed: () =>
widget.onDirectionChanged(Direction.up),
            ),
        ),
        // Down button – bottom centre.
        Positioned(
            top: buttonSize + 10,
            child: DpadButton(
                size: buttonSize,
                icon: Icons.arrow_drop_down,
                onPressed: () =>
widget.onDirectionChanged(Direction.down),
```

```
    ),  
    ),  
    // Left button - middle left.  
    Positioned(  
      top: buttonSize + 4,  
      left: MediaQuery.of(context).size.width / 2 - buttonSize  
* 1.5 - 10,  
      child: DpadButton(  
        size: buttonSize,  
        icon: Icons.arrow_left,  
        onPressed: () =>  
widget.onDirectionChanged(Direction.left),  
      ),  
    ),  
    // Right button - middle right.  
    Positioned(  
      top: buttonSize + 4,  
      left: MediaQuery.of(context).size.width / 2 + buttonSize  
* 0.5 + 10,  
      child: DpadButton(  
        size: buttonSize,  
        icon: Icons.arrow_right,  
        onPressed: () =>  
widget.onDirectionChanged(Direction.right),  
      ),  
    ),  
  ],  
);  
}
```

File: /lib/game/controller/dpad_button.dart

```
import 'dart:async';

import 'package:flutter/material.dart';

/// A single circular D-pad button that fires repeatedly while
held.
///
/// On press-down [onPressed] fires immediately, then repeats
every 80 ms
/// until the finger is lifted. Visual feedback (colour +
border change) is
/// provided via an [AnimatedContainer].
class DpadButton extends StatefulWidget {
  const DpadButton({
    super.key,
    required this.size,
    required this.icon,
    required this.onPressed,
  });

  /// Diameter of the circular button in logical pixels.
  final double size;

  /// Icon displayed in the centre of the button.
  final IconData icon;

  /// Called immediately on press and then every 80 ms while
held.
  final VoidCallback onPressed;
```

```
@override
State<DpadButton> createState() => _DpadButtonState();
}

class _DpadButtonState extends State<DpadButton> {
  // — State

  bool _isPressed = false;
  Timer? _repeatTimer;

  // — Gesture handlers

  void _startPress() {
    setState(() => _isPressed = true);
    widget.onPressed();
    _repeatTimer = Timer.periodic(
      const Duration(milliseconds: 80),
      (_) => widget.onPressed(),
    );
  }

  void _endPress() {
    setState(() => _isPressed = false);
    _repeatTimer?.cancel();
    _repeatTimer = null;
  }

  // — Lifecycle

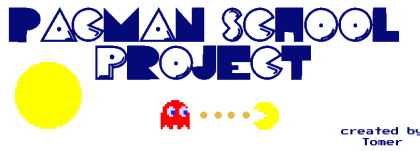
  @override
```

```
void dispose() {  
    _repeatTimer?.cancel();  
    super.dispose();  
}
```

```
// — Widget
```

```
@override  
Widget build(BuildContext context) => GestureDetector(  
    onTapDown: (_) => _startPress(),  
    onTapUp: (_) => _endPress(),  
    onTapCancel: () => _endPress(),  
    child: AnimatedContainer(  
        duration: const Duration(milliseconds: 80),  
        width: widget.size,  
        height: widget.size,  
        decoration: BoxDecoration(  
            shape: BoxShape.circle,  
            color: _isPressed ? Colors.grey[700] : Colors.grey[900],  
            border: Border.all(  
                color: _isPressed ? Colors.amber : Colors.grey[700]!,  
                width: 2,  
            ),  
            boxShadow: _isPressed  
                ? []  
                : const [  
                    BoxShadow(  
                        color: Color(0x7F000000),  
                        offset: Offset(0, 2),  
                        blurRadius: 4,  
                    ),  
                ],  
        ),  
    ),
```

Pacman Android Edition



תומר כץ

```
child: Icon(  
  widget.icon,  
  color: _isPressed ? Colors.amber : Colors.white,  
  size: widget.size * 0.55,  
),  
),  
);  
}
```

File: /lib/game/controller/end_game_buttons.dart

```
import 'package:flutter/material.dart';

/// A pair of styled buttons shown at the end of the game:
/// **RETRY** and **EXIT**.
///
/// Each button uses an [AnimatedContainer] to provide a
/// pressed-state visual
/// effect. [onRetry] and [onExit] are called as soon as the
/// finger touches
/// the button (not on release), matching arcade game feel.
class EndGameButtons extends StatefulWidget {
  const EndGameButtons({
    super.key,
    required this.size,
    required this.onRetry,
    required this.onExit,
  });

  /// Height of each button in logical pixels.
  final double size;

  /// Called when the RETRY button is tapped.
  final VoidCallback onRetry;

  /// Called when the EXIT button is tapped.
  final VoidCallback onExit;

  @override
  State<EndGameButtons> createState() => _EndGameButtonsState();
}
```

```
class _EndGameButtonsState extends State<EndGameButtons> {  
  // — State  
  
  —  
  
  bool _isRetryPressed = false;  
  bool _isExitPressed = false;  
  
  // — Gesture handlers  
  
  void _startRetryPress() {  
    setState(() => _isRetryPressed = true);  
    widget.onRetry();  
  }  
  
  void _endRetryPress() => setState(() => _isRetryPressed =  
false);  
  
  void _startExitPress() {  
    setState(() => _isExitPressed = true);  
    widget.onExit();  
  }  
  
  void _endExitPress() => setState(() => _isExitPressed =  
false);  
  
  // — Widget  
  
  @override  
  Widget build(BuildContext context) => Column(  
    crossAxisAlignment: CrossAxisAlignment.start,
```



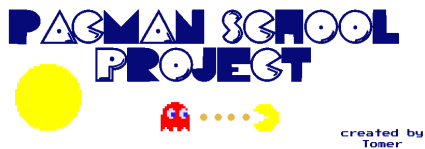
```
mainAxisAlignment: MainAxisAlignment.start,
children: [
  _buildButton(
    label: "RETRY",
    isPressed: _isRetryPressed,
    onStart: _startRetryPress,
    onEnd: _endRetryPress,
    bottomMargin: 10,
  ),
  _buildButton(
    label: "EXIT",
    isPressed: _isExitPressed,
    onStart: _startExitPress,
    onEnd: _endExitPress,
    bottomMargin: 0,
  ),
],
);

// — Private helpers

/// Builds a single labelled button with pressed-state
animation.
Widget _buildButton({
  required String label,
  required bool isPressed,
  required VoidCallback onStart,
  required VoidCallback onEnd,
  required double bottomMargin,
}) {
  return GestureDetector(
    onTapDown: (_) => onStart(),
    onTapUp: (_) => onEnd(),
    onTapCancel: onEnd,
```

```
child: AnimatedContainer(  
  margin: EdgeInsets.fromLTRB(20, 10, 20, bottomMargin),  
  duration: const Duration(milliseconds: 80),  
  height: widget.size,  
  decoration: BoxDecoration(  
    shape: BoxShape.rectangle,  
    color: isPressed ? Colors.red[300] : Colors.red[900],  
    border: Border.all(  
      color: isPressed ? Colors.amber : Colors.grey[700]!,  
      width: 2,  
    ),  
    borderRadius: BorderRadius.circular(20),  
    boxShadow: isPressed  
      ? []  
      : const [  
        BoxShadow(  
          color: Color(0x7F000000),  
          offset: Offset(0, 2),  
          blurRadius: 4,  
        ),  
      ],  
  ),  
  child: Center(  
    child: Text(  
      label,  
      style: TextStyle(  
        fontSize: 40,  
        fontFamily: "PressStart",  
        fontWeight: FontWeight.bold,  
        color: Colors.blue[900],  
      ),  
    ),  
  ),  
),
```

Pacman Android Edition



תומר כץ

```
);  
}  
}
```

File: /lib/game/animation/animated_character.dart

```
import 'package:flutter/material.dart';
import
'package:pacman_application/game/animation/sprite_animation.dart
';
import 'package:pacman_application/game/direction.dart';
import 'package:pacman_application/game/game_manager.dart';
import 'package:pacman_application/game/map/game_map.dart';
import 'package:pacman_application/game/game_screen.dart';
import 'package:pacman_application/game/pacman.dart';

/// Base class for every character that moves and animates on
the game map.
///
/// Subclasses must implement:
/// - [name]           - unique identifier string.
/// - [x] / [y]        - current tile-space position.
/// - [currentDirection] / [nextDirection] - movement state.
/// - [move]           - positional update logic.
/// - [updateAnimation] - animation selection logic.
/// - [reset]          - restore initial state.
abstract class AnimatedCharacter {
  /// Creates an [AnimatedCharacter] bound to the given
  [gameManager].
  AnimatedCharacter({required this.gameManager});

  // — Required identity

  /// Unique display/asset name for this character (e.g.
  `"Pacman"`, `"Blinky"`).
```

```
abstract final String name;
```

```
// — Position & direction (mutable)
```

```
/// Current horizontal position in tile units.  
abstract double x;
```

```
/// Current vertical position in tile units.  
abstract double y;
```

```
/// The direction this character is currently moving.  
abstract Direction currentDirection;
```

```
/// The direction queued for the next opportunity to turn.  
abstract Direction nextDirection;
```

```
// — Animation state
```

```
/// All [SpriteAnimation] instances owned by this character.  
late List<SpriteAnimation> animations = [];
```

```
/// The animation that is currently being rendered.  
late SpriteAnimation currentAnimation;
```

```
// — Dependencies
```

```
/// Reference to the central [GameManager].  
final GameManager gameManager;
```

```
// — Public API
```

```
/// Returns a [Positioned] widget showing the current
animation frame.
///
/// Pass [color] to tint the sprite (used for ghost / Pac-Man
customisation).
Widget getSprite(double tileSize, {Color? color}) =>
Positioned(
  left: x * tileSize,
  top: y * tileSize,
  child: currentAnimation.getImage(color: color),
);

/// Advances movement and animation by [dt] seconds.
void update(double dt) {
  move(dt);
  updateAnimation(dt);
}

/// Restores the character to its initial position and state.
void reset();

// — Abstract callbacks



---



/// Updates [x] and [y] based on [currentDirection] and
game-map collision.
void move(double dt);

/// Selects [currentAnimation] based on current state and
advances its clock.
void updateAnimation(double dt);
```

```
// — Convenience accessors
```

```
/// Shortcut to the shared [GameMap].
GameMap getGameMap() => gameManager.gameMap;

/// Shortcut to the currently active [GameScreen].
GameScreen getGameScreen() => gameManager.currentScreen;

/// Shortcut to the [Pacman] instance managed by
[gameManager].
Pacman getPacman() => gameManager.pacman;
}
```

File: /lib/game/animation/sprite_animation.dart

```
import 'package:flutter/material.dart';

/// Base path under which all character sprite sheets are
stored.
const String _kAssetsPath = "assets/game/characters/";

/// File extension shared by all sprite images.
const String _kSpriteFileExtension = ".png";

/// Manages a single named sprite animation for a game
character.
///
/// Sprites are loaded from:
///
/// `assets/game/characters/<characterName>/<animationName>/<characterName>_<animationName>_<index>.png`
///
/// Call [update] every frame to advance the animation, and
[idle] to reset it.
class SpriteAnimation {
  /// Creates a [SpriteAnimation] and builds the asset-path list
  for every frame.
  ///
  /// [characterName] - folder name of the character
  (case-insensitive on disk).
  /// [animationName] - sub-folder / state name (e.g. "left",
  "death").
  /// [spritesAmount] - total number of frames in the
  animation.
```



```
/// [isLoop]          - whether the animation wraps back to
frame 0 after finishing.
/// [frameSpeed]      - seconds each frame is displayed
(default  $\approx 8$  fps).
SpriteAnimation({
    required this.characterName,
    required this.animationName,
    required this.spritesAmount,
    this.isLoop = false,
    this.frameSpeed = 1 / 8,
}) {
    for (int i = 0; i < spritesAmount; i++) {
        _paths.add(

"$_kAssetsPath${characterName.toLowerCase()}/$animationName/${ch
aracterName.toLowerCase()}_$animationName_$i$_kSpriteFileExten
sion",
        );
    }
}

// — Identity

/// Name of the character this animation belongs to (e.g.
"Pacman", "Blinky").
final String characterName;

/// Name of the animation state (e.g. "left", "frightened0",
"death").
final String animationName;

/// Total number of frames in this animation.
final int spritesAmount;
```

```
/// Seconds each frame is held before advancing. Defaults to
1/8 s (8 fps).
final double frameSpeed;

/// Whether the animation loops continuously (`true`) or plays
once (`false`).
final bool isLoop;

// — State


---



/// Resolved asset paths for every frame, built in the
constructor.
final List<String> _paths = [];

/// Accumulator used to determine when to advance to the next
frame.
double _clock = 0;

/// Zero-based index of the frame currently being displayed.
int _currentIndex = 0;

/// Whether [update] has been called at least once since the
last [idle].
bool _started = false;

/// `true` once a non-looping animation has shown its last
frame.
bool isDone = false;

/// When `true`, [update] has no effect and the current frame
is held.
```

```
bool isPaused = false;
```

```
// — Public API
```

```
/// Returns the [Image] widget for the currently active frame.
///
/// Pass [color] to tint the image (useful for ghost colour
customisation).
Image getImage({Color? color}) {
    return Image.asset(_paths[_currentIndex], color: color);
}

/// Advances the animation by [dt] seconds.
///
/// Has no effect when [isPaused] is `true` or [isDone] is
`true`.
void update(double dt) {
    if (isPaused) return;

    _clock += dt;

    if (isDone) return;

    if (!_started) {
        _started = true;
    }

    while (_clock >= frameSpeed) {
        if (_currentIndex + 1 == spritesAmount) {
            if (isLoop) {
                _clock -= frameSpeed;
                _currentIndex = 0;
            } else {
```

```
        isDone = true;
        return;
    }
} else {
    _clock -= frameSpeed;
    _currentIndex++;
}
}
}

/// Resets the animation to its first frame and clears all
timing state.
///
/// Called on animations that are not the active one so they
are ready to play
/// from the beginning the next time they are selected.
void idle() {
    _started = false;
    isDone = false;
    _clock = 0;
    _currentIndex = 0;
}
}
```

File: /lib/database/firebase_options.dart

```
import 'package:firebase_core/firebase_core.dart' show
FirebaseOptions;
import 'package:flutter/foundation.dart'
    show defaultTargetPlatform, kIsWeb, TargetPlatform;

/// Provides the correct [FirebaseOptions] for the current
runtime platform.
///
/// Usage:
/// ```dart
/// import 'firebase_options.dart';
///
/// await Firebase.initializeApp(
///   options: DefaultFirebaseOptions.currentPlatform,
/// );
/// ```
class DefaultFirebaseOptions {
  //
  -----
  // Platform configurations
  //
  -----

  /// Firebase options for Android builds.
  static const FirebaseOptions android = FirebaseOptions(
    apiKey: 'AIzaSyA-i3m8tWc91L3hzah7_vnD1_S1i64DKG8',
    appId: '1:391973598755:android:c6f9cb7212f42c115aa396',
    messagingSenderId: '391973598755',
```

```
projectId: 'pacman-school',
databaseURL:

'https://pacman-school-default-rtdb.europe-west1.firebaseio.com',
  storageBucket: 'pacman-school.firebaseio.com',
);

/// Firebase options for iOS builds.
static const FirebaseOptions ios = FirebaseOptions(
  apiKey: 'AIzaSyCvNmV5jLcj-cn0QncdPhnxhHQgJg0H0Vs',
  appId: '1:391973598755:ios:b0e1a292883162275aa396',
  messagingSenderId: '391973598755',
  projectId: 'pacman-school',
  databaseURL:

'https://pacman-school-default-rtdb.europe-west1.firebaseio.com',
  storageBucket: 'pacman-school.firebaseio.com',
  iosBundleId: 'com.example.pacmanApplication',
);

/// Firebase options for macOS builds.
///
/// Shares the same credentials as [ios] because macOS and iOS
targets use
/// the same Apple bundle.
static const FirebaseOptions macos = FirebaseOptions(
  apiKey: 'AIzaSyCvNmV5jLcj-cn0QncdPhnxhHQgJg0H0Vs',
  appId: '1:391973598755:ios:b0e1a292883162275aa396',
  messagingSenderId: '391973598755',
  projectId: 'pacman-school',
  databaseURL:
```

```
'https://pacman-school-default-rtdb.europe-west1.firebaseiobas
e.app',
  storageBucket: 'pacman-school.firebaseiostorage.app',
  iosBundleId: 'com.example.pacmanApplication',
);

/// Firebase options for Web builds.
static const FirebaseOptions web = FirebaseOptions(
  apiKey: 'AIzaSyDc6ve6jeS5cNAiyZTx6M_m6eyDkTqTjfI',
  appId: '1:391973598755:web:e9cc61a4040b8b0a5aa396',
  messagingSenderId: '391973598755',
  projectId: 'pacman-school',
  authDomain: 'pacman-school.firebaseio.com',
  databaseURL:

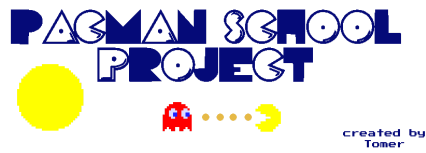
'https://pacman-school-default-rtdb.europe-west1.firebaseiobas
e.app',
  storageBucket: 'pacman-school.firebaseiostorage.app',
);

/// Firebase options for Windows builds.
static const FirebaseOptions windows = FirebaseOptions(
  apiKey: 'AIzaSyDc6ve6jeS5cNAiyZTx6M_m6eyDkTqTjfI',
  appId: '1:391973598755:web:a57414f7160afb765aa396',
  messagingSenderId: '391973598755',
  projectId: 'pacman-school',
  authDomain: 'pacman-school.firebaseio.com',
  databaseURL:

'https://pacman-school-default-rtdb.europe-west1.firebaseiobas
e.app',
  storageBucket: 'pacman-school.firebaseiostorage.app',
);
```

```
//  
-----  
-----  
// Platform resolver  
//  
-----  
-----  
  
/// Returns the [FirebaseOptions] that match the current  
runtime platform.  
///  
/// Throws an [UnsupportedError] for Linux or any unrecognized  
platform,  
/// because FlutterFire has not been configured for those  
targets.  
static FirebaseOptions get currentPlatform {  
  if (kIsWeb) return web;  
  
  switch (defaultTargetPlatform) {  
    case TargetPlatform.android:  
      return android;  
    case TargetPlatform.iOS:  
      return ios;  
    case TargetPlatform.macOS:  
      return macos;  
    case TargetPlatform.windows:  
      return windows;  
    case TargetPlatform.linux:  
      throw UnsupportedError(  
        'DefaultFirebaseOptions have not been configured for  
Linux. '  
        'Re-run the FlutterFire CLI to add Linux support.',  
      );  
  }  
}
```


Pacman Android Edition



תומר כץ

```
default:
    throw UnsupportedError(
        'DefaultFirebaseOptions are not supported for this
platform.',
    );
}
}
}
```

File: /lib/database/game_user.dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_database/firebase_database.dart';
import 'package:flutter/foundation.dart';
import 'package:pacman_application/utils/constants.dart';
import 'package:pacman_application/utils/util.dart';
import 'package:pacman_application/utils/bonus_type.dart';

/// Represents a registered (or anonymous) player stored in
/// Firebase
/// Realtime Database under the path `users/<uid>`.
///
/// The class mirrors the following database schema:
/// ```
/// users/
///   <uid>/
///     name      : String
///     email     : String
///     highScore : int
///     bonusTable/
///       apple   : int
///       bell    : int
///       ...
/// ```
///
/// Use the default constructor when you already have all field
/// values
/// (e.g. after reading from SharedPreferences). Use
/// [GameUser.fromUid]
/// when you only have a Firebase UID and want the data fetched
/// asynchronously from the database.
```

```
class GameUser {  
    // — Fields  
  
    /// Firebase Auth UID that uniquely identifies this user.  
    final String uid;  
  
    /// Player display name shown in the UI and on the  
    leaderboard.  
    String name;  
  
    /// The player's registered e-mail address.  
    String email;  
  
    /// Maps each [BonusType] to the number of times the player  
    has collected it.  
    Map<BonusType, int> bonusTable;  
  
    /// Backing field for [highScore].  
    int _highScore;  
  
    // — Getters / Setters  
  
    /// The player's all-time best score.  
    int get highScore => _highScore;  
  
    /// Updates [highScore] in memory **and** persists the new  
    value to  
    /// Firebase Realtime Database.  
    set highScore(int score) {  
        _highScore = score;  
        _updateScore(score);  
    }  
}
```

// — Constructors

```
/// Creates a [GameUser] with all fields provided directly.
///
/// [highScore] defaults to [anonymousHighScore] when omitted.
GameUser({
    required this.uid,
    this.name = anonymousName,
    int? highScore,
    this.email = anonymousEmail,
    this.bonusTable = anonymousBonusTable,
}) : _highScore = highScore ?? anonymousHighScore;

/// Creates a [GameUser] by fetching its data from Firebase
using [uid].
///
/// Data is loaded asynchronously via [_setData]; the object
fields will
/// hold their default values until that future completes. Use
/// [haveInitialized] to check whether loading is done.
GameUser.fromUid({required this.uid})
    : name = anonymousName,
      email = anonymousEmail,
      bonusTable = anonymousBonusTable,
      _highScore = anonymousHighScore {
    try {
        final ref = FirebaseDatabase.instance.ref('users/$uid');
        _setData(ref);
    } on FirebaseAuthException catch (e) {
        if (kDebugMode) {
            print('FirebaseAuthException ${e.code} – defaulting
user');
```

```
    }  
    } on FirebaseException catch (e) {  
        if (kDebugMode) {  
            print('FirebaseException ${e.code} - defaulting user');  
        }  
    } catch (e) {  
        if (kDebugMode) {  
            print(e);  
        }  
    }  
}
```

```
// — Public methods
```

```
    /// Returns `true` once the async data fetch initiated by  
    [GameUser.fromUid]  
    /// has populated [name] with a real value (i.e. it is no  
    longer the  
    /// anonymous placeholder).  
    bool haveInitialized() => name != anonymousName;  
  
    /// Increments the collected count for [bonus] in memory and  
    persists the  
    /// update to Firebase Realtime Database.  
    void addBonus(BonusType bonus) {  
        bonusTable[bonus] = bonusTable[bonus]! + 1;  
        _updateBonusTable(bonus);  
    }
```

```
// — Private methods
```

```
/// Writes the new [score] to `users/<uid>/highScore` in the  
database.
```

```
void _updateScore(int score) async {  
    final ref = FirebaseDatabase.instance.ref('users/$uid');  
    await ref.update({'highScore': score});  
}
```

```
/// Writes the updated count for [bonus] to  
`users/<uid>/bonusTable/<name>`.
```

```
void _updateBonusTable(BonusType bonus) async {  
    final ref =  
FirebaseDatabase.instance.ref('users/$uid/bonusTable');  
    await ref.update({'bonus.name': bonusTable[bonus]});  
}
```

```
/// Fetches all user fields from [ref] and populates this  
object's
```

```
/// properties. Called once during [GameUser.fromUid]  
construction.
```

```
void _setData(DatabaseReference ref) async {  
    var event = await ref.child('name').once();  
    name = tryCast(event.snapshot.value) ?? '';  
  
    event = await ref.child('highScore').once();  
    _highScore = tryCast(event.snapshot.value) ?? 0;  
  
    event = await ref.child('email').once();  
    email = tryCast(event.snapshot.value) ?? '';  
  
    final entries = await Future.wait(  
        BonusType.values.map((bonus) => _getBonusEntry(bonus,  
ref)),  
    );  
    bonusTable = <BonusType, int>{}..addEntries(entries);
```

```
}

    /// Reads the stored count for a single [bonus] from the
    database and
    /// returns it as a [MapEntry] suitable for building
    [bonusTable].
    Future<MapEntry<BonusType, int>> _getBonusEntry(
        BonusType bonus,
        DatabaseReference ref,
    ) async {
        final event = await
ref.child('bonusTable').child(bonus.name).once();
        return MapEntry(bonus, tryCast(event.snapshot.value) ?? 0);
    }

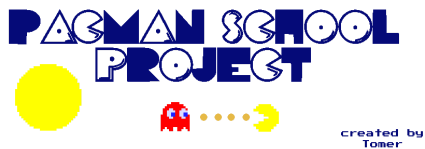
    // — Object overrides

    /// Two [GameUser] instances are considered equal when they
    share the same [uid].
    @override
    bool operator ==(Object other) =>
        identical(this, other) || (other is GameUser && uid ==
other.uid);

    @override
    int get hashCode => uid.hashCode;

    @override
    String toString() => '''
        uid:          $uid
        name:          $name
        high score:    $highScore
        email:         $email
    '''
```

Pacman Android Edition



תומר כץ

```
bonus table: $bonusTable
''';
}
```

File: /lib/database/session.dart

```
import 'package:pacman_application/database/game_user.dart';

/// The [GameUser] object for the player who is currently logged
in.
///
/// Initialised to the anonymous guest (uid ``0``) at app
start-up and
/// replaced once a successful sign-in completes in [Login] or
[Signup].
/// All screens read from this global rather than passing the
user down the
/// widget tree, keeping the session state centrally accessible.
GameUser currentUser = GameUser(uid: '0');

/// Whether the current session is a guest (unauthenticated)
session.
///
/// Set to `true` when the user taps "Enter as a guest" on the
[Login]
/// screen, and `false` after any successful Firebase sign-in.
Screens and
/// game logic gate score-saving and collectable-saving
behaviour behind
/// this flag.
bool isAnonymous = true;
```

מילון מושגים

הסבר	מושג
יחידה תכנותית המכילה נתונים (שדות) והתנהגות (פעולות). בפרויקט: פקמן, רוחות, נקודות ובונוסים הם אובייקטים.	אובייקט
אלגוריתם חיפוש לפי רוחב המשמש לקביעת המסלול הקצר ביותר של הרוחות לעבר פקמן. סורק את המפה שכבה-שכבה מנקודת המוצא.	אלגוריתם BFS
כל רכיב גרפי בממשק Flutter: טקסט, כפתור, תמונה, רשימה וכו'. כל מסך בנוי מהרכבת widgets.	אלמנט תצוגה
החלפה מהירה של תמונות (frames) ליצירת תחושת תנועה. בפרויקט פותחה מחלקת אנימציה עצמאית (SpriteAnimation) מכיוון שFlutter אינה מספקת מנגנון מובנה מסוג זה.	אנימציה
מבנה כולל של הפרויקט: כיצד המחלקות, המסכים ומערכת הנתונים מאורגנים ומתקשרים זה עם זה.	ארכיטקטורה
שירותי Firebase לאימות זהות המשתמש. מנהל הרשמה, כניסה ויציאה מהאפליקציה באמצעות דוא"ל וסיסמה.	אימות משתמשים
פירות וחפצים המופיעים באופן אקראי במרכז המבוך. אכילתם מוסיפה ניקוד. בנוסף, ישנו אתגר לאסוף את כל סוגי הבונוסים.	בונוס
אלגוריתם חכם שמנחה כל רוח לרדוף אחרי פקמן בצורה שונה. מממש לוגיקת מעקב מורכבת (complexChase) המבוססת על BFS.	בינה מלאכותית (של רוחות)
מאגר נתונים בענן של Firebase. מאחסן נתוני משתמשים: שם, שיא, דוא"ל וטבלת בונוסים שנאספו.	בסיס נתונים
ווידג'ט גרפי עגול המוצג במסך הבית. מציג לשחקן את אחוז התקדמותו בסיום איסוף כל הבונוסים.	גרף התקדמות

מושג	הסבר
דיאלוג	חלון קופץ המוצג מעל המסך הנוכחי. בפרויקט: דיאלוג בונוס (לצפייה בפרטי בונוס) ודיאלוג שינוי צבע (בהגדרות).
דירוגים	מסך המציג טבלת דירוגים של כל השחקנים הרשומים, ממוינת לפי ניקוד שיא יורד.
הגדרות	מסך המאפשר למשתמש לשנות את צבעי האלמנטים במשחק. הבחירות נשמרות באמצעות SharedPreferences.
הורשה	מנגנון תכנות מונחה עצמים שבו מחלקה יורשת תכונות ופעולות ממחלקת-אב. לדוגמה: Blinky, Ghost, Pinky, Inky, Clyde יורשים מ-Ghost.
טיימר ספירה לאחור	טיימר המחליף את מצב הרוחות מ-chase ל-scatter כל 5 שניות, ומנהל את מגבלת הזמן של מצב הנבהל.
חיים	מספר הפעמים שנשאר לשחקן "למות" לפני שהמשחק נגמר. מוצג בחלק העליון של מסך המשחק.
כפתור כיוון	כפתור בשלט הדיגיטלי המייצג כיוון תנועה (מעלה / מטה / שמאל / ימין). השלט מכיל 4 כפתורי כיוון.
מבוך	מפת המשחק הדו-מימדית. מיוצגת כמערך של מספרים, כל מספר מסמל סוג תא (קיר, מסלול, נקודה וכו'). מאפשרת מעבר בין הקצוות (מנהרה).
מחלקה	תבנית תכנותית להגדרת אובייקטים עם תכונות ופעולות משותפות. בפרויקט: Ghost, Pacman, GameManager ועוד.
מחלקת Enum	סוג מחלקה המגדיר קבוצת ערכים קבועים ומפורטים. לדוגמה: chase, GhostState, scatter, frightened, eaten, idle, Direction, BonusType.
מנהרה	מנגנון המאפשר ל-Pacman ולרוחות לעבור מקצה אחד של המפה לקצה השני (שמאל-ימין).

מושג	הסבר
מסד נתונים בזמן אמת	שירות Firebase לאחסון וסנכרון נתונים בזמן אמת. מעדכן את ניקוד השיא ואת הבונוסים שנאספו ישירות מהמשחק.
מסך הבית	המסך המרכזי של האפליקציה. מציג שם משתמש, ניקוד שיא, התקדמות בבונוסים וכפתור למשחק מהיר.
מסך הכניסה	מסך ראשוני לאימות משתמש. מאפשר כניסה עם חשבון קיים או כניסה כאורח (אנונימי).
מסך ההרשמה	מסך ליצירת חשבון משתמש חדש. מבצע הרשמה דרך Firebase Authentication.
מסך המשחק	המסך הראשי בו מתרחש המשחק. מכיל את המבוך, דמויות, שלט, ניקוד, חיים ובונוסים.
מצב נבהל	מצב הרוחות לאחר שפקמן אכל נקודת-על. הרוחות הופכות לפגיעות ופקמן יכול לאכול אותן לזמן קצר.
מצב מעקב	מצב פעולה של הרוחות שבו הן רודפות אחרי פקמן. כל רוח מממשת לוגיקת מעקב שונה.
מצב פיזור	מצב שבו הרוחות מתפזרות לפינות המפה (נקודת יעד ייחודית לכל רוח). מתחלף עם מצב המעקב כל 5~ שניות.
מצב נאכל	מצב שבו הרוח נאכלה על ידי פקמן (לאחר מצב נבהל). הרוח חוזרת למאורה ומתאפסת.
מצב בטלה	מצב ההמתנה של הרוחות לפני תחילת המשחק או בזמן אתחול.
מנהל המשחק	המחלקה המרכזית המתאמת את לוגיקת המשחק: ניהול ניקוד, חיים, בדיקת אכילת נקודות ובונוסים, ותזמון מצבי הרוחות.
מעטפת אפליקציה	נקודת הכניסה הראשית לאפליקציה. מאתחלת את Firebase ומגדירה את מבנה הניווט הכולל.
נקודה	פריט בסיסי שפקמן אוסף במהלך המשחק. קיימים שני סוגים: נקודה רגילה ונקודת-על (Super Dot).

מושג	הסבר
סופר נקודה	נקודה גדולה הממוקמת בפינות המבוך. אכילתה מעבירה את הרוחות למצב נבהל ומאפשרת לפקמן לאכול אותן.
ניקוד	מספר הנקודות שצבר השחקן במהלך המשחק הנוכחי. מוצג בזמן אמת במסך המשחק.
ניווט	מעבר בין המסכים השונים באפליקציה. מתבצע דרך תפריט הצד ותפריט העליון.
סייקל משחק	לולאת עדכון המתרחשת בכל פריים. בודקת אכילת נקודות, בונסים ורוחות, מעדכנת ניקוד וחיים, ומזיזה את הדמויות.
סשן / ישיבת משחק	אובייקט גלובלי השומר את המשתמש המחובר כרגע (currentUser) ודגל אנונימיות (isAnonymous) לאורך כל חיי האפליקציה.
ספריית צד שלישי	חבילת קוד חיצונית. הפרויקט משתמש בספריות Firebase לניהול נתונים ואימות.
עצמי אנונימי	סוג משתמש לא מזוהה הנכנס כאורח. אינו שומר ניקוד שיא או בונסים שנאספו בבסיס הנתונים.
פאקמן (הדמות)	הדמות הראשית הנשלטת על ידי השחקן. משימתה לאסוף נקודות ולברוח מהרוחות.
פאייר-בייס	פלטפורמת פיתוח ענן של Google המספקת שירותי אימות משתמשים ובסיס נתונים בזמן אמת לפרויקט.
פלאטר	סביבת הפיתוח בה נכתב הפרויקט. מסגרת עבודה מבית Google לפיתוח אפליקציות Android/iOS עם שפת Dart.
פרמטר	ערך המועבר לפונקציה בעת קריאה לה. בפרויקט מוגדרים פרמטרים חובה (required) ואופציונליים (nullable).
רוח	ארבע הדמויות האנטגוניסטיות הרודפות אחרי פקמן. לכל רוח אלגוריתם מעקב שונה: Blinky, Pinky, Inky, Clyde.
רינדור	תהליך ציור מסגרות הגרפיקה על המסך. מתבצע בכל עדכון של לולאת המשחק.

מושג	הסבר
רשימה	ווידג'ט Flutter להצגת רשימה גוללת. בפרויקט: תצוגת חיים, בונוסים שנאספו ושורות דירוגים.
שחקן משתמש	מחלקה המייצגת את נתוני המשתמש המחובר: uid, שם, דוא"ל, ניקוד שיא וטבלת בונוסים.
שיא	ניקוד הגבוה ביותר שהשיג השחקן. מוצג במסך הבית ובמסך המשחק, ומתעדכן בבסיס הנתונים בסיום המשחק.
שלט	ממשק שליטה דיגיטלי בחלק התחתון של מסך המשחק. מורכב מ-4 כפתורי כיוון (מעלה, מטה, שמאל, ימין).
שמירת העדפות	אמצעי Flutter לשמירת נתוני העדפות קטנים במכשיר באופן מקומי. בפרויקט: שמירת בחירת הצבעים בהגדרות.
תפריט עליון	רכיב ניווט עליון המוצג בכל מסך. מראה את שם המסך הנוכחי, כפתור התנתקות וכפתור פתיחת תפריט הצד.
תפריט צד	תפריט ניווט הנפתח מהצד. מכיל ברכת שלום למשתמש ורשימת קישורים לכל מסכי האפליקציה.
תרשים UML	שיטה — Unified Modeling Language תרשימית תקנית לתייעוד מבנה המחלקות, קשרי הירושה והתלויות בפרויקט.
תכנות מונחה עצמים	פרדיגמת תכנות המבוססת על מחלקות ואובייקטים. הפרויקט כולו בנוי על עקרונות OOP: הורשה, פולימורפיזם, אנקפסולציה.

מסך סיום

שם הפרויקט:

PACMAN ANDROID EDITION

שם החלופה: תכנות טלפונים חכמים אנדרואיד (Flutter)

The logo for the Pacman School Project. It features the text "PACMAN SCHOOL PROJECT" in a stylized, blocky font. Below the text are three icons: a large yellow circle, a small red ghost, and a small yellow Pacman character.

created by
Tomer

שם התלמיד: תומר כץ

ת.ז התלמיד: 333040616

שם המנחה: שרה סולומון

תאריך הגשה: מאי, 2026